MODULE 00 · EXAM STRATEGY - ALL DOMAINS

SnowPro Core COF-C03

The complete course. Just under ten hours, thirty-one modules, one hundred and eighty-one worked scenarios.

DOMAINS 5 SCENARIOS 181 RUNTIME 9h 57m

admin@sthithapragna.com

Every fact here was **checked against Snowflake's own documentation** while the course was being written, and the check dates are in the package.

Who this is for

- Anyone sitting **SnowPro Core COF-C03**. It is the current version, released February 2026.
- You need **no prior Snowflake experience**. The course builds from the architecture upward.
- It suits people who want to **understand why**, not memorise a list. Most modules derive facts rather than assert them.
- It is **not** a braindump, and it contains no leaked exam questions. Every scenario here was written for this course.

> ON THE EXAM
>
> If you have used Snowflake for years, start with the domain drills in modules 7, 12, 16, 22 and 25 to find your gaps.

Why a new course was needed

- **COF-C03 replaced COF-C02**, which was retired for English in May 2026. The blueprint changed substantially.
- New and examinable: **Cortex AI, Snowpipe Streaming, dynamic tables, Apache Iceberg, data clean rooms**.
- Also new: **Trust Center, Git integration, data lineage, Snowflake CLI, Streamlit, Notebooks**.
- Material written for COF-C02 is missing all of that, and still recommends **SnowSQL**, which Snowflake now documents as legacy.

> ON THE EXAM
>
> Roughly a fifth of this course covers material that simply did not exist on the previous exam.

The exam at a glance

Questions and time	About 100 questions in 115 minutes. Roughly 69 seconds each
Format	Multiple choice and multiple select
Passing standard	A scaled score of 750 out of 1000. Not a raw percentage
Delivery	Pearson VUE, online-proctored or at a test centre
Cost and validity	$175, and the credential is valid for two years

ON THE EXAM

Confirm all of this on Snowflake's own certification page before you book. These details do change.

The five domains

DOMAIN	WEIGHT	MODULES
1. Snowflake AI Data Cloud Features and Architecture	31%	01 to 07
2. Account Management and Data Governance	20%	08 to 12
3. Data Loading, Unloading, and Connectivity	18%	13 to 16
4. Performance Optimization, Querying, and Transformation	21%	17 to 22
5. Data Collaboration	10%	23 to 25

ON THE EXAM

Course time is allocated in proportion to these weights, so Domain 1 gets the most and Domain 5 the least.

How every module is built

- **Teach, then test.** Concepts first, then worked scenarios in the shape the exam actually uses.
- **Every scenario is worked aloud**, including why each wrong option is wrong. That is usually where the learning is.
- **Pause when prompted.** Commit to an answer before I speak, or the scenario teaches you very little.
- **Each module closes with five things to carry forward**, and names what the next module covers.
- **Currency slides** flag where the platform has moved past the exam, and tell you which answer to give.

> ON THE EXAM
>
> 181 scenarios across the course, with the correct answer spread evenly across A, B, C and D so no letter is a safe guess.

Domain 1 - architecture and features, 31%

STARTS AT	MODULE
10:26	01. Three layers, and why everything else follows
30:26	02. Editions, and the 'minimum edition' question
49:03	03. Micro-partitions, pruning and clustering
1:12:19	04. Table types and view types
1:32:54	05. Interfaces, drivers, connectors and Git
1:51:45	06. Cortex, Snowpark, Streamlit, Notebooks and ML
2:17:08	07. Domain 1 drill - eight worked scenarios

ON THE EXAM

Module 1 is the foundation. Almost every later explanation refers back to the three-layer architecture.

Domain 2 - governance, 20%

STARTS AT	MODULE
2:35:17	08. The object hierarchy and the RBAC model
2:56:46	09. Authentication, network policies and encryption
3:15:31	10. Masking, row access, tags, classification and lineage
3:35:14	11. Credits, resource monitors and the usage views
3:53:04	12. Domain 2 drill - six worked scenarios

ON THE EXAM

Module 9 is where this course is most obviously newer than its competitors. User types and MFA enforcement changed the answers.

Domain 3 - loading and connectivity, 18%

STARTS AT	MODULE
4:07:32	13. Stages, file formats and the load path
4:25:38	14. COPY INTO, error handling and unloading
4:42:55	15. Snowpipe, Streaming, streams, tasks and dynamic tables
5:05:44	16. Connectors, integrations, and the Domain 3 drill

> ON THE EXAM
>
> Module 15 is the densest in the course. It is also where the Openflow situation is addressed honestly.

Domain 4 - performance, 21%

STARTS AT	MODULE
5:21:39	17. Warehouse sizing, scaling and what it costs
5:39:34	18. The three caches, and why a query is slow
5:56:07	19. Reading a Query Profile
6:13:53	20. Clustering, materialized views, search optimization, QAS
6:31:04	21. SQL features, semi-structured and unstructured data
6:50:51	22. UDFs, procedures, Snowpark, and the Domain 4 drill

ON THE EXAM

Modules 19 and 20 are the pair most likely to move a borderline score. Read a profile, then pick the right fix.

Domain 5 and the final section

STARTS AT	MODULE
7:08:37	23. Time Travel, Fail-safe and zero-copy cloning
7:24:47	24. Secure sharing, listings, clean rooms and native apps
7:40:20	25. Replication, failover, and the Domain 5 drill
7:55:07	26. Confusion pairs - twenty things that look alike
8:18:19	27. How Snowflake writes questions
8:34:39	28. Practice exam 1 - twenty questions
9:01:01	29. Practice exam 2 - twenty questions
9:28:58	30. Practice exam 3 - twenty questions

ON THE EXAM

The three exams test 60 distinct topics with no repeats, and cover every teaching module in the course.

How to use this course

- **Watch in order the first time.** Later modules assume earlier ones, and the callbacks are deliberate.
- **Sit each practice exam in one timed session**, then read every explanation, including for questions you got right.
- **Score by domain, not by total.** A domain you miss in two exams is the one to rewatch.
- **The day before: module 26, then module 27.** Learn nothing new the day before an exam.
- Watching at 1.25 times speed is fine. **Do not skip the pauses.**

> ON THE EXAM
>
> Roughly a third of the runtime is worked scenarios. That is the part that transfers to the exam, so do not skip it.

What to have alongside

- **A Snowflake trial account.** Thirty days and free credits. Running the commands is worth more than watching them.
- **The official study guide PDF** from Snowflake's certification page. It is the authoritative blueprint.
- **The docs.** When this course and the documentation disagree, the documentation is right and the course is out of date.
- **Check the exam format before booking.** Question count, duration and fee are all subject to change.

> ON THE EXAM
>
> The VERIFIED file shipped with this course lists every fact checked and the date it was checked, so you can re-verify anything.

The exam is not testing whether you have memorised Snowflake. It is testing whether you can **recognise which fact a question is about**.

MODULE 01 · DOMAIN 1 - SNOWFLAKE AI DATA CLOUD FEATURES AND ARCHITECTURE

Three layers, and why everything else follows

The one architectural decision that explains cloning, sharing, elastic scale and per-second billing.

DOMAIN 1 of 5 WEIGHT 31% MODULE 1 of 31

admin@sthithapragna.com

In a traditional warehouse the data lives on the machine that queries it. Snowflake broke that link.

admin@sthithapragna.com

Snowflake is a self-managed service

What the documentation means by that, and what it rules out

- **No hardware** to select, install, configure or manage. Virtual or physical.
- **Virtually no software** to install or manage. Maintenance, upgrades and tuning are Snowflake's job.
- You **cannot** run Snowflake locally, on-premises, or on private cloud infrastructure.
- It runs on public cloud infrastructure: **AWS, Azure or Google Cloud**. You choose the platform and region.

> ON THE EXAM
>
> The last two points are a question waiting to happen. 'Deploy Snowflake in our own data centre' is never the answer.

The three layers, bottom to top

01

Database storage

Data, reorganised into compressed columnar micro-partitions. Snowflake manages all of it.

02

Compute

Virtual warehouses. Independent MPP clusters that read the storage layer.

03

Cloud services

The coordinator. Authentication, metadata, optimisation, access control.

ON THE EXAM

Learn these names exactly. 'Which layer is responsible for X' is one of the most reliable question formats on the paper.

What actually lives in each layer

LAYER	RESPONSIBILITIES
Storage	Your table data. Micro-partitions, compression, file size, structure, statistics. Not directly visible to you.
Compute	Query execution. Loading, DML, Snowpark code in Java, Python and Scala.
Cloud services	Authentication and access control. Metadata. Query parsing and optimisation. Infrastructure management. Horizon Catalog. Compliance.

ON THE EXAM

The result cache and the metadata cache both live in cloud services. That is why some queries return with no warehouse running.

Where the design came from

SHARED-DISK

One central store, many nodes

- All nodes reach the same data
- Simple to manage: one copy of the truth
- Nodes contend for the same store
- Scaling hits a ceiling

SHARED-NOTHING

Each node owns its slice

- Data partitioned across nodes
- Massively parallel, scales out well
- Adding a node means reshuffling data
- Storage and compute must grow together

HOW SNOWFLAKE RESOLVES IT

Snowflake is a hybrid: the management simplicity of shared-disk with the scale-out performance of shared-nothing.

Every feature you will study in domains two to five is a **receipt** for this one decision.

admin@sthithapragna.com

Four features that are really one idea

- **Zero-copy cloning.** A clone points at existing micro-partitions. No data is duplicated, so no storage is charged until it diverges.
- **Secure Data Sharing.** No data is copied to the consumer. They read your storage using their own compute.
- **Workload isolation.** Each warehouse is independent and has no effect on the performance of any other.
- **Elastic, per-second compute.** A warehouse holds no data, so suspending it loses nothing but the local cache.

> ON THE EXAM
>
> When a question describes one of these, the reasoning behind the right answer is always separation of storage and compute.

Three meters, billed three different ways

WHAT YOU PAY FOR	HOW IT IS CHARGED
Virtual warehouse compute	Credits, per second, 60-second minimum on each start or resume.
Storage	Flat monthly rate per compressed terabyte. Varies by provider and region.
Cloud services	Only the portion exceeding 10% of that day's warehouse credits.
Serverless compute	Each feature's own rate. Snowpipe, clustering, search optimization, tasks.
Data transfer	Egress across regions or clouds. Ingress is free.

ON THE EXAM

Separate meters. A cost question is usually asking which one an activity lands on.

The cloud services allowance, precisely

- Cloud services is charged **only if** the day's cloud services credits exceed **10% of that day's warehouse credits**.
- Calculated **daily, in UTC**. The monthly figure is the sum of those daily calculations.
- The adjustment never exceeds actual usage. If you used less than the allowance, you pay nothing for it.
- **Serverless compute does not count** towards the 10% calculation.
- Resource monitors **ignore** the adjustment and count all cloud services credits, even unbilled ones.

> ON THE EXAM
>
> Metadata-heavy patterns push you over: thousands of tiny queries, or heavy INFORMATION_SCHEMA use with little warehouse work.

What the storage layer will hold

KIND	MEANING
Structured	Rows and columns. A strict tabular schema.
Semi-structured	JSON, XML, Avro, ORC, Parquet. A flexible schema.
Unstructured	Documents, images, audio. No inherent schema.

ON THE EXAM

All three, in one platform, queried with ANSI SQL. Semi-structured lands in a VARIANT column; unstructured is reached through stages and the FILE type.

Three families of table

FAMILY	WHERE THE DATA SITS, AND WHAT IT IS FOR
Snowflake tables	Snowflake's own storage, in micro-partitions. The default. Ideal for data warehousing.
Apache Iceberg tables	External cloud storage that you manage. For data lakes you cannot or will not move into Snowflake.
Hybrid tables	Optimised for low-latency reads and writes. Row locking, unique and referential constraints. Unistore.

ON THE EXAM

Iceberg and hybrid tables are new to COF-C03. Recognising which family a scenario needs is enough at this stage.

Snowgrid, and why regions matter

- An account lives in **one cloud provider and one region**. You choose both when it is created.
- **Snowgrid** is the cross-region, cross-cloud layer that connects accounts across that boundary.
- It underpins **listings** for collaboration across clouds, consistent policy across regions, and **replication** for disaster recovery.
- Region matters. A direct share does not cross regions on its own. Domain five deals with what does.

> ON THE EXAM
>
> When a scenario names two regions or two cloud providers, that detail was planted deliberately. It usually rules out the obvious answer.

How this module gets tested

- **'Which layer handles X.'** Authentication, metadata, optimisation and access control are all cloud services.
- **'Why is this billed.'** Work out which of the meters the activity lands on.
- **'Two teams interfering.'** Separate warehouses. Almost never a bigger single warehouse.
- **Anything on-premises.** Not possible. Distrust the option immediately.

> ON THE EXAM
>
> Four question shapes, all answerable from one diagram. This is why architecture is worth 31%.

> An analytics team runs interactive dashboards during the working day. A data engineering team runs a heavy nightly transformation that has begun overrunning into the morning.
>
> When the two overlap, dashboard queries that normally return in two seconds take over a minute. Both teams currently use the same virtual warehouse. Budget is under scrutiny.

What is the most appropriate remedy?

A. Resize the shared warehouse from Medium to 2X-Large so it can absorb both workloads.

B. Create a second virtual warehouse and point the transformation job at it.

C. Move the dashboard tables into a separate database so the two workloads read different storage.

D. Enable multi-cluster scaling on the shared warehouse and set the scaling policy to Economy.

◆ PAUSE AND DECIDE

What is the most appropriate remedy?

A. Resize the shared warehouse from Medium to 2X-Large.

Would help, and costs sixteen times as much per hour while the two workloads still contend. Budget was named as a constraint.

B. Create a second virtual warehouse for the transformation job.

Warehouses are independent and have no effect on each other. The workloads stop competing, and each is sized for its own job.

C. Move the dashboard tables into a separate database.

Databases are a logical grouping. Contention was for compute, and storage was never the bottleneck.

D. Multi-cluster with an Economy scaling policy.

Multi-cluster answers concurrency, not two workloads of different shapes. Economy also makes it slower to add a cluster.

THE SIGNAL

'Budget is under scrutiny' eliminated A. Read those sentences as instructions, not background.

> An analyst runs a query at nine in the morning. A colleague runs the identical query twenty minutes later against unchanged data, and it returns in well under a second.
>
> Checking Snowsight, the colleague finds that no virtual warehouse was running at the time, and no warehouse credits were consumed by their query.

Which layer served the second query?

A The compute layer, using a warehouse that auto-resumed and suspended too quickly to appear.

B The database storage layer, which serves cached micro-partitions directly.

C The cloud services layer, from the result cache.

D The compute layer, from the warehouse local disk cache retained after suspension.

◆ PAUSE AND DECIDE

Which layer served the second query?

A Compute, from a warehouse that resumed and suspended quickly.

A resume would have consumed the 60-second minimum. The scenario states no warehouse credits were used.

B Storage, serving cached micro-partitions directly.

The storage layer holds data. It does not execute or serve queries on its own.

C Cloud services, from the result cache.

The result cache lives in cloud services, which is precisely why it can serve a query with every warehouse suspended.

D Compute, from the warehouse local disk cache.

That cache is on the warehouse's local storage and is lost when the warehouse suspends.

THE SIGNAL

'No warehouse credits were consumed' is the whole question. Only one layer can serve a result without compute.

A finance analyst is reviewing a monthly statement. On one day the account consumed 400 virtual warehouse credits and 32 cloud services credits.

On another day it consumed 50 warehouse credits and 30 cloud services credits. The analyst wants to understand why the second day contributed a cloud services charge and the first did not.

Which explanation is correct?

A Cloud services is charged only above 10% of that day's warehouse credits, calculated daily.

B Cloud services is free below a fixed monthly ceiling, and the second day happened to cross it.

C Cloud services is charged whenever serverless features run, and serverless ran only on the second day.

D Cloud services is free on days when a resource monitor is attached to the account.

◆ PAUSE AND DECIDE

Which explanation is correct?

A Charged only above 10% of that day's warehouse credits, daily.

Day one: 10% of 400 is 40, and 32 is under it. Day two: 10% of 50 is 5, so 25 credits are billable.

B A fixed monthly ceiling that the second day crossed.

The allowance is proportional to warehouse usage and recalculated every day, not a fixed monthly figure.

C Charged whenever serverless features run.

Backwards. Serverless is billed separately and is explicitly excluded from the 10% calculation.

D Free when a resource monitor is attached.

Resource monitors observe and cap credit usage. They never change what is billable.

THE SIGNAL

A low-warehouse, high-metadata day is the expensive one. The allowance shrinks with the warehouse usage it is proportional to.

A public sector organisation has a policy requiring that analytical data be processed on infrastructure it controls. Its architects want to keep using Snowflake and propose installing it in the organisation's own data centre, alongside its existing systems.

A second requirement states that compute resources must not be shared with other Snowflake customers.

How should an architect respond?

A Deploy Snowflake on-premises using the private cloud installer, then enable Tri-Secret Secure.

B Use a reader account, which runs inside the organisation's own cloud subscription.

C Deploy a Business Critical account and add PrivateLink, which makes the deployment on-premises.

D An on-premises deployment is not possible. Virtual Private Snowflake gives a dedicated, isolated environment instead.

◆ PAUSE AND DECIDE

How should an architect respond?

A Deploy on-premises with a private cloud installer.

No such installer exists. The documentation states you cannot run Snowflake locally or on private cloud infrastructure.

B A reader account in the organisation's own subscription.

A reader account is provisioned and owned by a sharing provider. It is not customer-hosted infrastructure.

C Business Critical plus PrivateLink.

PrivateLink gives private network connectivity. It does not move where Snowflake runs.

D Not possible on-premises. Virtual Private Snowflake gives isolation instead.

Correctly refuses the impossible half and answers the real requirement, which was isolation rather than location.

THE SIGNAL

Two requirements, and only one is satisfiable. Saying so is the answer, not a failure to find one.

What to carry into the next module

- **Three layers.** Storage holds. Compute executes. Cloud services coordinates, and holds the metadata and result caches.
- **Hybrid architecture.** Central repository from shared-disk, MPP scale-out from shared-nothing.
- **Separation is the reason** cloning is free, sharing copies nothing, warehouses cannot interfere, and billing is per second.
- **Cloud services allowance:** 10% of the day's warehouse credits, daily, in UTC. Serverless excluded.
- **Never on-premises.** Public cloud only, on one provider and one region per account.

> ON THE EXAM
>
> Next: editions. Not a topic so much as a question format, because 'what is the MINIMUM edition' is guaranteed to appear.

MODULE 02 · DOMAIN 1 - SNOWFLAKE AI DATA CLOUD FEATURES AND ARCHITECTURE

Editions, and the 'minimum edition' question

Less a topic than a question format. Four tiers, and the boundaries the exam actually asks about.

DOMAIN 1 of 5 WEIGHT 31% MODULE 2 of 31

admin@sthithapragna.com

Editions are **cumulative**. So the answer is never 'which edition has it', it is **'where does it first appear'**.

admin@sthithapragna.com

The four tiers, in one line each

EDITION	WHAT IT IS FOR
Standard	The introductory tier. Full, unlimited access to all standard features.
Enterprise	Large-scale organisations. Adds performance features and the governance policy family.
Business Critical	Extremely sensitive data. Adds encryption control, private connectivity, compliance, and failover.
Virtual Private Snowflake	The strictest requirements. A completely separate environment sharing no hardware with other accounts.

ON THE EXAM

Business Critical was formerly called Enterprise for Sensitive Data. Older material still uses the old name.

What Standard already includes

The ones people get wrong, by assuming security must mean Enterprise

- **Federated authentication and SSO. OAuth. MFA. Network policies.** All four are Standard.
- **Fail-safe, 7 days.** Every edition. Only Time Travel is tiered.
- **Replication** of databases and shares between accounts in an organisation.
- **Dynamic tables, external tables, hybrid tables, Iceberg tables.** Clustering keys and Automatic Clustering.
- **Snowpipe, Snowpipe Streaming, Kafka connector, streams, tasks.**

> ON THE EXAM
>
> If a question implies you must upgrade to get authentication, encryption or continuous ingestion, distrust it.

Standard also includes all of this

- **The entire Cortex surface.** AI functions, Copilot, Cortex Analyst, Cortex Search, Cortex Fine-tuning.
- **All of Snowflake ML.** ML Functions, Model Registry, Feature Store.
- **Snowpark and Streamlit in Snowflake.** External functions and external network access.
- **Marketplace, public and private listings**, and cross-cloud auto-fulfilment.
- **Snowflake Optima**, at no extra cost, though it needs a Gen2 or Adaptive warehouse.

> ON THE EXAM
>
> None of the AI content is edition-gated. A scenario cannot force an upgrade by mentioning Cortex.

What Enterprise adds

CLUSTER	FEATURES
Time Travel	Extended retention, up to 90 days. Standard is capped at 1 day.
Performance	Multi-cluster warehouses. Materialized views. Search optimization. Query acceleration.
Governance	Masking. Row access. Aggregation and projection policies. Differential privacy.
Audit	Classification. ACCESS_HISTORY view. Data metric functions.
Other	Periodic rekeying. Synthetic data. Your own clean rooms. Early release access.

ON THE EXAM

Two clusters do most of the work: the four accelerators, and the policy family.

What Business Critical adds

CLUSTER	FEATURES
Encryption	Tri-Secret Secure. Customer-managed keys plus Snowflake-managed keys.
Networking	PrivateLink, Azure Private Link, Google Private Service Connect.
Continuity	Failover and failback between accounts. Client redirect.
Compliance	PHI under HIPAA and HITRUST, needing a signed BAA. PCI DSS. FedRAMP. ITAR.

ON THE EXAM

Four signals: customer-managed keys, private networking, failover, a named regulation.

Virtual Private Snowflake, and the quirk

- **A dedicated metadata store and a dedicated pool of compute.** No hardware shared with accounts outside the VPS.
- Full encryption of **all** data transmissions, including internal ones inside the isolated environment.
- The **account locator format differs** from every other edition.
- **Marketplace and public listings are not available.** Private listings, Universal Search and cross-cloud auto-fulfilment are.
- VPS accounts cannot collaborate in Snowflake Data Clean Rooms, though they can create their own.

> ON THE EXAM
>
> VPS is the one tier that is not strictly a superset. Isolation costs reach, which is the whole point of it.

The trap the exam sets most often

STANDARD IS ENOUGH

Replication

- Keep a synchronised copy in another region
- Databases and shares, within one organisation
- Read-only secondary you can promote manually
- Available in every edition

BUSINESS CRITICAL REQUIRED

Failover and failback

- Automatic promotion of the secondary
- Client redirect, so applications follow
- Failover groups for coordinated recovery
- Business Critical or higher, no exceptions

THE TIEBREAKER

Read the verbs. 'Keep a copy' is replication. 'Fail over automatically' or 'redirect clients' is Business Critical.

Minimum edition: the greatest hits

IF THE SCENARIO NEEDS...	MINIMUM EDITION
Time Travel beyond 1 day, up to 90	Enterprise
Multi-cluster warehouse, materialized view, search optimization, query acceleration	Enterprise
Masking policy, row access policy, classification, ACCESS_HISTORY	Enterprise
Customer-managed keys, private connectivity, automatic failover	Business Critical
PHI, PCI, FedRAMP, or any named regulation	Business Critical
No shared hardware, dedicated metadata store	Virtual Private Snowflake

ON THE EXAM

Six rows will answer almost every minimum-edition question on the paper. Learn these before anything else in this module.

How to answer one under time pressure

- **Find the named feature first.** The scenario dressing rarely matters. One noun decides it.
- **If two features are named, the higher one wins.** Masking plus PHI is Business Critical, not Enterprise.
- **Do not upgrade for security basics.** SSO, OAuth, MFA and network policies are Standard.
- **Do not upgrade for AI.** Cortex and Snowflake ML are Standard throughout.
- If nothing in the list appears, the answer is probably **Standard**, and that is a real answer.

> ON THE EXAM
>
> The commonest error is answering one tier too high. Snowflake is more generous at Standard than most candidates expect.

> A hospital group is designing a Snowflake account. Patient records containing protected health information will be stored and analysed. Analysts must see a masked version of the patient identifier, while a small clinical role sees the full value.
>
> The compliance team also requires that the organisation controls one of the encryption keys protecting the data at rest.

What is the minimum Snowflake edition required?

A. Standard, because column-level security and encryption are included in all editions.

B. Enterprise, because masking policies are an Enterprise feature.

C. Business Critical, because of the PHI and customer-managed keys.

D. Virtual Private Snowflake, because health data requires dedicated infrastructure.

◆ PAUSE AND DECIDE

What is the minimum Snowflake edition required?

A Standard.

Encryption is universal, but column-level security is not. Masking policies start at Enterprise.

B Enterprise.

Correct about masking, and it stops reading. PHI and customer-managed keys both push higher.

C Business Critical.

PHI needs HIPAA and HITRUST support, and Tri-Secret Secure delivers the customer-managed key. Both are Business Critical.

D Virtual Private Snowflake.

Would work, and nothing asked for isolated hardware. Minimum means minimum, so this overshoots.

THE SIGNAL

Two requirements, both Business Critical. The masking detail was there to make Enterprise look sufficient.

> A retailer runs its production account on Standard Edition in one region. After an outage at its cloud provider, the board asks for a disaster recovery plan.
>
> The requirement is that a copy of the production databases exists in a second region at all times, and that if the primary region is lost, the secondary is promoted and client applications reconnect to it without being reconfigured.

What must change, if anything?

A. Upgrade to Business Critical Edition, which adds failover, failback and client redirect.

B. Upgrade to Enterprise Edition, which adds failover and client redirect.

C. Nothing. Replication, failover and client redirect are all available in Standard Edition.

D. Nothing. Use zero-copy cloning into a second region on a scheduled task instead.

◆ PAUSE AND DECIDE

What must change, if anything?

A Upgrade to Business Critical.

Failover, failback and redirecting client connections all first appear at Business Critical.

B Upgrade to Enterprise.

Enterprise adds performance and governance features. It adds nothing to business continuity.

C Nothing. All three are in Standard.

Half right. Replication is Standard, but failover, failback and client redirect are not.

D Scheduled zero-copy clones into a second region.

A clone cannot cross regions. Cloning is metadata pointing at micro-partitions that live in one region's storage.

THE SIGNAL

The first half of the requirement is Standard. The second half is not. Requirements often split across a boundary like this.

> A data team on Standard Edition wants to build a natural-language interface over its sales tables using Cortex Analyst, deploy a Streamlit app for the results, and refresh the underlying tables automatically with dynamic tables.
>
> Their account manager has suggested that this work requires an Enterprise upgrade. Finance has asked for that claim to be checked.

Is an upgrade required, and why?

A. Yes. Cortex Analyst and Cortex Search both require Enterprise Edition.

B. Yes. Dynamic tables require Enterprise, because they are a form of materialized view.

C. Yes. Streamlit in Snowflake requires Enterprise because it runs application code.

D. No. Cortex Analyst, Streamlit in Snowflake and dynamic tables are all available in Standard.

◆ PAUSE AND DECIDE

Is an upgrade required, and why?

A Yes, Cortex requires Enterprise.

The entire Cortex surface is Standard. No AI feature is edition-gated.

B Yes, dynamic tables are a materialized view.

Materialized views are Enterprise, and dynamic tables are a different object and are Standard.

C Yes, Streamlit runs application code.

Streamlit in Snowflake, Snowpark and external access are all Standard.

D No. All three are available in Standard.

Correct on all three, and 'no upgrade needed' is a real answer the exam does use.

THE SIGNAL

B is the best distractor here. Dynamic tables and materialized views both maintain results automatically, and only one is Enterprise.

> A financial institution runs a Virtual Private Snowflake account for its most sensitive analytics. A business unit wants to publish a curated dataset as a public listing on the Snowflake Marketplace to generate revenue.
>
> An architect has assumed this is possible, on the grounds that VPS includes everything in Business Critical.

What should the architect be told?

A. It works as assumed. VPS is a strict superset of Business Critical.

B. Marketplace and public listings are not available in VPS. Private listings are.

C. It works, but only after enabling cross-cloud auto-fulfilment, which VPS disables by default.

D. It works only if the listing is published from a reader account created by the VPS account.

◆ PAUSE AND DECIDE

What should the architect be told?

A VPS is a strict superset.

The stated assumption, and the official matrix contradicts it. VPS trades reach for isolation.

B No Marketplace or public listings in VPS. Private listings are available.

Matches the matrix. Public reach conflicts with the isolation VPS exists to provide.

C Enable cross-cloud auto-fulfilment first.

Auto-fulfilment is available in VPS, and it is not the blocker. Invented configuration.

D Publish from a reader account.

A reader account is a consumer-side mechanism for people without a Snowflake account. It does not publish listings.

THE SIGNAL

'Higher edition' is not the same as 'more capable'. VPS buys isolation, and isolation costs reach.

What to carry into the next module

- **Cumulative tiers.** The question is always where a feature *first* appears, never which tier has it.
- **Enterprise:** 90-day Time Travel, the four accelerators, the whole policy family, classification and ACCESS_HISTORY.
- **Business Critical:** customer-managed keys, private connectivity, failover and failback, named regulations.
- **Standard is generous.** SSO, MFA, network policies, replication, Fail-safe, all table types, all of Cortex.
- **VPS is not a strict superset.** No Marketplace, no public listings.

> ON THE EXAM
>
> Next: micro-partitions and clustering. The storage internals that most performance questions actually resolve to.

MODULE 03 · DOMAIN 1 - SNOWFLAKE AI DATA CLOUD FEATURES AND ARCHITECTURE

Micro-partitions, pruning and clustering

The storage internals that most performance questions quietly resolve to.

DOMAIN 1 of 5 WEIGHT 31% MODULE 3 of 31

admin@sthithapragna.com

Snowflake has no indexes to create. Query speed comes from **not reading data**, and the layout decides how much you skip.

What a micro-partition is

The numbers here are quoted directly in questions, so learn them as written

- **50 MB to 500 MB of uncompressed data.** The stored size is smaller, because data is always stored compressed.
- A **contiguous unit of storage** holding a group of rows, organised **columnar** within the partition.
- Created **automatically on every Snowflake table**. You never define or maintain them.
- Partitioned **using the order the data arrives in** as it is inserted or loaded.
- **Immutable.** DML never edits one. It writes new micro-partitions and marks the old ones as no longer current.

> ON THE EXAM
>
> A large table can hold millions, even hundreds of millions of micro-partitions. That granularity is the point.

Why the design works

The documented benefits, and each one is a possible answer

- **Derived automatically.** No static partitioning scheme to define up front or maintain.
- **Small.** That enables efficient DML and fine-grained pruning.
- **They may overlap in value range.** Combined with uniform small size, that is what *prevents skew*.
- **Columnar.** Only the columns a query references are scanned.
- **Compressed per column, per partition**, with Snowflake choosing the algorithm each time.

> ON THE EXAM
>
> Overlap is listed as a benefit, not a defect. Traditional static partitioning cannot overlap, and that is exactly why it skews.

The metadata that makes it all work

SNOWFLAKE RECORDS	USED FOR
The range of values for each column	Deciding a partition cannot contain a match, so it can be skipped.
The number of distinct values	Cost estimation and plan choice by the optimiser.
Additional properties	Optimisation and efficient query processing.

ON THE EXAM

This lives in the cloud services layer, which is why COUNT(*) and MIN/MAX can answer with no warehouse running.

How pruning actually runs

01

Read metadata

Compare the query's filter against each partition's value ranges.

02

Prune partitions

Skip every micro-partition that cannot contain a match.

03

Prune columns

Within survivors, read only the columns the query references.

04

Scan what is left

Whatever remains is the only data the warehouse touches.

ON THE EXAM

Two independent savings. Fewer partitions, and fewer columns within them. A SELECT * gives up the second one.

How good is the pruning, and when does it fail

- **The documented ideal:** a filter selecting 10% of a range should scan about **10% of the micro-partitions**.
- The closer the **ratio of partitions scanned to data actually selected**, the better the pruning.
- Pruning works on **semi-structured columns** too, not just ordinary ones.
- **Snowflake does not prune on a predicate containing a subquery**, even if that subquery returns a constant.
- Wrapping the filter column in a **function** generally defeats pruning. Filter on the raw column where you can.

> ON THE EXAM
>
> Query Profile reports partitions scanned against partitions total. That ratio is the diagnosis.

Two consequences of immutability

- **Some operations are metadata-only.** Deleting every row from a table does not rewrite anything.
- **Dropping a column does not rewrite the micro-partitions.** The dropped column's data stays in storage.
- Old versions of partitions are what **Time Travel** reads. Nothing was overwritten, so the past is still there.
- A **clone** is a new set of pointers to existing partitions. Immutability is why that is safe.

> ON THE EXAM
>
> If you drop a column expecting the storage bill to fall, it will not. That surprises people, and it is documented behaviour.

Pruning only helps if the rows you want are **together**. Which brings us to clustering.

Natural clustering, and how it decays

- **Every table is clustered by something**, whether you asked for it or not: the order the data was loaded in.
- Time-series data loaded chronologically is **naturally well clustered on time**. Often nothing more is needed.
- **DML degrades it.** INSERT, UPDATE, DELETE, MERGE and COPY gradually scatter related rows across partitions.
- Loading in an order unrelated to how you query means **poor clustering from day one**.

> ON THE EXAM
>
> Snowflake does not recluster a table with no explicit CLUSTER BY. Natural clustering decays and stays decayed.

Measuring it: overlap and depth

- Snowflake tracks the **total partitions**, how many **overlap** in value range, and the **depth** of that overlap.
- **Clustering depth** is the average depth of overlapping partitions. It is 1 or greater, and **smaller is better**.
- An **empty table has a depth of 0**. A table with no overlap at all is in a **constant state**.
- Depth is **not an absolute measure**. The documentation says query performance is the real indicator.

> ON THE EXAM
>
> Reaching a constant state on a real table is described as 'neither likely nor required'. Do not chase depth 1.

The two functions to know by name

FUNCTION	WHAT IT GIVES YOU
SYSTEM$CLUSTERING_DEPTH	The average clustering depth for the specified columns. A single number.
SYSTEM$CLUSTERING_INFORMATION	Depth plus the fuller picture: partition counts, overlap histogram, and which clustering version the table uses.
AUTOMATIC_CLUSTERING_HISTORY	Account Usage view. Credits consumed reclustering, per table, over time.

ON THE EXAM

Learn the two SYSTEM$ names as written. Questions quote them exactly and the distractors are near-misses.

When to define a clustering key

- The table is **large**. Snowflake's guidance points at tables in the terabyte range, with many micro-partitions.
- Queries **filter or join selectively** on the same column or two, repeatedly.
- Query performance has **degraded over time** as DML accumulated.
- **Not** for small tables. Reclustering cost will exceed any benefit.
- **Not** for tables written far more often than they are read. Every write makes more work.

> ON THE EXAM
>
> Clustering is not free and not automatic-by-default. It is an opt-in trade of ongoing credits for faster reads.

Choosing the key

- **Order from lowest to highest cardinality.** A high-cardinality column first ruins clustering on the ones after it.
- **Cardinality in the middle.** High enough to prune well, low enough to group rows together.
- Prefer columns used in **selective filters**, then **join predicates**. Favour those over ORDER BY or GROUP BY columns.
- Snowflake recommends **no more than three or four columns**. Each one adds reclustering cost.
- For a very high-cardinality column, cluster on an **expression** that coarsens it, such as a truncated date.

> ON THE EXAM
>
> A unique identifier is the classic bad key. Perfect cardinality, ruinous maintenance cost, and it prunes nothing useful.

Clustering key or search optimization?

CLUSTERING KEY

Changes where the data sits

- Reorganises the physical layout
- Ranges and equality: BETWEEN, >=, =
- Medium-cardinality columns
- 'All events for tenant 42 last week'

SEARCH OPTIMIZATION SERVICE

Adds a separate access path

- Builds a search access path alongside
- Point lookups, IN lists, VARIANT, geospatial
- High-cardinality columns
- 'The one event with this exact id'

THE TIEBREAKER

They are not substitutes and can be used together. Range scan, cluster. Needle in a haystack, search optimization.

What Automatic Clustering costs, and how to stop it

- Runs in the **background with no warehouse** of yours, and bills as **serverless** credits.
- **Non-blocking.** DML against the table continues while reclustering runs.
- Reclustering deletes and re-inserts rows into new partitions, so it costs **storage as well as credits**.
- **ALTER TABLE … SUSPEND RECLUSTER** stops it, and stops the charges, without dropping the key.
- Dropping the clustering key prevents all future reclustering permanently.

> ON THE EXAM
>
> Reclustering does not begin the moment you define a key. Snowflake reclusters only if the table will benefit.

New: Optima Clustering, from 1 September 2026

- Automatic Clustering now has two versions. **Newly clustered tables use Optima Clustering** from that date.
- Tables clustered **before** then stay on **Clustering Classic**. There is no forced migration.
- **Classic uses only the first 5 bytes** of each clustering key column. Optima allows up to 1 KB across all columns.
- Optima bills on **data volume ingested** rather than compute hours, which makes the cost predictable.
- This landed days before this course was recorded. **Answer exam questions from the Classic behaviour.**

> ON THE EXAM
>
> The 5-byte limit is why the docs suggest clustering on a substring when a string column's leading characters are identical.

A 6 TB events table is queried many times a day with filters on event_date, usually for a single week. The table is loaded continuously throughout the day from several sources, and rows arrive out of order.

The Query Profile for a typical query shows partitions scanned of 41,900 out of 42,300 total. Bytes spilled to local storage is zero. The warehouse is not queuing.

What is the most effective change?

A Increase the warehouse from Large to 3X-Large so the scan completes faster.

B Define a clustering key on event_date so related rows are grouped into fewer micro-partitions.

C Enable the Search Optimization Service on the table.

D Create a materialized view over the table filtered to the last seven days.

◆ PAUSE AND DECIDE

What is the most effective change?

A. Increase the warehouse to 3X-Large.

Zero spilling and no queuing means the warehouse is not the constraint. This buys speed by paying eight times more to read data you should be skipping.

B. Define a clustering key on event_date.

Scanning 99% of partitions to return one week is a pruning failure, and out-of-order loading is the documented cause.

C. Enable the Search Optimization Service.

Built for point lookups on high-cardinality columns. This is a range scan on a date.

D. A materialized view for the last seven days.

'Usually a week' is not always the same week. The view would miss any query outside its filter.

THE SIGNAL

Almost 42,000 partitions scanned of 42,300, to return one week of a multi-year table. That single ratio is the diagnosis.

> A team read that clustering improves performance and applied clustering keys across their warehouse, including a 3 GB reference table holding product categories that is joined into most queries.
>
> Two weeks later, the Automatic Clustering line on the bill has grown noticeably. Query times on the reference table are unchanged.

What is happening, and what should they do?

A. The key is on the wrong column. Recreate it on the join column instead.

B. Suspend Automatic Clustering account-wide, and recluster manually each month.

C. Reclustering has not finished. Wait for it to reach a constant state.

D. The table is too small to benefit. Drop the clustering key from it.

◆ PAUSE AND DECIDE

What is happening, and what should they do?

A Wrong column. Recreate on the join column.

Assumes clustering was the right tool. A 3 GB table has too few partitions for any key to help.

B Suspend account-wide and recluster manually.

Manual reclustering is deprecated, and this would also disable clustering on the large tables where it is working.

C Wait for a constant state.

The docs call reaching a constant state neither likely nor required. Waiting means paying indefinitely.

D Too small to benefit. Drop the key.

Clustering pays off on large tables with many partitions. Dropping the key stops all future reclustering charges.

THE SIGNAL

The right answer here is to stop doing something. 'Apply the good practice everywhere' is itself the mistake.

> An engineer is defining a clustering key for a large multi-tenant events table. Queries almost always filter on both event_date and tenant_id together. There are around 900 tenants and four years of daily data.
>
> The engineer's first draft is CLUSTER BY (event_id, tenant_id, event_date, region, source_system), where event_id is a unique identifier.

What is the main problem with this key?

A. A unique column leads, and the columns are not ordered from lowest to highest cardinality.

B. Nothing. More columns give the optimiser more opportunities to prune.

C. Clustering keys cannot contain more than two columns, so the statement will fail.

D. event_date should be removed, because dates are already handled by natural clustering.

◆ PAUSE AND DECIDE

What is the main problem with this key?

A A unique column leads, and the order is wrong.

A unique leading column is the worst case. It also destroys clustering on every column after it.

B Nothing. More columns prune more.

Backwards. Each extra column adds reclustering cost, and Snowflake recommends no more than three or four.

C Keys are limited to two columns.

There is no such limit. Three to four is a recommendation, not a hard cap.

D Remove event_date, natural clustering handles it.

Natural clustering is load order, which decays. event_date is the best candidate for the leading position.

THE SIGNAL

Better: CLUSTER BY (event_date, tenant_id). Coarsest first, two columns, and both appear in the actual filters.

A table is clustered on region, a column holding values such as 'EMEA' and 'APAC'. Queries against it were fast for months.

A reporting team has rewritten its dashboard queries to filter with WHERE UPPER(region) = 'EMEA', because some newer rows arrived in mixed case. Since then the dashboards scan almost the whole table, and the clustering depth reported for the table has not changed.

Why did pruning stop working?

A The clustering key was invalidated by the mixed-case rows and must be redefined.

B Automatic Clustering suspended itself because the table reached a constant state.

C The function on the filter column defeats pruning, because the stored metadata describes the raw values.

D String columns cannot be clustered, so the key was never having any effect.

◆ PAUSE AND DECIDE

Why did pruning stop working?

A The key was invalidated by mixed-case rows.

Clustering depth is unchanged, which the scenario states. The physical layout is fine; the query changed.

B Automatic Clustering suspended at a constant state.

Reaching a constant state does not suspend anything, and it would not slow queries if it did.

C The function defeats pruning. Metadata describes raw values.

Ranges are recorded for the stored column. Snowflake cannot tell which partitions satisfy a predicate on the function's output.

D String columns cannot be clustered.

They can. Classic uses only the first 5 bytes of each column, which is a different limitation entirely.

THE SIGNAL

The clue is that clustering depth did not change. The data is laid out exactly as before. Only the query stopped being able to use it.

What to carry into the next module

- **50 to 500 MB uncompressed**, columnar, immutable, automatic. Metadata: value ranges, distinct counts, more.
- **Pruning is two-stage:** partitions first, then columns. Subqueries and functions in a predicate defeat it.
- **Natural clustering is load order**, and it decays under DML. Snowflake will not fix it unless you define a key.
- **Depth: smaller is better, 1 or more, 0 for an empty table.** Not an absolute measure.
- **Do not cluster small or write-heavy tables.** Lowest cardinality first, three or four columns at most.

> ON THE EXAM
>
> Next: table types and view types. Permanent, transient, temporary, external, Iceberg, dynamic, hybrid, and the three kinds of view.

MODULE 04 · DOMAIN 1 - SNOWFLAKE AI DATA CLOUD FEATURES AND ARCHITECTURE

Table types and view types

Seven kinds of table, three kinds of view, and the retention behaviour that separates them.

DOMAIN 1 of 5 WEIGHT 31% MODULE 4 of 31

admin@sthithapragna.com

Every table type is a trade between **how much protection you get** and **how much storage you pay for**.

The grid to memorise

TABLE TYPE	TIME TRAVEL (DAYS)	FAIL-SAFE	HISTORICAL DATA, MIN TO MAX
Permanent, Standard	0 or 1	7 days	7 to 8 days
Permanent, Enterprise	0 to 90	7 days	7 to 97 days
Transient	0 or 1	None	0 to 1 day
Temporary	0 or 1	None	0 to 1 day

ON THE EXAM

Fail-safe is 7 days and belongs to permanent tables only. It is never configurable and never extendable.

The details that get tested

- **The default table type is permanent.** CREATE TABLE with no keyword gives you the expensive one.
- **A transient table cannot be converted** to any other type after creation. Nor can a permanent one become transient.
- **Temporary tables are invisible outside their session**, and are dropped when it ends. The data is then unrecoverable.
- If a permanent table has **Time Travel set to 0, it enters Fail-safe immediately** when dropped.
- Once transient Time Travel passes, the data cannot be recovered **by you or by Snowflake**.

> ON THE EXAM
>
> That last point is the one candidates resist. There is no support ticket that recovers a transient table. None.

Choosing between the three

Snowflake's own guidance, which is also the exam's

- **Long-lived tables, such as fact tables, should always be permanent.** They need Fail-safe.
- **Short-lived ETL work tables can be transient**, to eliminate Fail-safe cost.
- **Session-scoped scratch data** that nothing else needs to see is temporary.
- If downtime and reload time matter, permanent may still be the better answer **despite** the Fail-safe cost.

> ON THE EXAM
>
> Shared by more than one session or job? Then it cannot be temporary, whatever the cost argument says.

The other four table types

TYPE	WHAT IT IS
External	Read-only over files in cloud storage. No DML at all. CSV, JSON, Parquet and more.
Iceberg	Apache Iceberg format in your own storage. ACID, schema evolution, readable by other engines.
Dynamic	Materialises a SELECT and keeps it fresh to a target lag. Snowflake manages the refresh.
Hybrid	Row-based, for transactional work. Enforced constraints, indexes, row-level locking. Unistore.

ON THE EXAM

External, Iceberg, dynamic and hybrid are all available in Standard Edition. None of them is an upgrade trigger.

Iceberg: the two flavours

SNOWFLAKE AS THE CATALOG

Full platform support

- Read and write, including DML
- Snowflake handles compaction and maintenance
- Best performance of the two
- The default choice for Snowflake-first work

EXTERNAL CATALOG

Limited platform support

- AWS Glue, Open Catalog, any Iceberg REST catalog
- Reached through a catalog integration
- Snowflake assumes no lifecycle management
- For lakes another system already owns

COMMON TO BOTH

Both sit on an external volume, which bundles your storage location with the IAM trust Snowflake needs.

External table or Iceberg table?

- **External table:** read-only, no DML, no transactions. A view over files in a bucket.
- **Iceberg table:** a real table format. ACID transactions, schema evolution, its own snapshots and time travel.
- Iceberg is **readable and writable by other engines**. Spark, Trino, Flink and others share the same files.
- An external-catalog Iceberg table **can be converted** to use Snowflake as the catalog later.
- A **catalog-linked database** auto-discovers and stays in sync with a remote catalog's namespaces and tables.

> ON THE EXAM
>
> If a scenario mentions other engines writing the same data, it is pointing at Iceberg, not external tables.

Tables store rows. **Views** store a query. The interesting question is what, if anything, gets materialised.

The three kinds of view

KIND	BEHAVIOUR
Standard	Stores only the definition. The query runs every time the view is referenced. No storage cost.
Materialized	Stores the results and maintains them automatically. Storage plus serverless maintenance cost. Enterprise.
Secure	Not a third thing. A property. Both standard and materialized views can be made secure.

> ON THE EXAM
>
> 'Secure' is a modifier, not a category. That framing alone answers several questions.

Secure views

- They hide the **view definition itself**, not just the data. Ordinary views expose their SQL to anyone who can see the view.
- They bypass certain query optimisations, which means a documented **performance penalty**.
- **Only secure views can be shared.** A non-secure view cannot be added to a share.
- **SHOW VIEWS** has an IS_SECURE column, which is how you check.

> ON THE EXAM
>
> The optimisations a normal view enjoys can leak information about rows the user was not meant to see. That is the whole reason.

Materialized view limitations

The restriction list is the question. Learn the first line above all

- **A single table only. No joins, including self-joins.** This is the one that decides most questions.
- Cannot query another view, a materialized view, a **hybrid table**, a **dynamic table**, or a UDTF.
- No **window functions**, no UDFs, no HAVING, no ORDER BY, no LIMIT, no nested subqueries.
- **No Time Travel** on a materialized view. **Not monitored by resource monitors.** No DML, no TRUNCATE.
- Cannot be cloned directly, but **are** cloned when their schema or database is cloned.

> ON THE EXAM
>
> When a scenario needs a join and offers a materialized view, that option is wrong. It is the commonest single distractor here.

Materialized view or dynamic table?

MATERIALIZED VIEW

Always current, tightly limited

- No lag. Results are always current
- One table, no joins, no window functions
- Enterprise Edition
- Automatic, no warehouse of yours

DYNAMIC TABLE

Slight lag, very few limits

- Minimum target lag of 1 minute
- Joins, aggregations, chained pipelines
- Standard Edition
- Runs on a warehouse you name

THE TIEBREAKER

Needs a join? Dynamic table. Needs zero lag? Materialized view. That single question settles most of them.

Dynamic tables, the details

- **Minimum target lag is 1 minute**, and target lag is a target rather than a guarantee.
- **TARGET_LAG = DOWNSTREAM** refreshes an intermediate table only when a dependent needs it.
- Refresh modes: **AUTO** is the default and tries incremental, falling back to full. Also INCREMENTAL, FULL and ADAPTIVE.
- **The query definition is immutable.** Changing it means drop and recreate, and a full initial refresh.
- **Cannot be downstream from a materialized view, an external table, or a stream.**

> ON THE EXAM
>
> If the last table in a chain is set to DOWNSTREAM, nothing downstream is asking for it, so nothing ever refreshes.

Three ways to keep derived data fresh

REQUIREMENT	REACH FOR
Always-current data with no lag	Materialized view
Sub-minute freshness, or custom logic such as MERGE	Streams and tasks
Anything between: joins, aggregations, chained pipelines	Dynamic table

ON THE EXAM

This is Snowflake's own decision guide, near enough verbatim. It settles a whole family of questions.

A nightly ETL pipeline builds a 4 TB staging table from files in a landing bucket. The table is fully rebuilt every night, and is read by three separate transformation jobs running in different sessions before being discarded.

The finance team has flagged that storage costs for this table are unexpectedly high. The source files are retained in the bucket for 30 days.

Which table type should the staging table use?

A Permanent, so that Fail-safe protects the pipeline against a failed rebuild.

B Temporary, so that the table is purged automatically and costs nothing beyond the session.

C Transient, with the Time Travel retention set to 0.

D External, so that the data is never stored inside Snowflake at all.

◆ PAUSE AND DECIDE

Which table type should the staging table use?

A. Permanent with Fail-safe.

Seven days of Fail-safe on 4 TB rebuilt nightly, for data reproducible from files kept for 30 days. This is the current bill.

B. Temporary.

Three jobs read it in different sessions. A temporary table is invisible outside the session that created it.

C. Transient, Time Travel 0.

No Fail-safe, no Time Travel storage, persists across sessions, and the data is reproducible from the retained files.

D. External table.

Read-only over raw landing files. The pipeline builds a transformed table, which an external table cannot hold.

THE SIGNAL

'Different sessions' eliminated B, and '30 days of source files' made C safe. Both were planted deliberately.

> An engineer discovers that a 900 GB transient table holding three months of reconciled billing data was dropped by a faulty script four days ago. Nobody noticed until now.
>
> The table had DATA_RETENTION_TIME_IN_DAYS set to 1. The upstream system that produced the data has since been decommissioned.

What are the recovery options?

A. There are none. Transient tables have no Fail-safe, and Time Travel expired three days ago.

B. Raise a Snowflake Support ticket to recover the data from Fail-safe.

C. Use UNDROP TABLE, which works for 7 days on any table type.

D. Clone the table from a point in time four days ago using AT with an OFFSET.

◆ PAUSE AND DECIDE

What are the recovery options?

A There are none.

No Fail-safe, and Time Travel of 1 day expired three days before anyone noticed. The data is gone.

B Support can recover it from Fail-safe.

Transient tables have no Fail-safe period at all. There is nothing for Support to recover from.

C UNDROP works for 7 days on any table type.

UNDROP is bounded by Time Travel, not a fixed 7 days. Here that was 1 day, and it expired.

D Clone using AT with an OFFSET.

Cloning at a past point is itself a Time Travel operation. It is bounded by exactly the same expired window.

THE SIGNAL

The docs say it plainly: after the Time Travel period, transient data cannot be recovered by you or by Snowflake.

> A team needs a pre-computed dataset joining an orders table to a customers table, with a running total per customer. Around 40 dashboards read it, and roughly fifteen minutes of staleness is acceptable.
>
> Their first attempt was a materialized view, which failed to create. They are on Enterprise Edition.

What should they build instead, and why did the first attempt fail?

A. A secure materialized view. The original failed because materialized views must be secure to be queried by dashboards.

B. A dynamic table. Materialized views support neither joins nor window functions.

C. A standard view. The original failed because Enterprise Edition caps materialized views per schema.

D. A stream and a task. Materialized views cannot be read by more than ten concurrent sessions.

◆ PAUSE AND DECIDE

What should they build instead, and why did the first attempt fail?

A A secure materialized view.

Security is unrelated to the failure, and making it secure does not lift the join restriction.

B A dynamic table. No joins, no window functions.

Two separate violations, either alone fatal. A dynamic table has neither restriction and 15 minutes is well above the 1-minute minimum.

C A standard view, capped per schema.

No such cap exists. A standard view also recomputes on every read, which is what they were avoiding.

D Streams and tasks, concurrency limit.

No such limit. Streams and tasks would work but add custom logic the requirement never asked for.

THE SIGNAL

Fifteen minutes of acceptable staleness is the tell. It is far above the 1-minute minimum, so a dynamic table fits comfortably.

> An organisation has a large data lake in S3 in Apache Iceberg format. A Spark platform writes to it continuously, and AWS Glue is the Iceberg catalog. Several teams query the lake with Trino.
>
> They now want Snowflake to query the same data, without duplicating it and without changing which system owns the writes.

What should they create in Snowflake?

A An external table over the S3 prefix, refreshed on a schedule.

B A permanent table loaded from S3 with COPY INTO, refreshed nightly.

C An Iceberg table with Snowflake as the catalog, pointed at the same location.

D An Iceberg table using a catalog integration to AWS Glue.

◆ PAUSE AND DECIDE

What should they create in Snowflake?

A An external table over the prefix.

Ignores the Iceberg format entirely. No transactions, no schema evolution, and worse performance.

B COPY INTO a permanent table nightly.

Duplicates the data, which was explicitly ruled out, and makes Snowflake a stale copy of the lake.

C Iceberg with Snowflake as the catalog.

Would give better performance and full DML, and it takes catalog ownership away from Glue. The scenario forbids that.

D Iceberg table with a catalog integration to Glue.

Glue keeps ownership of the catalog and Spark keeps the writes. Snowflake reads the same files, no copy.

THE SIGNAL

C is the better table and the wrong answer. 'Without changing which system owns the writes' was the constraint.

What to carry into the next module

- **Fail-safe is 7 days, permanent tables only**, never configurable. Everything else maxes out at 1 day of Time Travel.
- **Transient data past its Time Travel is gone.** Not by you, not by Support. Transient cannot be converted later.
- **Secure is a property, not a view type**, and only secure views can be shared.
- **Materialized view: one table, no joins, no window functions.** That line decides most of these questions.
- **No lag, materialized view. Sub-minute or custom logic, streams and tasks. Everything else, dynamic table.**

> ON THE EXAM
>
> Next: interfaces, drivers and connectors, including the SnowSQL and Snowflake CLI pair the new exam clearly loves.

MODULE 05 · DOMAIN 1 - SNOWFLAKE AI DATA CLOUD FEATURES AND ARCHITECTURE

Interfaces, drivers, connectors and Git

Many doors into one engine. The exam tests which door, and which one cannot open.

DOMAIN 1 of 5 WEIGHT 31% MODULE 5 of 31

admin@sthithapragna.com

All of these are clients. The engine is identical. What differs is **what each client can ask for**.

Snowsight, the web interface

- **Snowsight is the only supported web UI.** The Classic Console was deprecated back in 2023.
- SQL editing, object browsing, dashboards, monitoring, cost management, admin, and the **Query Profile**.
- **Universal Search** across objects, and the Marketplace, are both reached from here.
- **PUT and GET cannot be run from the web UI.** That restriction has its own slide shortly.

> ON THE EXAM
>
> If an option mentions the Classic Console, it is describing an interface that no longer exists.

Worksheets became Workspaces

- **Workspaces replaced Worksheets on 20 April 2026.** Files and folders, sharing, and Git integration.
- **Removal of Legacy Worksheets and Dashboards began 22 June 2026.** Reader accounts follow on 20 September.
- **No native replacement for Dashboards.** A Dashboard-to-Streamlit conversion tool, or third-party BI.
- First use auto-creates a **personal database**, which cannot hold ordinary tables or views.
- The study guide is dated **19 January 2026**, before this change. The exam may still say 'Worksheets'.

> ON THE EXAM
>
> Know both names. No question should hinge on it, because the exam predates the rename.

SnowSQL or Snowflake CLI?

SNOWSQL - LEGACY

snowsql

- SQL, DDL, DML, loading and unloading
- Interactive shell or batch with -f
- Variables use &var syntax, now deprecated
- No new features will be added

SNOWFLAKE CLI - CURRENT

snow

- Everything SnowSQL does, plus more
- Streamlit, Snowpark, Container Services, Native Apps, dbt
- Open-source and developer-centric
- All new features land here

THE TIEBREAKER

Snowflake's own docs call SnowSQL legacy and recommend transitioning. If a task involves deploying anything, it is Snowflake CLI.

SnowSQL, the details worth keeping

- Runs as an **interactive shell**, or in batch mode via stdin or the **-f** option.
- Internal SnowSQL commands all begin with an **exclamation mark**, such as !exit and !source.
- Built *using* the Snowflake Connector for Python, though the connector is **not a prerequisite** to install it.
- Support runs by minor release. 1.2.x ended December 2025, 1.3.x ended May 2026, 1.4.x runs to May 2027.
- From version 1.3.0, **automatic upgrades are disabled**. You reinstall manually.

> ON THE EXAM
>
> SnowSQL still works and is still supported. Legacy means no new features, not switched off.

PUT and GET: the client-only commands

- **Neither can be executed from the Worksheets page in either web interface.** This is documented explicitly.
- Use **SnowSQL, Snowflake CLI, or a driver** instead. Those can reach your local file system.
- Snowsight separately offers a **file-upload UI** for named internal stages. That is not the same as the PUT command.
- An EXECUTION_STATUS of success in QUERY_HISTORY **does not mean the files actually transferred**.

> ON THE EXAM
>
> The reason is obvious once stated: a browser tab cannot be trusted with arbitrary access to your local disk.

Drivers, connectors and the SQL API

WHAT	WHEN IT IS THE ANSWER
JDBC and ODBC	Native drivers. Third-party BI and ETL tools, and Java or C-based applications.
Python, Node.js, Go, .NET, PHP	Programmatic interfaces for application code in those languages.
Spark connector	Reading and writing Snowflake from an existing Spark cluster.
Kafka connector	Streaming topics into Snowflake tables. Covered properly in module 15.
SQL API	A REST API. No driver to install, so it suits serverless functions and lightweight automation.
SnowCD	Not a connection. A diagnostic tool for troubleshooting connectivity.

ON THE EXAM

All of these are Standard Edition. Connectivity is never an edition question.

Git integration: three objects, in order

01

SECRET

TYPE = password. Username plus a personal access token.

02

API INTEGRATION

API_PROVIDER = git_https_api, with allowed prefixes and allowed secrets.

03

GIT REPOSITORY

API_INTEGRATION, ORIGIN, and optionally GIT_CREDENTIALS.

ON THE EXAM

The API_PROVIDER value git_https_api is the detail questions quote. Learn it as written.

What a Git repository actually is in Snowflake

- **A special kind of stage.** Once you see that, everything else about it follows.
- Paths are **@repo/branches/name**, **@repo/tags/name**, and **@repo/commits/hash**.
- **ALTER GIT REPOSITORY … FETCH** pulls a full clone of every branch, tag and commit, and prunes what was deleted upstream.
- **EXECUTE IMMEDIATE FROM @repo/branches/main/setup.sql** runs SQL straight out of the repository.
- SHOW GIT REPOSITORIES, DESCRIBE GIT REPOSITORY, SHOW GIT BRANCHES. An empty repository is not supported.

> ON THE EXAM
>
> EXECUTE IMMEDIATE FROM is the payoff. Version-controlled SQL, deployed by running one statement.

Which door for which job

THE JOB	THE CLIENT
Explore data, read a Query Profile, manage the account	Snowsight
Upload or download files with PUT or GET	SnowSQL, Snowflake CLI, or a driver
Deploy a Streamlit app, Native App or Snowpark object from CI	Snowflake CLI
Connect a BI or ETL tool	JDBC or ODBC
Query from a serverless function with nothing installed	SQL API
Run version-controlled SQL from a repository	Git repository plus EXECUTE IMMEDIATE FROM
Diagnose why a connection is failing	SnowCD

ON THE EXAM

Seven rows that answer most of this module. The middle two are where candidates lose marks.

> A platform team is building a deployment pipeline in GitHub Actions. Each run must create or replace several Snowpark stored procedures, deploy a Streamlit in Snowflake application, and then run a set of SQL migration scripts.
>
> Their existing scripts use SnowSQL with the -f option. An engineer proposes keeping SnowSQL to avoid a migration.

What should the team use, and why?

A Keep SnowSQL. It performs all DDL and DML operations, which covers everything described.

B Snowflake CLI, because SnowSQL cannot deploy Streamlit apps or Snowpark objects.

C The SQL API, because command-line clients cannot be used inside GitHub Actions.

D Snowsight Workspaces with a Git-synced workspace, because deployment must originate in Snowflake.

◆ PAUSE AND DECIDE

What should the team use, and why?

A. Keep SnowSQL. It does all DDL and DML.
True as far as it goes, and deploying a Streamlit app is not a DDL statement. SnowSQL has no command for it.

B. Snowflake CLI. SnowSQL cannot deploy those.
Application deployment is exactly what Snowflake CLI adds, and it also runs the SQL migrations SnowSQL was doing.

C. The SQL API, since CLIs cannot run in Actions.
CLIs run perfectly well in GitHub Actions. The premise is invented.

D. A Git-synced workspace in Snowsight.
Workspaces is an interactive editor. It is not a headless CI runner, and deployment does not have to originate in Snowflake.

THE SIGNAL

A is the trap because its stated reason is true. SnowSQL does do all DDL and DML. The task needed something that is neither.

> An analyst has a 40 MB CSV file on their laptop. They want it in a Snowflake table. Working in Snowsight, they open a SQL editor and run PUT file://~/data/sales.csv @%sales_staging.
>
> The statement fails. They check QUERY_HISTORY and are confused by what they find there.

What is happening?

A PUT cannot be executed from the web interface. It needs SnowSQL, the Snowflake CLI, or a driver.

B The table stage syntax is wrong. PUT requires a named internal stage, not @%table.

C The file exceeds the 25 MB limit for PUT operations and must be split first.

D PUT requires the ACCOUNTADMIN role, which the analyst does not hold.

◆ PAUSE AND DECIDE

What is happening?

A PUT cannot run from the web interface.

Documented explicitly. A browser cannot reach the local file system, so PUT needs a client that can.

B @%table syntax is invalid for PUT.

Table stages are a valid PUT target. The syntax in the scenario is correct.

C Exceeds a 25 MB limit.

Invented. There is no such per-file limit on PUT.

D PUT requires ACCOUNTADMIN.

It requires privileges on the stage, not an elevated system role.

THE SIGNAL

The Query History confusion is the second half of the trap: for PUT and GET, EXECUTION_STATUS success does not mean the file moved.

A team keeps all its Snowflake object definitions in a private GitHub repository. They want Snowflake itself to run those SQL files, so that a deployment is a single statement rather than a script that pipes files through a client.

They have created a secret holding a personal access token, and a Git repository object referencing it.

What else is required, and how do they run the files?

A Nothing else. Run COPY INTO from the repository stage to load and execute the files.

B A notification integration, then a task that reads the repository on a schedule.

C A storage integration pointing at GitHub, then run the files with EXECUTE IMMEDIATE FROM.

D An API integration with API_PROVIDER = git_https_api, then EXECUTE IMMEDIATE FROM the repository path.

◆ PAUSE AND DECIDE

What else is required, and how do they run the files?

A Nothing else. Use COPY INTO.

COPY INTO loads file contents into table rows. It never executes SQL, and the integration is mandatory.

B A notification integration and a scheduled task.

Notification integrations carry cloud messaging events. Nothing here delivers or executes repository files.

C A storage integration to GitHub.

Storage integrations address cloud object storage such as S3 or Blob. GitHub is reached over an HTTPS API.

D API integration with git_https_api, then EXECUTE IMMEDIATE FROM.

The Git repository object requires an API integration of that provider, and EXECUTE IMMEDIATE FROM runs SQL from the clone.

THE SIGNAL

Four integration types exist and they are not interchangeable. Storage for buckets, API for HTTPS endpoints, notification for messaging, catalog for Iceberg.

> An organisation runs a small serverless function that fires a handful of queries after each nightly load, then writes a summary to a monitoring system. The function runs in a constrained runtime where installing and maintaining native libraries is awkward.
>
> The team also has an existing Spark cluster performing heavy transformations against the same data.

Which combination is most appropriate?

A ODBC for the serverless function, and the Spark connector for the cluster.

B The SQL API for both, since it avoids installing anything anywhere.

C The SQL API for the serverless function, and the Spark connector for the cluster.

D SnowSQL for the serverless function, and JDBC for the cluster.

◆ PAUSE AND DECIDE

Which combination is most appropriate?

A. ODBC plus the Spark connector.

Right for Spark, wrong for the function. ODBC is a native driver, which is exactly the install the scenario wants to avoid.

B. SQL API for both.

A REST call per statement is wrong for heavy distributed transformations. The Spark connector pushes work down and moves data in parallel.

C. SQL API plus the Spark connector.

No driver for the constrained runtime, and the purpose-built connector for the cluster that already exists.

D. SnowSQL plus JDBC.

SnowSQL is an interactive client, not a library to call from function code. JDBC would work for Spark but is not the purpose-built option.

THE SIGNAL

Two different constraints in one scenario. 'Installing native libraries is awkward' and 'existing Spark cluster' each point at a different answer.

What to carry into the next module

- **SnowSQL is legacy.** Snowflake CLI gets all new features and handles Streamlit, Snowpark, Native Apps and dbt.
- **PUT and GET cannot run from the web UI.** Use a client. Success in QUERY_HISTORY does not mean the file moved.
- **Git needs three objects:** secret, API integration with git_https_api, and the Git repository. Run files with EXECUTE IMMEDIATE FROM.
- **The SQL API is the no-driver option.** Serverless functions and lightweight automation.
- **Snowsight is the only web UI.** Classic Console went in 2023, Worksheets became Workspaces in April 2026.

> ON THE EXAM
>
> Next: Cortex, Snowpark, Streamlit, Notebooks and ML. The biggest single block of new material on this exam.

MODULE 06 · DOMAIN 1 - SNOWFLAKE AI DATA CLOUD FEATURES AND ARCHITECTURE

Cortex, Snowpark, Streamlit, Notebooks and ML

The largest block of genuinely new material on this exam, and the one older courses do not have.

DOMAIN 1 of 5 WEIGHT 31% MODULE 6 of 31

admin@sthithapragna.com

Every service here exists so the **model comes to the data**, rather than the data being exported to a model.

admin@sthithapragna.com

The Cortex landscape

- **AISQL functions.** LLM operations you call from SQL, row by row. Complete, classify, filter, aggregate, translate, sentiment.
- **Cortex Search.** Semantic and keyword retrieval over *unstructured* documents. The retrieval half of RAG.
- **Cortex Analyst.** Natural language to SQL over *structured* tables, grounded on a semantic model.
- **Document AI.** Structured field extraction from PDFs and images.

> ON THE EXAM
>
> Structured or unstructured is the first question to ask. It separates Analyst from Search immediately.

The AISQL functions

FUNCTION	WHAT IT DOES
AI_COMPLETE	Sends a prompt to a model and returns the completion. The general-purpose one.
AI_CLASSIFY	Sorts text into categories you supply. Multi-label, images and documents.
AI_FILTER	Returns true or false. Usable in SELECT, WHERE, and JOIN … ON.
AI_AGG	Aggregates a text column across rows. Not subject to context-window limits.
AI_PARSE_DOCUMENT	Extracts content from a document file.
AI_SENTIMENT, AI_TRANSLATE, AI_REDACT	Sentiment, translation, and PII scrubbing.

ON THE EXAM

AI_FILTER in a WHERE clause makes an LLM part of the query plan itself.

A naming change in progress

- The **AI_ prefixed functions are the current, canonical names**. Use them.
- The older **SNOWFLAKE.CORTEX namespace is documented as legacy**, to be deprecated by the end of 2026.
- Old to new: **COMPLETE becomes AI_COMPLETE**. CLASSIFY_TEXT becomes AI_CLASSIFY. PARSE_DOCUMENT becomes AI_PARSE_DOCUMENT.
- **Cortex Guard** filters unsafe model responses, and is built on Llama Guard.

> ON THE EXAM
>
> The study guide says 'Cortex AI SQL' without naming functions. Expect either naming, and recognise both.

Cortex Search or Cortex Analyst?

CORTEX SEARCH

Unstructured retrieval

- Documents, contracts, tickets, articles
- Embeddings plus keyword ranking
- Returns relevant passages
- 'What does our returns policy say?'

CORTEX ANALYST

Structured question answering

- Tables, columns, metrics
- Natural language translated into SQL
- Requires a semantic model in YAML
- 'What were our top products last month?'

THE DOCUMENTED FAILURE MODE

Point a structured question at Search, or an unstructured one at Analyst, and you get an answer that is valid and useless.

What makes Cortex Analyst work

- It requires a **semantic model**, a lightweight YAML file. Similar in shape to a schema, but far richer.
- The semantic model carries **business definitions and metric handling** that a bare schema cannot express.
- The generated SQL **runs in your own virtual warehouse**, under **the same RBAC** as the underlying data.
- Snowflake states that Cortex Analyst **does not train on customer data**.

> ON THE EXAM
>
> A thin semantic model produces SQL that runs cleanly and answers the wrong question. That is the practical risk.

Documents, agents, and fine-tuning

- **Document AI** and **AI_PARSE_DOCUMENT** extract structured fields from PDFs and images. Extraction, not retrieval.
- **Cortex Agents** orchestrate. They call Analyst and Search rather than replacing either.
- **Snowflake Intelligence** is the end-user surface sitting over the top of the agent layer.
- **Cortex Fine-tuning** adapts selected base models on your own data, without full training costs.

> ON THE EXAM
>
> Document AI extracts fields into columns. Cortex Search finds passages. Both read documents, for different purposes.

Who is allowed to use Cortex

- Access needs **two** things: the **USE AI FUNCTIONS** account privilege, and a database role such as **SNOWFLAKE.CORTEX_USER**.
- **Both are granted to PUBLIC by default**, and PUBLIC is granted to every user. So by default, everyone can use Cortex.
- To restrict it: **revoke from PUBLIC as ACCOUNTADMIN**, then grant to the specific roles that should have it.
- **Database roles cannot be granted directly to a user.** Grant to a role, then assign that role.
- Narrower roles exist: **CORTEX_ANALYST_USER**, CORTEX_AGENT_USER, CORTEX_EMBED_USER, AI_FUNCTIONS_USER.

> ON THE EXAM
>
> 'Granted to PUBLIC by default' is the governance answer. Nobody enabled Cortex; it was on from the start.

Cross-region inference

- Not every model is available in every region. If yours is not, the call fails unless cross-region inference is enabled.
- Controlled by **CORTEX_ENABLED_CROSS_REGION**. Default is **DISABLED**. Values include ANY_REGION and specific scopes such as AWS_US.
- **Account level only**, not user or session. Only **ACCOUNTADMIN** can set it, and **ORGADMIN cannot**.
- **Credits are consumed in the requesting region**, and there are **no data egress charges**.
- Your stored data never moves. Only the prompt and response travel, transiently, encrypted in transit.

> ON THE EXAM
>
> A compliance-flavoured question about data residency and AI usually turns on this parameter.

How the AI services are billed

SERVICE	CHARGED AS
AISQL functions	AI Credits per million tokens processed. The rate depends on the model chosen.
Cortex Analyst	Its generated SQL runs in your virtual warehouse, so ordinary warehouse credits.
Cortex Agents	AI Credits per million tokens, and additive across every service the agent calls.
ML functions	Compute for training and prediction, plus storage for the trained model instances.
Notebooks	Warehouse credits, or compute pool credits on the Container Runtime.

ON THE EXAM

Model choice is a cost decision. A larger model on the same prompt can cost several times more per million tokens.

That is the AI. The rest of the module is about **running your own code** inside Snowflake.

Snowpark

- Libraries and runtimes for **Python, Java and Scala**, running inside Snowflake.
- A **DataFrame API**. Operations are **lazy** and get translated into SQL that runs on a warehouse.
- That translation is the point. Work is **pushed down** to the engine rather than pulling rows to a client.
- Used to write **UDFs, UDTFs and stored procedures** in those languages. Covered again in module 22.
- Available in **Standard Edition**, like everything else in this module.

> ON THE EXAM
>
> Snowpark is not a separate engine. A DataFrame chain becomes SQL, and you can see it in Query History.

A memory rule that surprises people

- **Standard warehouses have a lower Snowpark memory limit, and that limit does NOT increase with warehouse size.**
- For more memory per node you need a **Snowpark-optimized warehouse**. That is what they exist for.
- **Snowpark-optimized memory does not increase above Medium** either. Going bigger buys cores, not headroom.
- Guidance: train on a **dedicated warehouse** with no concurrent work. Prototype on an **X-Small standard** to control cost.

> ON THE EXAM
>
> An out-of-memory failure on a standard warehouse is not fixed by resizing it. That is a scenario waiting to happen.

Streamlit and Container Services

- **Streamlit in Snowflake** builds and hosts data applications from Python, next to the data, with no separate web server.
- Apps inherit Snowflake **authentication and RBAC**. Sharing an app is a grant, not a deployment.
- **Snowpark Container Services** runs containerised workloads on Snowflake-managed **compute pools**, including GPUs.
- That is what you reach for when the workload needs a language or library Snowflake does not host natively.

> ON THE EXAM
>
> Streamlit is also the migration path for retired Snowsight Dashboards, via the conversion tool from module 5.

Notebooks: two runtimes

WAREHOUSE RUNTIME

Runs on a virtual warehouse

- Fastest to start
- The familiar warehouse Python environment
- CPU only
- SQL analytics and general work

CONTAINER RUNTIME

Runs on compute pools

- Powered by Snowpark Container Services
- Install packages from PyPI or internal repos
- CPU or GPU, with ML packages pre-installed
- Machine learning and deep learning

THE DETAIL PEOPLE MISS

For BOTH runtimes, SQL and Snowpark queries always execute on the warehouse. Only the notebook's own Python differs.

The four ML functions

FUNCTION	WHAT IT ANSWERS
Forecasting	What will this metric be in future, based on past trends? Time series.
Anomaly Detection	Which values differ from what we would expect? Time series.
Classification	Which class does each row belong to? Does NOT require a timestamp.
Top Insights	Which dimensions and values are driving the change in this metric? Root cause.

ON THE EXAM

Top Insights was formerly called Contribution Explorer. Older material and some questions still use the old name.

The rest of Snowflake ML

- **Model Registry.** The central repository for machine learning models across the organisation.
- **Feature Store.** A repository of data transformations that can be reused to train models.
- Container Runtime gives **automatic lineage capture** and Model Registry integration.
- All of it is **Standard Edition**. No part of the AI or ML surface is edition-gated.

> ON THE EXAM
>
> Registry stores models. Feature Store stores the transformations that feed them. Two repositories, two contents.

Which tool for which job

THE REQUIREMENT	THE ANSWER
Summarise or classify text held in a table column	AISQL functions
Let staff ask questions of a document library	Cortex Search
Let staff ask questions of sales tables in plain English	Cortex Analyst plus a semantic model
Pull named fields out of scanned invoices	Document AI or AI_PARSE_DOCUMENT
Process large data in Python without moving it	Snowpark
Train on GPUs, or install arbitrary packages	Notebooks on Container Runtime, or Container Services
Predict a metric, or explain why it moved	ML functions: Forecasting, or Top Insights

ON THE EXAM

Seven rows covering the module. The second and third are the pair worth over-learning.

> A legal team holds eight years of supplier contracts as PDFs in a Snowflake stage, with a directory table over them. Lawyers want to ask questions such as which contracts contain a change-of-control clause, and to be shown the relevant passages.
>
> Separately, the finance team wants to ask questions such as which suppliers we spent the most with last quarter, against ordinary tables.

Which services fit these two requirements?

A. Cortex Analyst for both, since it answers natural-language questions.

B. Cortex Search for the contracts, and Cortex Analyst for the finance tables.

C. Document AI for the contracts, and Cortex Search for the finance tables.

D. AI_COMPLETE for both, prompted with the relevant rows each time.

◆ PAUSE AND DECIDE

Which services fit these two requirements?

A Cortex Analyst for both.

Analyst generates SQL over structured tables. It has nothing to run against a library of PDFs.

B Search for contracts, Analyst for finance.

Unstructured retrieval with passages returned, and structured question answering. Each requirement gets the matching service.

C Document AI for contracts, Search for finance.

Both halves inverted. Document AI extracts known fields; Search does not query structured tables.

D AI_COMPLETE for both.

Would need the right content in the prompt already. Finding that content is the actual problem, and it is what Search does.

THE SIGNAL

'Shown the relevant passages' is the tell for Search. Retrieval returns passages; extraction returns fields.

> A data science team runs a Snowpark Python job that trains a model over a wide feature table. On a Large standard warehouse it fails with an out-of-memory error.
>
> They resize the warehouse to 2X-Large and rerun. It fails identically. They resize to 4X-Large. It fails identically again, and the credits consumed have risen sharply.

Why is resizing not helping, and what should they do?

A Keep scaling. Memory grows with warehouse size, so 5X-Large or 6X-Large will eventually succeed.

B The Query Acceleration Service should be enabled to offload the memory-intensive portion.

C Enable multi-cluster scaling, which distributes the job's memory across clusters.

D Standard warehouse memory limits do not increase with size. Use a Snowpark-optimized warehouse.

◆ PAUSE AND DECIDE

Why is resizing not helping, and what should they do?

A Keep scaling to 5X-Large or 6X-Large.

What they have already tried twice. Standard warehouse memory limits do not increase with size, so this never succeeds.

B Query Acceleration Service.

QAS offloads scan work for eligible SQL queries. It does not raise the memory ceiling for Snowpark code.

C Multi-cluster scaling.

Multi-cluster adds clusters for concurrency. A single job still runs within one cluster's memory.

D Standard memory does not scale. Use Snowpark-optimized.

Documented directly. Snowpark-optimized warehouses provide more memory per node, which is why the type exists.

THE SIGNAL

Snowpark-optimized memory does not rise above Medium either. Bigger buys cores, not headroom, on both warehouse types.

A team wants to develop a deep-learning model in a Snowflake Notebook. They need a specific open-source library that is not in Snowflake's package list, and the training step requires GPU acceleration.

They also run several large SQL aggregations in the same notebook to build the training set, and are worried these will be slow in a notebook environment.

Which runtime should they choose, and what happens to the SQL?

A Container Runtime. The SQL and Snowpark queries still execute on the warehouse regardless of runtime.

B Warehouse Runtime, because SQL must run on a warehouse and the runtime determines that.

C Container Runtime, but the SQL aggregations will now run on the compute pool and be slower.

D Warehouse Runtime with a Snowpark-optimized warehouse, which provides GPU access.

◆ PAUSE AND DECIDE

Which runtime should they choose, and what happens to the SQL?

A Container Runtime. SQL still runs on the warehouse.

Gives PyPI installs and GPU compute pools, and for both runtimes SQL and Snowpark always execute on the warehouse.

B Warehouse Runtime.

Cannot install arbitrary packages and offers no GPU. Both stated requirements fail.

C Container Runtime, but SQL moves to the pool.

Right runtime, wrong consequence. This is exactly the misconception the documentation corrects.

D Warehouse Runtime with Snowpark-optimized.

Snowpark-optimized warehouses provide more memory, not GPUs. GPUs come from Container Services compute pools.

THE SIGNAL

C is the best distractor: correct choice, invented downside. The runtime governs the notebook's Python, not its SQL.

> A subscription business has seen monthly conversion rate fall sharply. The data has dimensions for channel, region, device, plan tier and campaign. Leadership wants to know which combinations of those dimensions account for most of the fall.
>
> A analyst suggests building a forecasting model to project where conversion will go next.

Which ML function actually answers the question asked?

A Forecasting, as suggested, since it models the metric over time.

B Anomaly Detection, to flag the months where conversion was abnormal.

C Top Insights, which finds the dimensions and values driving the change.

D Classification, to sort customers into converting and non-converting classes.

◆ PAUSE AND DECIDE

Which ML function actually answers the question asked?

A Forecasting.

Projects where the metric is going. Leadership asked why it already moved, which is a different question.

B Anomaly Detection.

Identifies which months were abnormal. They already know the fall happened; they want the cause.

C Top Insights.

Purpose-built for root cause: which dimensions and values are affecting the metric in surprising ways.

D Classification.

Predicts a class per row. It does not attribute a change in an aggregate metric to segments.

NAMING

Top Insights was formerly Contribution Explorer. A question may use either name for the same function.

What to carry into the next module

- **Search is unstructured, Analyst is structured.** Analyst needs a semantic model in YAML.
- **AI_ functions are current**; SNOWFLAKE.CORTEX is legacy. Both granted to PUBLIC by default.
- **Standard warehouse memory does not grow with size.** Out-of-memory means Snowpark-optimized.
- **Container Runtime for GPUs and custom packages.** SQL runs on the warehouse either way.
- **ML functions:** Forecasting, Anomaly Detection, Classification, Top Insights.

> ON THE EXAM
>
> Next: the Domain 1 drill. Eight scenarios across everything from modules 1 to 6, and no new content.

MODULE 07 · DOMAIN 1 - SNOWFLAKE AI DATA CLOUD FEATURES AND ARCHITECTURE

Domain 1 drill – eight worked scenarios

No new content. Everything here comes from modules 1 to 6, in the form the exam uses.

DOMAIN 1 of 5 WEIGHT 31% MODULE 7 of 31

admin@sthithapragna.com

How to use this drill

- **Commit to an answer before I speak.** The pause is longer here than in the teaching modules.
- **Write down why.** A right answer for the wrong reason will not survive a rephrased question.
- Domain 1 is **31%**, so roughly **31 of your 100 questions** come from this material.
- Score yourself. **Six or more of eight** suggests this domain is solid. Fewer, and the last slide says where to go back to.

> ON THE EXAM
>
> Getting one wrong here is worth more than getting it right. It tells you something a passing score would have hidden.

A team runs a small reporting job every three minutes throughout the working day. Each run executes two short queries that complete in about four seconds in total.

The warehouse is X-Small with auto-suspend set to 60 seconds. Finance has noticed the warehouse consumes far more credits than the query durations suggest it should.

What best explains the cost?

A Cloud services credits are being charged because the queries are metadata-heavy.

B X-Small warehouses have a credit floor that makes them inefficient for short queries.

C Each resume bills a 60-second minimum, and auto-suspend keeps the warehouse resuming.

D The result cache is not being used, so every run rescans the underlying tables.

◆ PAUSE AND DECIDE

What best explains the cost?

A Cloud services, metadata-heavy queries.

Cloud services is only charged above 10% of the day's warehouse credits. Warehouse usage here is high, so the allowance is generous.

B X-Small has a credit floor.

Invented. The 60-second minimum applies at every size; X-Small is simply the cheapest per second.

C The 60-second minimum, paid on every resume.

Four seconds of work bills a full minute. Twenty runs an hour pays twenty minutes for roughly 80 seconds of querying.

D Result cache not used.

Would explain slow queries, not expensive idle time. The queries complete in four seconds already.

WHERE THIS CAME FROM

Module 1. Per-second billing with a 60-second minimum on each start or resume. Fix it by lengthening auto-suspend, not by resizing.

A regulator has asked a bank to demonstrate which users read a specific customer table over the past four months, and to show that masking was applied to analysts throughout.

The bank runs Snowflake Standard Edition. An architect proposes querying QUERY_HISTORY in ACCOUNT_USAGE to produce the evidence.

What is wrong with the proposal, and what is needed?

A Nothing is wrong. QUERY_HISTORY records which columns each user read.

B QUERY_HISTORY does not give column-level read detail. ACCESS_HISTORY and masking both require Enterprise.

C ACCOUNT_USAGE retains only 30 days, so the evidence must come from INFORMATION_SCHEMA instead.

D Nothing is wrong, but Business Critical is required because the data is financial.

◆ PAUSE AND DECIDE

What is wrong with the proposal, and what is needed?

A Nothing wrong, QUERY_HISTORY has it.

QUERY_HISTORY records statements. Column-level read detail is what ACCESS_HISTORY exists to provide.

B Needs ACCESS_HISTORY and masking, both Enterprise.

Two separate Enterprise features. The bank cannot produce this evidence on Standard, whatever query it writes.

C ACCOUNT_USAGE keeps only 30 days.

Wrong direction. ACCOUNT_USAGE has long retention; it is INFORMATION_SCHEMA that is the short-window view.

D Business Critical, because financial.

'Financial data' is not a trigger. PCI cardholder data would be; ordinary customer records are not.

WHERE THIS CAME FROM

Modules 2 and 11. ACCESS_HISTORY and Column-level Security are both Enterprise. The edition is the constraint, not the query.

A 30 TB table has a wide text column that is no longer used by any query. A team drops the column, expecting the monthly storage charge for the table to fall noticeably.

Three weeks later the storage figure is essentially unchanged, although the table has had almost no DML in that period.

Why has storage not fallen?

A Time Travel is retaining the pre-drop version, and storage will fall when retention expires.

B The column was moved into Fail-safe and will be released after 7 days.

C Dropped columns are retained until the next automatic clustering pass rewrites the table.

D Dropping a column does not rewrite micro-partitions. The data stays until they are rewritten for another reason.

◆ PAUSE AND DECIDE

Why has storage not fallen?

A Time Travel is holding the old version.

Plausible for a week or two, and three weeks exceeds the 90-day maximum only on Enterprise. But even after expiry, the data would remain.

B It is in Fail-safe for 7 days.

Fail-safe applies to dropped tables and historical versions, not to a column inside a live table. Seven days have long passed.

C Released at the next clustering pass.

Invented mechanism, and the table has no clustering key mentioned. Snowflake will not recluster a table without one.

D Micro-partitions are not rewritten on a column drop.

Documented behaviour. Micro-partitions are immutable, and DROP COLUMN is a metadata change that leaves the stored data in place.

WHERE THIS CAME FROM

Module 3. Immutability cuts both ways: it is why Time Travel works, and why dropping a column reclaims nothing.

A provider has built a view that joins three internal tables and filters to a single partner's rows. They add the view to a share and grant it to the partner account.

Creating the share succeeds, but adding the view to it fails. The provider is on Enterprise Edition and the underlying tables are permanent.

What is the most likely cause?

A The view is not secure. Only secure views can be added to a share.

B Views cannot be shared at all. Only tables and materialized views can.

C Views that join more than two tables cannot be shared, so it must be flattened first.

D The provider must upgrade to Business Critical before any object can be shared.

◆ PAUSE AND DECIDE

What is the most likely cause?

A The view is not secure.

Only secure views can be shared. A non-secure view would leak its definition, and the optimiser could leak the rows it filters out.

B Views cannot be shared.

Secure views are one of the standard things you share. Sharing a filtered view is the usual pattern.

C A three-table join is too many.

Invented limit. Materialized views cannot join; ordinary and secure views can join freely.

D Business Critical is required to share.

Secure Data Sharing is Standard Edition. Business Critical adds failover, not sharing.

WHERE THIS CAME FROM

Module 4. Secure is a property, not a view type, and it is the property that makes a view shareable.

> An application needs to record customer sessions, with single-row lookups and updates by session id arriving many times per second. It requires that session ids are unique and that a foreign key to the customers table is enforced by the database.
>
> The team also wants to run analytical queries over the same session data alongside their existing warehouse tables.

Which table type fits?

A. A permanent table with a clustering key on session_id.

B. An Iceberg table with Snowflake as the catalog.

C. A hybrid table.

D. A transient table with the Search Optimization Service enabled.

◆ PAUSE AND DECIDE

Which table type fits?

A Permanent table, clustered on session_id.

Ordinary Snowflake tables do not enforce unique or foreign key constraints. A unique id is also the classic bad clustering key.

B Iceberg with Snowflake as catalog.

An analytical open table format. No enforced constraints and no row-level locking for high-frequency single-row writes.

C A hybrid table.

Enforced primary key, unique and referential constraints, indexes and row-level locking. This is exactly what Unistore is for.

D Transient plus search optimization.

Search optimization speeds point lookups but enforces nothing, and transient adds a data-loss risk nobody asked for.

WHERE THIS CAME FROM

Module 4. 'Enforced by the database' is the tell. Ordinary Snowflake tables do not enforce constraints other than NOT NULL.

A new analyst cannot connect to Snowflake from a client on the corporate network. The connection times out. Colleagues on the same network connect without difficulty, and the analyst can sign in to Snowsight in a browser.

The team suspects a proxy or firewall issue but is not certain which host or port is being blocked.

What is the most appropriate first step?

A. Switch the analyst to the SQL API, which uses HTTPS and avoids proxy problems.

B. Run SnowCD, the connectivity diagnostic tool, to identify what is failing.

C. Add the analyst's IP address to a network policy allowed list.

D. Reinstall SnowSQL, since automatic upgrades were disabled from version 1.3.0.

◆ PAUSE AND DECIDE

What is the most appropriate first step?

A. Switch to the SQL API.

Works around the symptom without diagnosing it, and changes the analyst's tooling to avoid a problem nobody has identified.

B. Run SnowCD.

SnowCD exists for exactly this. It is a diagnostic tool rather than a way to connect, and it reports which hosts and ports are unreachable.

C. Add the IP to a network policy.

A network policy blocks with an explicit denial, not a timeout, and colleagues on the same network connect fine.

D. Reinstall SnowSQL.

True that automatic upgrades were disabled from 1.3.0, but nothing suggests a version problem. A true fact that answers nothing.

WHERE THIS CAME FROM

Module 5. SnowCD is the odd one out in the connectivity table: not a way to connect, but a way to find out why you cannot.

> A security review finds that every user in an account has been calling Cortex AI functions against tables containing customer correspondence. Nobody recalls enabling Cortex, and the security team assumed it was off by default.
>
> They want only a single data science role to retain access.

What happened, and what should they do?

A Someone granted CORTEX_USER to PUBLIC. Revoke that grant from the individual users.

B Cortex inherits the ACCOUNTADMIN role by default. Restrict ACCOUNTADMIN membership.

C Cortex is enabled per warehouse. Disable it on all warehouses except the data science one.

D CORTEX_USER and USE AI FUNCTIONS are granted to PUBLIC by default. Revoke from PUBLIC, then grant to the data science role.

◆ PAUSE AND DECIDE

What happened, and what should they do?

A Someone granted it. Revoke from the users.

Nobody granted anything; it is the default. And you cannot revoke a database role from a user, because it was never granted to one.

B Cortex inherits ACCOUNTADMIN.

Invented. Access comes from a database role plus an account privilege, both held via PUBLIC.

C Enabled per warehouse.

Warehouses supply compute. They are not an access control boundary for any feature.

D Both are granted to PUBLIC by default. Revoke, then grant.

The documented default and the documented remedy. PUBLIC is automatically granted to every user and role.

WHERE THIS CAME FROM

Module 6. Nobody switched Cortex on. It was on from account creation, because PUBLIC holds both grants.

> A compliance policy requires that a particular table can be restored to any point within the past 30 days. The table is currently transient, and the account is on Standard Edition.
>
> An engineer runs ALTER TABLE … SET DATA_RETENTION_TIME_IN_DAYS = 30 and reports that the requirement is now met.

Is the requirement met, and what is actually needed?

A. No. Two changes are needed: a permanent table, and Enterprise Edition or higher.

B. Yes. Retention can be set to 30 days on any table type in any edition.

C. No. The table must become permanent, but Standard Edition already supports 30 days.

D. No. Business Critical is required, because 30-day retention is a compliance feature.

◆ PAUSE AND DECIDE

Is the requirement met, and what is actually needed?

A Two changes: permanent table, and Enterprise.

Transient tops out at 1 day whatever the edition, and Standard tops out at 1 day whatever the table type. Both limits bind.

B Yes, 30 days works anywhere.

What the engineer believes. The statement may not even error, but the retention will not take the requested value.

C Permanent is enough on Standard.

Half right. Standard permanent tables are still capped at 1 day; 90 days is the Enterprise figure.

D Business Critical required.

Business Critical adds encryption control, private connectivity and failover. It changes nothing about Time Travel retention.

WHERE THIS CAME FROM

Modules 2 and 4 together. Two independent ceilings, and fixing only one of them leaves you at 1 day.

Where each question came from

QUESTION	MODULE TO REVISIT IF YOU MISSED IT
1. The 60-second minimum	Module 1 - three layers, and the billing meters
2. ACCESS_HISTORY and masking	Module 2 - editions, the Enterprise cluster
3. Dropped column, unchanged storage	Module 3 - micro-partitions and immutability
4. Sharing a view	Module 4 - secure is a property, not a type
5. Enforced constraints	Module 4 - the other four table types
6. SnowCD	Module 5 - drivers, connectors and the SQL API
7. Cortex granted to PUBLIC	Module 6 - who is allowed to use Cortex
8. Thirty days of Time Travel	Modules 2 and 4 together

ON THE EXAM

Two or more misses clustered in one module is a signal. Two misses spread across six is normal and not worth chasing.

What this drill was really testing

- **Constraints that stack.** Question 8 had two independent ceilings. Fixing one is the commonest way to lose the mark.
- **True statements that answer nothing.** Questions 1 and 6 both offered one. Check relevance, not just accuracy.
- **Defaults nobody chose.** Cortex on PUBLIC, permanent as the default table type, load order as natural clustering.
- **The phrase that decides it.** 'Enforced by the database', 'shown the relevant passages', 'without changing who owns the writes'.

> ON THE EXAM
>
> Next: Domain 2. Account management and governance, at 20%. Starting with the object hierarchy and RBAC.

MODULE 08 · DOMAIN 2 - ACCOUNT MANAGEMENT AND DATA GOVERNANCE

The object hierarchy and the RBAC model

Domain 2 begins. The role hierarchy diagram is the highest-value slide in this domain.

DOMAIN 2 of 5 WEIGHT 20% MODULE 8 of 31

admin@sthithapragna.com

Privileges are granted to **roles**. Roles are granted to **users**. Almost nothing is granted to a user directly.

The object hierarchy

01

Organization

One or more accounts, managed together. Billing and account lifecycle.

02

Account

One cloud provider, one region. Its own users, roles and warehouses.

03

Database

A logical container. Nothing is stored at this level.

04

Schema

Contains the objects. Tables, views, stages, functions, procedures.

ON THE EXAM

Every securable database object lives in a schema, inside a database. That containment is what makes USAGE privileges necessary.

Two access control models, working together

- **Discretionary Access Control.** Every object has an **owner**, and the owner can grant access to what it owns.
- **Role-Based Access Control.** Privileges are assigned to roles, and roles are assigned to users or to other roles.
- Snowflake uses **both**. Ownership decides who may grant; roles decide who may act.
- Newer: **user-based access control** lets a privilege be granted to a user. It only takes effect when **all secondary roles are enabled**.

> ON THE EXAM
>
> Disable UBAC if it complicates governance: ALTER ACCOUNT SET DISABLE_USER_PRIVILEGE_GRANTS = TRUE.

The system-defined roles

ROLE	WHAT IT IS FOR
ORGADMIN	Organization-level tasks. Account lifecycle and organization usage. Outside the account hierarchy.
ACCOUNTADMIN	Encapsulates SYSADMIN and SECURITYADMIN. Top-level. Account parameters, billing, killing queries.
SECURITYADMIN	Global MANAGE GRANTS: can grant or revoke on any object. Inherits USERADMIN.
USERADMIN	CREATE USER and CREATE ROLE. User and role management only.
SYSADMIN	Create warehouses, databases and all database objects.
PUBLIC	Pseudo-role granted automatically to every user and every role.

ON THE EXAM

Read the two middle rows carefully. USERADMIN creates users and roles. SECURITYADMIN grants privileges. Different jobs.

The hierarchy, bottom to top

01

USERADMIN

Granted to SECURITYADMIN.

02

SECURITYADMIN

Granted to ACCOUNTADMIN.

03

SYSADMIN

Also granted to ACCOUNTADMIN, in parallel.

04

ACCOUNTADMIN

Inherits both branches. ORGADMIN sits outside entirely.

ON THE EXAM

Two branches meeting at the top. SECURITYADMIN inherits USERADMIN, so it can do everything USERADMIN can. SYSADMIN does not.

ACCOUNTADMIN is not a superuser

- The documentation says this explicitly. **ACCOUNTADMIN is not a superuser role.**
- It can view and manage an object **only if it, or a role beneath it in a hierarchy, holds sufficient privileges** on that object.
- So an object owned by a custom role that is **not** connected to the hierarchy is invisible to ACCOUNTADMIN too.
- Do not make it anyone's **default role**, and do not use it in **automated scripts**. Both are documented recommendations.
- Snowflake recommends designating **at least two** account administrators.

> ON THE EXAM
>
> Forcing an explicit USE ROLE ACCOUNTADMIN is the point of the default-role advice. It makes the elevation deliberate.

Which role does what: the trap

THE TASK	THE ROLE
Create a user, or create a role	USERADMIN (or higher)
Grant a privilege on an object you do not own	SECURITYADMIN, via MANAGE GRANTS
Create a warehouse, database, schema or table	SYSADMIN (or a custom role with the privilege)
Change an account-level parameter	ACCOUNTADMIN only
View billing and credit usage, or kill a running query	ACCOUNTADMIN only
Create a new account in the organization	ORGADMIN

ON THE EXAM

A question that needs BOTH a user created and a warehouse created needs two roles, or ACCOUNTADMIN. That is a real question shape.

Custom roles, and the SYSADMIN rule

- Build a hierarchy of custom roles aligned to business functions, and grant the **top one to SYSADMIN**.
- If a custom role is **not** assigned to SYSADMIN, **system administrators cannot manage the objects it owns**.
- Objects are owned by **the role active when they were created**. Create under the wrong role and ownership goes to it.
- Ownership can be **transferred** with GRANT OWNERSHIP.

> ON THE EXAM
>
> Creating objects while using ACCOUNTADMIN is the classic accident. The objects end up owned by a role nobody should be using daily.

The commonest access failure

- To query a table you need **SELECT on the table**, plus **USAGE on the schema**, plus **USAGE on the database**.
- You also need **USAGE on a warehouse** to run the query at all.
- Missing container USAGE produces 'I can see it but I cannot query it', or the object appearing not to exist.
- This follows directly from containment. Objects live in schemas, and schemas live in databases.

> ON THE EXAM
>
> Count the levels when a scenario says access was granted and still fails. There are usually three, plus the warehouse.

Future grants

- Define privileges now on objects of a type that **do not exist yet**, in a schema or a database.
- **GRANT SELECT ON ALL TABLES IN SCHEMA covers existing tables only.** ON FUTURE is a separate statement.
- The global **MANAGE GRANTS privilege is required** to grant on future objects in a database.
- **Schema-level future grants take precedence** over database-level ones, and the database-level grants are then ignored.
- **Future grants are not applied when renaming or swapping a table.**

> ON THE EXAM
>
> 'New tables keep being invisible to the analysts' is the scenario. ON ALL was run once; ON FUTURE never was.

Managed access schemas

- Created with **CREATE SCHEMA … WITH MANAGED ACCESS**.
- **Object owners lose the ability to make grant decisions** inside that schema.
- Only the **schema owner**, or a role with **MANAGE GRANTS**, can grant on objects in it, including future grants.
- The point is **centralising privilege management**, so grants cannot proliferate quietly.

> ON THE EXAM
>
> This is the one place Snowflake deliberately switches off the discretionary half of the model.

Primary and secondary roles

- A session has **one primary role**, set with USE ROLE. **It owns anything you create.**
- **Secondary roles** add their privileges to the session without changing ownership.
- **USE SECONDARY ROLES ALL** activates every role the user holds. **NONE** disables them.
- Useful when a query joins objects governed by different roles, which would otherwise need a combined role.

> ON THE EXAM
>
> Secondary roles never own anything. If a scenario asks who owns a new table, it is the primary role at creation time.

Account roles and database roles

- **Account roles** are what we have been discussing. They exist across the account and can be activated in a session.
- **Database roles** are **scoped to a single database** and hold privileges only on objects in it.
- **A database role cannot be activated directly in a session.** Grant it to an account role, then use that.
- That rule explains something from module 6: SNOWFLAKE.CORTEX_USER is a database role, which is why it cannot be granted to a user.

> ON THE EXAM
>
> Database roles travel with the database. That makes them the natural unit for sharing and for native apps.

ORGADMIN is being replaced

- **ORGADMIN** is the role that uses a regular account to manage organization-level operations. It is what the exam will ask about.
- The documentation now states that **ORGADMIN will be phased out in a future release**.
- Its replacement is **GLOBALORGADMIN**, which exists **only in the organization account**, a dedicated account for organization administration.
- **Answer ORGADMIN.** The January 2026 study guide predates the transition.

> ON THE EXAM
>
> Recognise GLOBALORGADMIN so it does not throw you, but do not expect it to be the keyed answer.

A new data engineer is being onboarded. The onboarding runbook requires creating a Snowflake user for them, and a custom role for the engineering team. It also requires a dedicated virtual warehouse for their workload.

An administrator attempts all three while using the SECURITYADMIN role. The first two succeed and the third fails.

Why did the third step fail, and what is the correct approach?

A SECURITYADMIN cannot create warehouses. Use SYSADMIN for that step, or a custom role with the privilege.

B Warehouse creation requires ACCOUNTADMIN, because warehouses consume credits.

C SECURITYADMIN must first be granted to SYSADMIN so it inherits object creation privileges.

D The administrator must enable secondary roles, which would have granted the missing privilege automatically.

◆ PAUSE AND DECIDE

Why did the third step fail, and what is the correct approach?

A SECURITYADMIN cannot create warehouses. Use SYSADMIN.

SECURITYADMIN inherits USERADMIN, which is why the first two steps worked. It has no object-creation branch at all.

B Warehouse creation needs ACCOUNTADMIN.

SYSADMIN creates warehouses. ACCOUNTADMIN would work only because it inherits SYSADMIN, which makes it the wrong answer to a 'correct approach' question.

C Grant SECURITYADMIN to SYSADMIN.

Inverts the hierarchy. Granting a role to another gives the grantee the privileges, so this would give SYSADMIN security powers instead.

D Enable secondary roles.

Secondary roles activate roles the user already holds. If they were never granted SYSADMIN, nothing appears.

THE SIGNAL

The first two steps succeeding is the clue. SECURITYADMIN inherits USERADMIN, so user and role creation work. Warehouses are the other branch.

> An analyst role has been granted SELECT on a table in SALES.REPORTING. The analyst has USAGE on a warehouse and can log in normally.
>
> When they run a query against the table, Snowflake reports that the object does not exist or is not authorised. The table certainly exists, and the grant is visible in SHOW GRANTS.

What is the most likely cause?

A The table is in a managed access schema, which blocks all SELECT grants made by object owners.

B The role is missing USAGE on the SALES database, the REPORTING schema, or both.

C The grant was made with a future grant, which does not apply to tables that already exist.

D The analyst needs their secondary roles enabled before any SELECT grant takes effect.

◆ PAUSE AND DECIDE

What is the most likely cause?

A. Managed access schema blocks the grant.

Managed access restricts who may grant, not whether a granted privilege works. And the grant is visible in SHOW GRANTS.

B. Missing USAGE on the database, the schema, or both.

Objects are contained in schemas, inside databases. USAGE on both containers is required in addition to SELECT.

C. It was a future grant.

Genuinely true that future grants do not apply to existing tables. But then the grant would not appear in SHOW GRANTS for this table.

D. Secondary roles must be enabled.

That applies to privileges granted directly to a user under UBAC. This was granted to a role.

THE SIGNAL

The misleading error message is the point. Missing container USAGE often reports as 'does not exist' rather than 'access denied'.

A nightly pipeline creates several new tables in ANALYTICS.CURATED each week. A platform engineer ran GRANT SELECT ON ALL TABLES IN SCHEMA ANALYTICS.CURATED TO ROLE reporting three months ago.

Reporting users can query the older tables but not any table created since. The engineer is using a custom role that owns the schema but does not hold MANAGE GRANTS.

What should be done?

A Re-run the ON ALL grant after each pipeline run, since Snowflake has no mechanism for future objects.

B Convert the schema to a managed access schema, which automatically grants new objects to existing grantees.

C Add a future grant on the schema. The schema owner can do this without MANAGE GRANTS.

D Grant OWNERSHIP of the pipeline to the reporting role so it inherits access to what it creates.

◆ PAUSE AND DECIDE

What should be done?

A Re-run ON ALL after every run.

Correctly diagnoses that ON ALL covers existing tables only, then denies that future grants exist. They do.

B Convert to a managed access schema.

Managed access centralises who may grant. It does not create grants, so new tables would still be inaccessible.

C Add a future grant on the schema.

Exactly what future grants are for. MANAGE GRANTS is required for future grants on a database; the schema owner can do it at schema level.

D Grant OWNERSHIP of the pipeline.

Ownership of a task or procedure does not confer access to tables it creates. It would also make reporting the owner of production objects.

THE SIGNAL

MANAGE GRANTS is required for future grants at DATABASE level. The scenario mentions the missing privilege to test whether you know the difference.

> An engineer signed in, left their role as ACCOUNTADMIN, and created a database and several tables for a new project. A custom role hierarchy exists, with a top-level role granted to SYSADMIN.
>
> The data team, working under roles beneath SYSADMIN, now cannot see or manage any of the new objects. A team lead argues that ACCOUNTADMIN is the highest role, so its objects should be visible to everyone below it.

Why is the team lead wrong, and what is the fix?

A ACCOUNTADMIN objects are hidden by design. The database must be dropped and recreated under SYSADMIN.

B The team roles need SELECT granted on each new table individually. Ownership is irrelevant to visibility.

C Privileges flow downward from senior roles. The team simply needs their sessions restarted.

D Privileges are inherited upward, not downward. Transfer ownership to a role in the hierarchy with GRANT OWNERSHIP.

◆ PAUSE AND DECIDE

Why is the team lead wrong, and what is the fix?

A Hidden by design. Drop and recreate.

Not hidden by design, and dropping a populated database to fix a grant is drastic when ownership can simply be transferred.

B Grant SELECT on each table. Ownership is irrelevant.

Would restore reading, and leaves ACCOUNTADMIN owning production objects. Ownership decides who can grant and alter.

C Privileges flow downward. Restart sessions.

States the team lead's misconception. Inheritance runs upward: a parent role inherits from roles granted to it.

D Inheritance is upward. Transfer ownership.

ACCOUNTADMIN is not a superuser, and it inherits from roles granted to it rather than pushing privileges down.

THE SIGNAL

Objects are owned by the role active at creation. This is why 'do not make ACCOUNTADMIN a default role' is documented advice.

What to carry into the next module

- **USERADMIN to SECURITYADMIN to ACCOUNTADMIN**, with **SYSADMIN** as a parallel branch. ORGADMIN sits outside.
- **ACCOUNTADMIN is not a superuser.** Inheritance runs upward, and objects are owned by the role active at creation.
- **SELECT plus USAGE on the schema plus USAGE on the database**, plus a warehouse. Count the levels.
- **ON ALL is existing objects. ON FUTURE is new ones.** Schema-level future grants beat database-level.
- **Managed access:** owners stop granting. **Database roles** cannot be activated directly.

> ON THE EXAM
>
> Next: authentication, network policies and encryption. The area where this course is most obviously newer than its competitors.

MODULE 09 · DOMAIN 2 - ACCOUNT MANAGEMENT AND DATA GOVERNANCE

Authentication, network policies and encryption

Who you are, where you are connecting from, and whether the data is readable at all.

DOMAIN 2 of 5 WEIGHT 20% MODULE 9 of 31

admin@sthithapragna.com

How a user proves who they are

METHOD	NOTES
Password	Governed by password policies. No longer sufficient alone for human users.
Multi-factor	Duo, authenticator apps (TOTP), or passkeys. A second factor, never a replacement.
Federated / SSO	SAML 2.0 via an identity provider. Snowflake is the service provider.
OAuth	Snowflake OAuth, or External OAuth via your own authorisation server.
Key-pair	User holds the private key, Snowflake the public key. Standard for programmatic access.
Programmatic access token	Used in place of a password by tools that cannot do MFA.

ON THE EXAM

All Standard Edition. Authentication is never an edition question.

User types now matter

- **TYPE = PERSON.** A human being, signing in interactively. Subject to MFA enforcement.
- **TYPE = SERVICE.** A programmatic identity. **Exempt from MFA enforcement**, and cannot use a password.
- **TYPE = LEGACY_SERVICE.** A transitional type for service accounts still using passwords.
- Service users authenticate with **key-pair, OAuth, a programmatic access token, or workload identity federation**.
- Snowflake plans that no new LEGACY_SERVICE users can be created from mid-2026.

> ON THE EXAM
>
> This is the modern answer to 'how should an automated pipeline authenticate'. Older courses say key-pair and stop.

MFA is being enforced, not merely offered

- Snowflake is **blocking single-factor password sign-ins for human users**, rolled out through behaviour change bundles.
- **SSO and OAuth users are not affected.** The enforcement targets password sign-ins to Snowsight.
- Excluded from the rollout: **managed accounts, trial accounts and reader accounts**.
- MFA methods now include **authenticator apps** and **passkeys**, alongside Duo.
- **A passkey is a second factor**, not a replacement for the username and password.

> ON THE EXAM
>
> The exam still tests the classic list. Know that list, and know that 'password only' is no longer a viable answer for a human.

Authentication policies tie it together

- A schema-level object controlling **which authentication methods are permitted**, applied to an account or a user.
- Controls include **MFA_ENROLLMENT**, **CLIENT_TYPES**, **AUTHENTICATION_METHODS** and **PAT_POLICY**.
- By default, a user must be subject to a **network policy** before they can generate or use a **programmatic access token**.
- Password policies remain separate, governing length, complexity, age and retry limits.

> ON THE EXAM
>
> That PAT-and-network-policy dependency is a neat design: a token that works from anywhere is a password with extra steps.

Network rules and network policies

NETWORK RULE

A schema-level object

- Lives in a database and schema, like a table
- Holds identifiers in a VALUE_LIST
- MODE is INGRESS or EGRESS
- TYPE is IPV4, IPV6, AWSVPCEID, AZURELINKID, HOST_PORT

NETWORK POLICY

An account-level object

- Not contained in a database at all
- Holds rules in an allowed list and a blocked list
- Does nothing until activated on something
- Applied to an account, a security integration, or a user

THE TIEBREAKER

The object level is the distinction to memorise. Rules live in schemas; policies do not.

How the lists are evaluated

- **Precedence: user beats security integration beats account.** The most specific policy wins.
- **The blocked list is applied first.** An address in both lists is blocked.
- **Private connectivity rules take precedence** over IPv4 and IPv6 rules for a request arriving that way.
- **An empty network policy allows nothing.** Creating one with no allowed entries locks everybody out.
- To block all public access, put a rule containing **0.0.0.0/0 in the blocked list**, leaving only private connectivity.

> ON THE EXAM
>
> Always confirm your own access is allowed before applying a blocking policy. Locking out every administrator is a support ticket.

The encryption key hierarchy

01

Root key

Lives in a hardware security module and never leaves it.

02

Account master key

One per account. Wrapped by the root key.

03

Table master key

One per table. Wrapped by the account master key.

04

File key

Derived per file. Encrypts the actual micro-partition data.

ON THE EXAM

Each level narrows the scope. That containment is the stated purpose: one key protects less data, for less time.

Key rotation or rekeying?

KEY ROTATION

Automatic, every 30 days

- Account and table master keys over 30 days old
- Old key retired, new key created
- Retired keys used only for decryption
- Existing data is NOT re-encrypted

PERIODIC REKEYING

Optional, annual

- Data protected by retired keys IS re-encrypted
- New keys created; old ones destroyed
- Enterprise Edition or higher
- Must be enabled by an account administrator

THE TIEBREAKER

Rotation changes which key encrypts new data. Rekeying changes the key protecting data that already exists.

Tri-Secret Secure

- **Business Critical.** A Snowflake-maintained key combined with a **customer-managed key** in your cloud provider's KMS.
- Together they form a **composite master key**, which acts as the account master key and wraps the keys below it.
- The composite key **never encrypts raw data directly**. It wraps table master keys, which derive file keys.
- **If the customer-managed key is revoked, Snowflake can no longer decrypt the data.** That is the feature, and the risk.
- Only **one** CMK at a time. Deleting an older CMK version causes data loss.

> ON THE EXAM
>
> The genuine use case is a kill switch: revoke your key and the data becomes unreadable, including to Snowflake.

How this module gets tested

THE QUESTION SHAPE	WHAT IT IS REALLY ASKING
'How should this automated process authenticate?'	Key-pair, OAuth, PAT or WIF. Never a password.
'Only our office should reach the account'	Network rule plus network policy, activated.
'A user can connect despite the account policy'	Precedence. A user-level policy overrode it.
'We must re-encrypt existing data annually'	Periodic rekeying. Enterprise. Not rotation.
'We need to be able to make the data unreadable'	Tri-Secret Secure. Business Critical.
'Which key encrypts the micro-partition?'	The file key. Not the account or table master key.

ON THE EXAM

Six shapes cover this module. The middle two are where careless reading costs marks.

An orchestration tool runs unattended overnight and connects to Snowflake using a username and password. After a recent change, its connections have begun failing during the sign-in step, while the team's interactive Snowsight sessions are unaffected.

The team proposes storing an MFA push approval in the tool's configuration so it can complete the second factor automatically.

What is the correct approach?

A Store the MFA approval as proposed, which is the documented pattern for unattended tools.

B Create the identity as TYPE = SERVICE and authenticate with key-pair, OAuth, a PAT or workload identity federation.

C Move the tool to SSO, so that federated authentication bypasses the requirement.

D Grant the tool's user the ACCOUNTADMIN role, which is exempt from authentication policies.

◆ PAUSE AND DECIDE

What is the correct approach?

A. Store the MFA approval in config.

Not a documented pattern, and storing a second factor next to the first defeats the point of having two.

B. TYPE = SERVICE with key-pair, OAuth, PAT or WIF.

Service users are exempt from MFA enforcement and cannot use passwords. This is the documented answer for programmatic access.

C. Move the tool to SSO.

SSO users are genuinely exempt from the Snowsight enforcement. But SSO is an interactive browser flow, which an unattended job cannot complete.

D. Grant ACCOUNTADMIN for exemption.

Roles do not exempt anyone from authentication. It also violates the documented advice against ACCOUNTADMIN in automation.

THE SIGNAL

'Interactive sessions unaffected' is the clue: this is MFA enforcement on password sign-ins, and only human users are targeted.

An account-level network policy allows only the corporate office range. A contractor working from home reports that they can still connect normally, which the security team did not expect.

Investigation shows a second network policy exists, listing the contractor's home address in its allowed list, and it is applied to their user.

Why can the contractor connect, and what governs this?

A Both policies apply, and Snowflake takes the union of every allowed list in the account.

B The account policy is invalid because a second policy exists. Only one policy may be active per account.

C A user-level network policy overrides an account-level one. The most specific policy wins.

D Network policies only apply to interactive sign-ins, and the contractor is using a driver.

◆ PAUSE AND DECIDE

Why can the contractor connect, and what governs this?

A Snowflake takes the union of allowed lists.

There is no union. Precedence selects one policy, and the others do not contribute.

B Only one policy may be active per account.

Policies can be applied at three levels simultaneously. That is exactly why precedence rules exist.

C User-level overrides account-level.

Documented precedence: user beats security integration beats account. The account policy is not consulted for this user.

D Policies only apply to interactive sign-ins.

They apply to all connections. Different authentication methods do not escape network policy.

THE SIGNAL

The account policy is not weakened, it is simply not consulted for that user. Audit user-level policies, not just the account one.

> An internal audit requires that data at rest is re-encrypted with newly generated keys at least once a year. It also requires that keys older than one year are destroyed rather than merely retired.
>
> An architect states that Snowflake already rotates keys every 30 days, so the requirement is satisfied automatically. The account is on Enterprise Edition.

Is the architect correct, and what is required?

A. Correct. Rotation every 30 days re-encrypts all existing data, which exceeds the requirement.

B. Incorrect. Tri-Secret Secure must be enabled, which requires an upgrade to Business Critical.

C. Incorrect. Rekeying is not available in Snowflake, so the requirement must be waived.

D. Incorrect. Rotation does not re-encrypt existing data. Periodic rekeying must be enabled by an account administrator.

◆ PAUSE AND DECIDE

Is the architect correct, and what is required?

A Rotation re-encrypts everything.

The architect's belief, and the whole point of the question. Rotation retires a key and encrypts new data; existing files keep their old key.

B Tri-Secret Secure, needing Business Critical.

Tri-Secret Secure is about who controls the key, not about re-encrypting data. The audit said nothing about customer-managed keys.

C Rekeying is not available.

It is. Periodic rekeying is a documented Enterprise feature, and the account is already on Enterprise.

D Rotation does not re-encrypt. Enable periodic rekeying.

Annual re-encryption of data under retired keys, with old keys destroyed. Enterprise, and enabled by an account administrator.

THE SIGNAL

Rotation and rekeying sound like the same word. Rotation changes the key for new data; rekeying changes it for old data.

> A defence contractor requires the ability to render all of its Snowflake data permanently unreadable within minutes. That includes unreadable to Snowflake itself, in the event of a confirmed compromise. No support ticket or vendor action may be needed.
>
> They currently run Enterprise Edition with periodic rekeying enabled.

What should they implement?

A Tri-Secret Secure on Business Critical. Revoking the customer-managed key makes the data undecryptable.

B A network policy blocking all public access, which prevents any further reads of the data.

C Trigger an immediate rekeying operation, which destroys the previous keys and blocks access.

D Set Time Travel to 0 and drop the databases, which sends the data straight into Fail-safe.

◆ PAUSE AND DECIDE

What should they implement?

A Tri-Secret Secure. Revoke the customer-managed key.

The composite master key needs both halves. Revoke your half and Snowflake cannot decrypt anything, with no vendor action needed.

B Block all public access.

Stops connections. The data remains perfectly readable to anyone who does get in, and to Snowflake.

C Trigger an immediate rekeying.

Rekeying re-encrypts data with new keys that Snowflake still holds. The data stays readable throughout, by design.

D Time Travel 0, then drop the databases.

Destroys the data rather than protecting it, and it sits in Fail-safe for 7 days where only Snowflake can reach it.

THE SIGNAL

'Including to Snowflake itself' and 'no vendor action' are the two phrases. Only a key you hold satisfies both.

What to carry into the next module

- **Service users are exempt from MFA and cannot use passwords.** Key-pair, OAuth, PAT or WIF.
- **Network rules are schema objects; network policies are account objects** and do nothing until activated.
- **User beats security integration beats account.** Blocked list is applied first.
- **Root, account master, table master, file.** The file key encrypts the data.
- **Rotation is automatic and does not re-encrypt. Rekeying is annual, optional, Enterprise.** Tri-Secret Secure is Business Critical.

> ON THE EXAM
>
> Next: masking, row access policies, tags, classification and lineage. The governance half of Domain 2.

MODULE 10 · DOMAIN 2 - ACCOUNT MANAGEMENT AND DATA GOVERNANCE

Masking, row access, tags, classification and lineage

The governance half of Domain 2. Where the Enterprise policy family from module 2 finally gets used.

DOMAIN 2 of 5 WEIGHT 20% MODULE 10 of 31

admin@sthithapragna.com

A policy is an **object**, created once and **attached** to many things. It is not a property of a column.

The policy family

POLICY	WHAT IT CONTROLS
Masking	Rewrites a column value at query time. The stored data is unchanged.
Row access	Which rows a query returns at all.
Projection	Whether a column may appear in the output. It can still be joined on.
Aggregation	Forces queries to aggregate over a minimum group size.
Join	Forces a table to be joined before it can be read.

ON THE EXAM

All five are schema-level objects and all five need Enterprise. Masking and row access are the two you will be asked about most.

Masking: the part people miss

- Applied **at query runtime**. The underlying data is never altered, and different roles see different results from the same query.
- **Applied wherever the column is referenced.** Not just the SELECT list: **JOIN predicates, WHERE, ORDER BY and GROUP BY**.
- That is deliberate. It prevents de-anonymisation by filtering or joining on a column you cannot see.
- Documented consequence: a query touching masked columns **may not give the anticipated result**, because the mask blocks proper evaluation.
- Work on **tables and views**. Use **POLICY_CONTEXT** to simulate a query as another role.

> ON THE EXAM
>
> 'The filter returns no rows even though I know they exist' is a masking symptom, not a data problem.

Conditional masking and row access policies

- **Conditional masking** lets the policy read *another column* to decide. Mask the email unless a consent flag is set.
- A **row access policy** is attached to a **table or view**, not to a column, and filters rows out entirely.
- It typically compares the current role, or a mapping table, against a column value on the row.
- Rows removed by a row access policy **are not counted** in aggregations over the table.

> ON THE EXAM
>
> Columns or rows. Masking changes what a value looks like; a row access policy decides whether the row is there at all.

The documented order of evaluation

01

Row access

Filter rows out. Everything downstream sees only what survives.

02

Projection

Check whether any restricted column is being projected. If so, reject the query.

03

Masking

Apply column masks to what remains.

ON THE EXAM

Rows first, then whether the column may be shown at all, then what the value looks like. Each step narrows what the next one sees.

Projection policy or masking policy?

MASKING POLICY

The column appears, changed

- Value is rewritten at query time
- The column is still in the result set
- Applied everywhere the column is referenced
- 'Show the last four digits only'

PROJECTION POLICY

The column cannot appear

- The query is rejected if the column is projected
- The column can still be joined on or grouped by
- One projection policy per column at a time
- 'Join on the hash, never display it'

A DETAIL WORTH KEEPING

Projection policies are ignored in nested queries. They govern only what reaches the final result.

Object tagging

- A **tag** is a key/value label attached to tables, columns, schemas, databases, warehouses and more.
- **Tag lineage:** child objects **inherit tags from their parent**. Tag a schema and every table and column beneath it carries the tag.
- **ACCOUNT_USAGE.TAGS** catalogues every tag in the account, current and deleted.
- **TAG_REFERENCES_ALL_COLUMNS** returns every tag on every column of a table or view.

> ON THE EXAM
>
> Tags are also how cost gets attributed. Tag a warehouse by cost centre and the credits become attributable.

Tag-based masking: the scaling answer

- Attach a masking policy **to a tag** rather than to a column.
- Every column carrying that tag is **automatically protected**. No per-column attachment.
- Combined with classification, **newly discovered sensitive columns are masked as the tag is applied**.
- Change the policy once and every tagged column changes with it.

> ON THE EXAM
>
> 'We have hundreds of tables with email columns and need consistent masking' has exactly one good answer, and this is it.

Sensitive data classification

- Every column found to be sensitive gets **two** categories, and the exam expects both names.
- **Semantic category:** the *type* of attribute. Name, email address, credit card number.
- **Privacy category:** the *sensitivity*. **IDENTIFIER, QUASI_IDENTIFIER, or SENSITIVE.**
- Applied as **system tags** in the SNOWFLAKE.CORE schema: SEMANTIC_CATEGORY and PRIVACY_CATEGORY.
- A **tag map** can translate a system tag into your own user-defined tag automatically.

> ON THE EXAM
>
> Semantic is what the data is. Privacy is how dangerous it is. A postcode is a QUASI_IDENTIFIER: not identifying alone, but identifying in combination.

Classification profiles, and the defaults

- A **classification profile** holds the settings. **auto_tag** decides whether tags are applied or merely recommended.
- Default behaviour: **reclassifies every 30 days**, scans all native semantic categories.
- **Views are excluded** from classification.
- Classification samples **up to 10,000 randomly selected rows per table**. It does not read everything.
- Cost appears in METERING_HISTORY under service type **SENSITIVE_DATA_CLASSIFICATION**.

> ON THE EXAM
>
> Sampling matters. A sensitive value appearing in a small fraction of rows may not be found. Classification assists judgement, it does not replace it.

Lineage, Trust Center and Horizon Catalog

- **Data lineage** in Snowsight traces how data flows between objects, upstream and downstream.
- **Trust Center** is the Snowsight surface for security posture and for setting up classification without writing SQL.
- **Horizon Catalog** is the umbrella name for the whole governance surface, not a separate product.
- Under it: classification, tagging, **data metric functions**, masking and row access policies, lineage, **Universal Search** and Trust Center.

> ON THE EXAM
>
> Several Horizon components pre-date the name. If a question describes tagging or masking as 'part of Horizon', that is just current branding.

Which control for which requirement

THE REQUIREMENT	THE CONTROL
Analysts see only the last four digits of a card number	Masking policy
European staff see only European customers' rows	Row access policy
A partner may join on an identifier but never display it	Projection policy
No query may return a group smaller than 25 people	Aggregation policy
The same rule must apply across hundreds of tables	Tag-based masking
Find the sensitive columns we do not know about	Classification
Show where this figure came from	Data lineage

ON THE EXAM

Seven rows covering the module. Rows 1 to 3 are the ones the exam most likes to blur together.

A masking policy on CUSTOMER.EMAIL returns a hashed value for the analyst role. An analyst reports that a query joining CUSTOMER to a marketing table on EMAIL now returns almost no rows, although it worked before the policy was applied.

They also report that WHERE EMAIL = 'known@example.com' returns nothing, despite that row certainly existing.

What is happening?

A The masking policy is misconfigured and is masking rows as well as values.

B A row access policy must also have been applied to the table at the same time.

C Masking is applied wherever the column is referenced, including JOIN and WHERE predicates.

D Masked columns cannot be used in joins at all, so the query should have raised an error.

◆ PAUSE AND DECIDE

What is happening?

A The policy is masking rows as well.

Masking policies act on column values only. Removing rows is what a row access policy does, and none was applied.

B A row access policy was also applied.

Would explain missing rows, and the scenario describes only a masking policy. It also would not explain the WHERE clause failing on a known value.

C Masking applies to JOIN and WHERE predicates too.

Documented behaviour. The join compares hashes to plain text, and the WHERE compares a hash to a literal. Neither matches.

D Masked columns cannot be joined.

They can be joined. The join runs and simply matches nothing, which is why there is no error to help the analyst.

THE SIGNAL

Deliberate design. If masking applied only to output, you could recover values by filtering for them one at a time.

A retailer has around 900 tables across twelve databases. Many contain a column holding a customer email address, under a variety of names. New tables appear weekly from several pipelines.

Compliance requires that every email column is masked identically for all roles except a small privacy team. That applies now, and to tables that do not yet exist.

What is the appropriate design?

A Classify the databases, apply a tag to the identified columns, and attach the masking policy to the tag.

B Write a masking policy and attach it to each email column, with a weekly job to catch new tables.

C Create a secure view over every table exposing a masked email, and revoke access to the base tables.

D Apply a row access policy at database level, which cascades to all tables including future ones.

◆ PAUSE AND DECIDE

What is the appropriate design?

A Classify, tag, and attach the policy to the tag.

Classification finds the columns whatever they are named, the tag map applies the tag, and tag-based masking attaches the policy automatically.

B Attach per column, plus a weekly job.

Works today and leaves a gap every week between a table appearing and the job running. It is also 900 attachments to maintain.

C Secure views over every table.

900 views to build and keep in step with schema changes, and analysts lose access to the base tables entirely.

D A row access policy at database level.

Wrong policy type. Row access filters rows, not column values. Policies are also attached to objects, not cascaded from a database.

THE SIGNAL

'Under a variety of names' rules out anything based on matching column names. That phrase is why classification is in the answer.

A bank must ensure that relationship managers can query the shared accounts table but see only the accounts belonging to their own branch. Branch membership is held in a separate mapping table that links each role to its branch codes.

Managers must be able to run SUM and COUNT over the table, and those aggregates must reflect only their own branch.

What should be implemented?

A. A masking policy on the branch code column, so other branches' values are hidden.

B. A projection policy on the branch code column, preventing other branches being displayed.

C. An aggregation policy, which restricts results to the querying role's own group.

D. A row access policy on the table, referencing the mapping table to filter rows.

◆ PAUSE AND DECIDE

What should be implemented?

A Mask the branch code column.

Masking hides a value; the rows remain. Managers would still see other branches' balances, with the branch obscured. Worse, not better.

B A projection policy on branch code.

Projection policies govern whether a column may be displayed. They do not vary by row and would block the column for everyone.

C An aggregation policy.

Aggregation policies enforce a minimum group size to prevent isolating individuals. They do not scope results to a role.

D A row access policy referencing the mapping table.

Filters rows by role using a mapping table, and excluded rows are not counted in aggregations. Both requirements met.

THE SIGNAL

'Aggregates must reflect only their own branch' is the confirming detail: rows excluded by a row access policy are not counted.

> A data provider shares a table with a partner. The partner must be able to join their own records to the shared table on a hashed customer identifier, and to group results by it. Under no circumstances may the hashed identifier appear in any result the partner sees.
>
> A masking policy already replaces the raw identifier with the hash for all external roles.

What should be added?

A. A second masking policy on the hash column, returning NULL for external roles.

B. A projection policy on the hash column, denying projection to the partner's role.

C. An aggregation policy, since the partner only needs grouped results.

D. Remove the column from the shared secure view, since joins do not require the column to be present.

◆ PAUSE AND DECIDE

What should be added?

A. A second masking policy returning NULL.
Masking applies everywhere the column is referenced, so the join would compare NULL to NULL and break the requirement to join.

B. A projection policy denying projection.
Exactly the purpose: the column cannot be projected into the result, but remains usable in JOIN and GROUP BY.

C. An aggregation policy.
Enforces a minimum group size. It says nothing about whether a particular column may be displayed.

D. Remove the column from the view.
A column absent from the view cannot be joined on either. It removes the capability the partner needs.

THE SIGNAL

'Join and group by it, but never display it' is the definition of a projection policy. No other control separates use from display.

What to carry into the next module

- **Masking applies wherever the column is referenced.** JOIN, WHERE, ORDER BY, GROUP BY. Deliberately.
- **Masking is values, row access is rows, projection is presence in the output.** Three different questions.
- **Order:** row access filters, then projection rejects, then masking rewrites.
- **Tag-based masking** is the scaling answer. Attach the policy to a tag; classification applies the tag.
- **Classification samples 10,000 rows and excludes views.** It assists judgement rather than replacing it.

> ON THE EXAM
>
> Next: credits, resource monitors and the usage views. The last module of Domain 2 before its drill.

MODULE 11 · DOMAIN 2 - ACCOUNT MANAGEMENT AND DATA GOVERNANCE

Credits, resource monitors and the usage views

What things cost, how to cap them, and where to look afterwards.

DOMAIN 2 of 5 WEIGHT 20% MODULE 11 of 31

admin@sthithapragna.com

Warehouse credits per hour

SIZE	CREDITS/HOUR
X-Small	1
Small	2
Medium	4
Large	8
X-Large	16
2X-Large through 6X-Large	32, 64, 128, 256, 512

ON THE EXAM

Each size doubles the one below. Billed per second, with a 60-second minimum on every start or resume.

What actually consumes credits

- **Virtual warehouse compute.** The table above. The largest line on most bills.
- **Serverless compute.** Snowpipe, automatic clustering, materialized view maintenance, serverless tasks, search optimization.
- **Cloud services**, above the 10% daily allowance. Storage is separate and billed in currency, not credits.
- **Serverless is the category people forget**, and it is the one resource monitors cannot control.

> ON THE EXAM
>
> Every cost question starts with working out which of these meters the described activity lands on.

Resource monitors

- **Only ACCOUNTADMIN can create or own one.** That is a hard requirement, not a recommendation.
- **One account-level monitor per account.** You may have many warehouse-level monitors.
- Configured with a **CREDIT_QUOTA** and a **FREQUENCY**: monthly by default, or daily, weekly, yearly, or never.
- Usage **resets at midnight UTC**, whatever time you gave in START_TIMESTAMP.
- If an **END_TIMESTAMP** is reached, assigned warehouses are suspended **even if the quota was never hit**.

> ON THE EXAM
>
> A monitor with no CREDIT_QUOTA never reaches 100%, so its triggers never fire. An easy configuration to get wrong.

The three trigger actions

ACTION	WHAT HAPPENS
NOTIFY	A notification is sent. No action is taken on any warehouse.
SUSPEND	Notify, then suspend assigned warehouses after running statements have finished.
SUSPEND_IMMEDIATE	Notify and suspend at once, cancelling any statements currently executing.

ON THE EXAM

Thresholds above 100% are allowed. A common pattern is notify at 75, suspend at 100, suspend immediately at 110.

What resource monitors cannot do

- **They cannot track serverless features or AI services at all.** Use a **budget** for those.
- An **account-level** monitor does not control serverless credit usage. Snowpipe and automatic reclustering run on regardless.
- A **warehouse-level** monitor can **monitor but not suspend** cloud services usage.
- The quota **ignores the 10% cloud services adjustment**, counting credits you are never billed for.
- Suspension takes time even when immediate. Snowflake advises **buffers** and **one warehouse per monitor** for strict control.

> ON THE EXAM
>
> This slide is the module's highest-value content. Most resource monitor questions are really about a limitation.

Getting a suspended warehouse back

- Once a suspend trigger fires, assigned warehouses **cannot be resumed** until one of five things happens.
- **The next interval starts**, as dictated by the monitor's frequency and start date.
- **The credit quota is increased**, or **the suspend threshold is increased**.
- **The warehouse is unassigned** from the monitor, or **the monitor is dropped**.
- Simply issuing ALTER WAREHOUSE … RESUME will not work while the trigger is still active.

> ON THE EXAM
>
> That is a real operational scenario: a warehouse that will not start, and no error explaining why.

ACCOUNT_USAGE or INFORMATION_SCHEMA?

ACCOUNT_USAGE

In the shared SNOWFLAKE database

- Includes dropped objects
- Latency of 45 minutes to 3 hours
- Retention of 1 year, 365 days
- Account-wide, ACCOUNTADMIN by default

INFORMATION_SCHEMA

In every database

- Dropped objects are not included
- No latency at all
- Retention of 7 days to 6 months
- Scoped to its database, plus some account functions

THE TIEBREAKER

Two questions settle it. How far back do you need to look, and do you need the last few minutes?

The same name, two very different answers

VIEW OR FUNCTION	WHERE, AND HOW FAR BACK
QUERY_HISTORY	365 days as an Account Usage view. 7 days as an Information Schema table function.
WAREHOUSE_METERING_HISTORY	Credits consumed per warehouse, per hour. Account Usage.
AUTOMATIC_CLUSTERING_HISTORY	Serverless credits spent reclustering, per table.
METERING_HISTORY	Credits by service type. Where serverless and classification costs appear.
ACCESS_HISTORY	Who read and wrote which columns. Enterprise Edition.
COPY_HISTORY	Data loading history. Covered again in module 14.

ON THE EXAM

QUERY_HISTORY is the classic trap. Same name, same columns, and 365 days against 7.

Budgets, and the wider organization

- A **budget** monitors credit consumption including **serverless and AI services**, which resource monitors cannot see.
- **ORGANIZATION_USAGE** covers every account in the organization. Latency is typically **24 hours**.
- Reached from the organization account, or a regular account with **ORGADMIN** enabled.
- **READER_ACCOUNT_USAGE** covers reader accounts you have created, with an extra READER_ACCOUNT_NAME column. Empty if you have none.

> ON THE EXAM
>
> Three scopes: one warehouse, one account, one organization. Match the scope to what the question is asking about.

How this module gets tested

THE QUESTION SHAPE	THE ANSWER
'Cap this warehouse's spending'	Resource monitor with a quota and triggers.
'Snowpipe costs rose and the monitor did nothing'	Monitors cannot see serverless. Use a budget.
'The warehouse will not resume'	A suspend trigger is active. Raise the quota or threshold.
'Audit activity from nine months ago'	ACCOUNT_USAGE. INFORMATION_SCHEMA is too short.
'What happened in the last ten minutes'	INFORMATION_SCHEMA. ACCOUNT_USAGE lags by hours.
'Find the table that was dropped'	ACCOUNT_USAGE, which retains dropped objects.

ON THE EXAM

The last three are one distinction asked three ways: retention, latency, dropped objects.

> An account-level resource monitor has a quota of 10,000 credits with a suspend trigger at 100 percent. Finance reports that spending exceeded the quota by a noticeable margin last month, and no warehouse was suspended.
>
> Investigation shows the overspend came from Snowpipe ingestion and automatic clustering on several large tables.

Why did the monitor not act, and what should be used instead?

A. The trigger threshold was too high. Set it to 90 percent so it fires earlier.

B. Account-level monitors only cover the default warehouse. Add a warehouse-level monitor to each.

C. The monitor was not owned by ACCOUNTADMIN, so its triggers were ignored.

D. Resource monitors cannot track serverless features. A budget is required for that spending.

◆ PAUSE AND DECIDE

Why did the monitor not act, and what should be used instead?

A Lower the threshold to 90 percent.

A percentage of a quota that never counted this spending. The trigger cannot fire on credits the monitor cannot see.

B Account monitors cover only the default warehouse.

Invented. An account-level monitor covers all warehouses. It is serverless compute it cannot see, not other warehouses.

C Not owned by ACCOUNTADMIN.

Only ACCOUNTADMIN can create one, so this monitor necessarily was. Ownership does not silently disable triggers.

D Monitors cannot track serverless. Use a budget.

Snowpipe and automatic clustering are serverless. The documentation directs you to budgets for exactly this.

THE SIGNAL

'Snowpipe and automatic clustering' is the whole question. Both are serverless, and both are invisible to a resource monitor.

A reporting warehouse suspended itself mid-morning and will not restart. An engineer runs ALTER WAREHOUSE reporting_wh RESUME repeatedly, which appears to succeed, but the warehouse is suspended again moments later.

The warehouse is assigned to a monthly resource monitor whose suspend trigger fired at 100 percent of its quota. It is the tenth of the month.

What will actually restore service?

A Nothing until the first of next month, when the monthly interval resets.

B Increase the credit quota, or raise the suspend threshold, or unassign the warehouse from the monitor.

C Change the trigger from SUSPEND to SUSPEND_IMMEDIATE, which releases the current suspension.

D Resize the warehouse, since a resized warehouse is treated as a new warehouse by the monitor.

◆ PAUSE AND DECIDE

What will actually restore service?

A Nothing until next month.

The next interval is one of the five remedies, and it is not the only one. Twenty days of no reporting is not the answer.

B Raise the quota, raise the threshold, or unassign.

Three of the five documented ways to release the suspension. Dropping the monitor is a fourth.

C Switch to SUSPEND_IMMEDIATE.

Changes how future suspensions behave, and makes them harsher. It does not release the current one.

D Resize the warehouse.

Invented. Resizing does not create a new object, and the assignment to the monitor persists.

THE SIGNAL

The repeated RESUME appearing to succeed is realistic. The trigger is still active, so it suspends again immediately.

> A compliance investigation needs a list of every query run against a table over the previous nine months, including queries against two tables that have since been dropped.
>
> An engineer proposes using the QUERY_HISTORY table function in the INFORMATION_SCHEMA of the relevant database.

What is wrong with the proposal?

A. INFORMATION_SCHEMA retains 7 days and excludes dropped objects. Use the ACCOUNT_USAGE view instead.

B. Nothing is wrong. QUERY_HISTORY behaves identically in both schemas.

C. QUERY_HISTORY does not exist in INFORMATION_SCHEMA. It is an ACCOUNT_USAGE view only.

D. ACCOUNT_USAGE would also fail, since its retention is 90 days. The data is unrecoverable.

◆ PAUSE AND DECIDE

What is wrong with the proposal?

A 7 days, and no dropped objects. Use ACCOUNT_USAGE.

Both requirements fail in INFORMATION_SCHEMA. ACCOUNT_USAGE gives 365 days and retains records for dropped objects.

B They behave identically.

Identical names and columns, and very different retention. That similarity is exactly what makes the trap work.

C It does not exist in INFORMATION_SCHEMA.

It does, as a table function. The problem is its retention window, not its existence.

D ACCOUNT_USAGE retains only 90 days.

ACCOUNT_USAGE retains 1 year. Ninety days is the maximum Time Travel figure, which is a different feature entirely.

THE SIGNAL

Two independent failures: nine months exceeds 7 days, and dropped objects are excluded. Either alone rules it out.

A nightly transformation takes four hours on a Large warehouse. A team proposes moving it to X-Large to finish sooner, and a manager objects that this will double the cost.

The job is a single long-running query over a very large table, with no queueing and no spilling to remote storage.

How should the cost be assessed?

A The manager is right. X-Large always costs twice as much for the same work.

B Cost falls automatically, because larger warehouses receive a volume discount per credit.

C If runtime halves, credits are roughly unchanged: 16 per hour for 2 hours equals 8 per hour for 4.

D Cost is irrelevant, because the 60-second minimum dominates billing for long-running jobs.

◆ PAUSE AND DECIDE

How should the cost be assessed?

A X-Large always costs twice as much.

True per hour, and the hours are what change. It assumes runtime stays the same, which is the thing being disputed.

B A volume discount per credit.

Invented. A credit costs the same whatever size consumed it.

C If runtime halves, credits are roughly unchanged.

16 x 2 equals 8 x 4. Doubling size doubles the rate and, for a well-parallelised job, roughly halves the time.

D The 60-second minimum dominates.

The minimum matters for very short queries. Over four hours it is negligible.

THE SIGNAL

'No queueing and no spilling' says the job is not constrained in a way resizing fixes. Scaling is rarely perfectly linear.

What to carry into the next module

- **Each warehouse size doubles the credits of the one below.** XS is 1, 6XL is 512.
- **Resource monitors cannot see serverless or AI spending.** That is what budgets are for.
- **SUSPEND waits for running statements. SUSPEND_IMMEDIATE cancels them.** Thresholds above 100% are allowed.
- **ACCOUNT_USAGE:** dropped objects, 45 min to 3 hours latency, 365 days.
  INFORMATION_SCHEMA: none, none, 7 days to 6 months.
- **Only ACCOUNTADMIN** creates resource monitors. One account-level monitor per account.

> ON THE EXAM
>
> Next: the Domain 2 drill. Six worked scenarios across modules 8 to 11, and no new content.

MODULE 12 · DOMAIN 2 - ACCOUNT MANAGEMENT AND DATA GOVERNANCE

Domain 2 drill – six worked scenarios

No new content. Everything here comes from modules 8 to 11, in the form the exam uses.

DOMAIN 2 of 5 WEIGHT 20% MODULE 12 of 31

admin@sthithapragna.com

How to use this drill

- **Commit before I speak.** The pause is longer than in the teaching modules and it is there to be used.
- **Write down why.** The exam rephrases; a right answer for the wrong reason will not survive that.
- Domain 2 is **20%**, so about **20 of your 100 questions** come from this material.
- **Four or more of six** suggests the domain is solid. The final slide maps every question back to its module.

> ON THE EXAM
>
> Two misses in the same module is a signal. Two misses spread across four is normal.

A governance team creates a database role in the SALES database, grants it SELECT on several tables, and grants that database role directly to three analysts.

The statement granting the role to the analysts fails. The team then tries USE ROLE with the database role name in a session, which also fails.

What is the correct approach?

A Grant the database role to an account role, then grant that account role to the analysts.

B Database roles must be created by SECURITYADMIN. Recreate it using that role.

C Grant the database role to PUBLIC, which every user already holds.

D Enable secondary roles for the analysts, which allows database roles to be activated directly.

◆ PAUSE AND DECIDE

What is the correct approach?

A Grant it to an account role, then grant that to the analysts.

Database roles cannot be granted to users or activated in a session. Routing through an account role is the documented pattern.

B It must be created by SECURITYADMIN.

Who created it is irrelevant. The failure is about what a database role can be granted to, not who made it.

C Grant it to PUBLIC.

PUBLIC is an account role, so this would technically work and gives every user in the account the access. Not the intent.

D Enable secondary roles.

Secondary roles activate account roles the user already holds. They do not make a database role directly activatable.

WHERE THIS CAME FROM

Module 8. Both failures have the same cause: a database role cannot be granted to a user, and cannot be activated directly.

A security engineer creates a network rule of type IPV4 containing the corporate office CIDR range, in a schema called SECURITY.RULES. They confirm the rule exists with SHOW NETWORK RULES.

A week later, an audit finds that connections are still being accepted from arbitrary addresses worldwide.

What was missed?

A Network rules must be created in the SNOWFLAKE database to take effect account-wide.

B A network rule does nothing alone. It must be added to a network policy, which must then be activated.

C The rule type should have been AWSVPCEID, since IPV4 rules are advisory only.

D Network rules only apply to Snowsight sign-ins, not to driver connections.

◆ PAUSE AND DECIDE

What was missed?

A Rules must live in the SNOWFLAKE database.

Network rules are ordinary schema-level objects and can live in any schema. Their location is not the problem.

B It must be added to a policy, and the policy activated.

Rules hold identifiers. Policies hold rules in allowed and blocked lists, and do nothing until activated on an account, integration or user.

C IPV4 rules are advisory only.

Invented. IPV4 rules are enforced. Private connectivity rules take precedence over them, which is a different point.

D They only apply to Snowsight.

Network policies apply to all connections regardless of client or authentication method.

WHERE THIS CAME FROM

Module 9. Two objects, two steps. Creating the rule is the easy half; the policy and its activation are the half that enforces anything.

A privacy team enables automatic classification across a database with default settings, and asks Snowflake to auto-apply tags. Two weeks later an internal review finds a column of passport numbers that was never tagged, in a table where only about 0.4 percent of rows have a value in that column.

The review also finds several views exposing sensitive columns, none of which were classified.

Why were these missed?

A Classification only runs once. It must be re-run manually after any new data arrives.

B Auto-apply must be disabled for sparse columns, which are only ever recommended, never tagged.

C Classification samples up to 10,000 rows per table and excludes views from classification entirely.

D Passport numbers have no native semantic category, so no column of them can ever be classified.

◆ PAUSE AND DECIDE

Why were these missed?

A It only runs once.

Default behaviour reclassifies previously classified objects every 30 days. Re-running is not the issue.

B Sparse columns are only recommended.

Invented. auto_tag governs whether tags are applied at all; it does not vary by how sparse a column is.

C Samples 10,000 rows, and excludes views.

Both documented defaults, and each explains one half of the finding. A value in 0.4% of rows may not appear in the sample.

D No native category for passport numbers.

Custom categories can be created for anything not natively covered. It would also not explain the views.

WHERE THIS CAME FROM

Module 10. Two findings, two documented limits. Classification assists human judgement; it does not certify that nothing was missed.

A platform team builds an operational dashboard on SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY, intending to alert on-call staff within minutes when a query exceeds a runtime threshold.

In testing, alerts arrive reliably but always well over an hour after the offending query has finished. The dashboard query itself runs in seconds.

What is the cause, and what should they use?

A QUERY_HISTORY excludes queries still within their Time Travel window, which delays their appearance.

B The dashboard warehouse is too small. Increase its size so the view refreshes faster.

C ACCOUNT_USAGE only refreshes when queried by ACCOUNTADMIN. Change the dashboard's role.

D ACCOUNT_USAGE views have latency of up to 3 hours. Use the INFORMATION_SCHEMA table function for recent activity.

◆ PAUSE AND DECIDE

What is the cause, and what should they use?

A Time Travel delays appearance.

Time Travel governs historical data in tables. It has nothing to do with metadata view latency.

B The dashboard warehouse is too small.

The dashboard query already runs in seconds. Latency is in the metadata pipeline, not in the query.

C It refreshes only for ACCOUNTADMIN.

Invented. The role governs whether you can read the view, not how current its data is.

D ACCOUNT_USAGE latency. Use INFORMATION_SCHEMA.

Latency runs from 45 minutes to 3 hours, most views around 2. The Information Schema function has no latency at all.

WHERE THIS CAME FROM

Module 11. Same view name, opposite trade. ACCOUNT_USAGE buys a year of history at the price of hours of lag.

An analytics team owns several tables in a schema and wants to grant a partner team SELECT on one of them. The grant statement fails, although the analytics role holds OWNERSHIP on the table.

The schema was created using CREATE SCHEMA ... WITH MANAGED ACCESS. The analytics role does not hold MANAGE GRANTS and does not own the schema.

Why did the grant fail?

A OWNERSHIP does not include the ability to grant. A separate GRANT privilege must be held on the table.

B Managed access schemas prevent all grants. Access must be given through a secure view instead.

C In a managed access schema, object owners lose the ability to grant. Only the schema owner or a role with MANAGE GRANTS can.

D The partner team must first be granted USAGE on the schema by the analytics role, which it cannot do.

◆ PAUSE AND DECIDE

Why did the grant fail?

A OWNERSHIP does not include granting.

Normally it does. That is the discretionary half of the model, and a managed access schema is the exception to it.

B Managed access prevents all grants.

It centralises who may grant, not whether granting is possible. The schema owner grants freely.

C Owners lose the ability to grant. Schema owner or MANAGE GRANTS only.

Exactly what managed access does, and the scenario states the analytics role has neither qualification.

D USAGE on the schema must come first.

USAGE is genuinely required to reach the table, and it is not why this GRANT statement failed.

WHERE THIS CAME FROM

Modules 8 and 10 together. Ownership normally carries the right to grant. Managed access is the one place Snowflake switches that off.

A regulator asks a company to confirm, within the hour, which users read a specific customer column during the previous ninety minutes. The company runs Snowflake Standard Edition.

A data engineer proposes querying SNOWFLAKE.ACCOUNT_USAGE.ACCESS_HISTORY and filtering to the last ninety minutes.

What is wrong with the proposal?

A ACCESS_HISTORY records writes only. QUERY_HISTORY should be used for reads instead.

B Nothing. ACCESS_HISTORY is available in all editions and ACCOUNT_USAGE has no latency.

C Only that the engineer needs ACCOUNTADMIN. Otherwise the approach is correct.

D Two problems: ACCESS_HISTORY requires Enterprise, and ACCOUNT_USAGE latency means the last 90 minutes may not be present.

◆ PAUSE AND DECIDE

What is wrong with the proposal?

A ACCESS_HISTORY records writes only.

It records reads and writes at column level. Reads are precisely what it is for.

B Nothing is wrong.

Wrong on both counts. ACCESS_HISTORY is Enterprise, and every ACCOUNT_USAGE view has latency.

C Just needs ACCOUNTADMIN.

True that ACCOUNT_USAGE is ACCOUNTADMIN by default, and it is the least of the problems here.

D Enterprise required, and latency hides the last 90 minutes.

Two independent blockers. ACCESS_HISTORY is Enterprise, and ACCOUNT_USAGE lags 45 minutes to 3 hours.

WHERE THIS CAME FROM

Modules 2, 10 and 11. Edition, then latency. Fixing either one alone still leaves the request unanswerable.

Where each question came from

QUESTION	MODULE TO REVISIT IF YOU MISSED IT
1. Database role cannot be granted to a user	Module 8 - account roles and database roles
2. Network rule with no policy	Module 9 - rules are schema objects, policies are account objects
3. Classification missed a sparse column and all views	Module 10 - classification profiles and the defaults
4. Dashboard alerts arrive an hour late	Module 11 - ACCOUNT_USAGE latency
5. Owner cannot grant in a managed access schema	Modules 8 and 10 together
6. ACCESS_HISTORY within the hour on Standard	Modules 2, 10 and 11 together

ON THE EXAM

Questions 5 and 6 span more than one module. Those are the ones worth re-reading even if you got them right.

What this drill was really testing

- **Two objects, two steps.** A rule needs a policy. A database role needs an account role. A policy needs activating.
- **Documented defaults you did not choose.** 10,000-row sampling, views excluded, hours of latency, ACCOUNTADMIN-only.
- **Exceptions to a rule you already learned.** Owners can grant, except in a managed access schema.
- **Constraints that stack.** Question 6 had two blockers. Fixing one leaves you no better off.

> ON THE EXAM
>
> Next: Domain 3. Data loading, unloading and connectivity at 18%, beginning with stages and file formats.

MODULE 13 · DOMAIN 3 - DATA LOADING, UNLOADING, AND CONNECTIVITY

Stages, file formats and the load path

Domain 3 begins. The stage symbols are free marks, and candidates routinely guess at them.

DOMAIN 3 of 5 WEIGHT 18% MODULE 13 of 31

admin@sthithapragna.com

Loading is always **two steps**. Get the files to a stage. Then **COPY INTO** a table.

The four stage types

STAGE	HOW YOU REFERENCE IT	WHO IT BELONGS TO
User	@~	One user. Allocated automatically. Nobody else can see it.
Table	@%mytable	One table. Allocated automatically. Same name as the table.
Named internal	@mystage	A database object. Shared by whoever has privileges on it.
Named external	@myextstage	Points at S3, Azure Blob or Google Cloud Storage.

ON THE EXAM

Learn the three symbols cold. Tilde is the user, percent is the table, and a bare name is a named stage.

Table stages have real restrictions

- **An implicit stage tied to the table. Not a separate database object.** So it has **no grantable privileges of its own**.
- To stage, list, query or drop files on it you must **be the table owner**.
- **Cannot be altered or dropped.** It exists because the table exists.
- **Does not support transforming data while loading.** You cannot use a query as the source for COPY.
- **Iceberg tables do not support table stages** at all.

> ON THE EXAM
>
> Every restriction follows from one fact: it is not an object, so there is nothing to grant on and nothing to alter.

When the automatic stages are not enough

USER AND TABLE STAGES

Convenient, and narrow

- Allocated automatically, nothing to create
- User stage: only that user, ever
- Table stage: only the table owner, one table
- Neither can be granted, altered or dropped

NAMED INTERNAL STAGES

The flexible option

- A real database object with privileges
- Multiple users can load from it
- Data can go to multiple tables
- Supports transformation during load

THE TIEBREAKER

Two questions force a named stage: does more than one person need it, and does more than one table need it?

External stages and storage integrations

- A named external stage points at a location in **S3, Azure Blob Storage or Google Cloud Storage**.
- A **storage integration** holds the identity and access configuration, so credentials never appear in the stage definition.
- The integration is created once by an administrator and reused by many stages. Credentials are never seen by the people creating stages.
- Embedding cloud credentials directly in CREATE STAGE is possible in some cases and is **the wrong answer** on a security question.

> ON THE EXAM
>
> Storage integration is one of four integration types. Storage for buckets, API for HTTPS, notification for messaging, catalog for Iceberg.

Directory tables and stage encryption

- A **directory table** sits over a stage and holds file-level metadata, so files can be listed and queried in SQL.
- It is not a separate object. It is a **property of the stage**, enabled with DIRECTORY = (ENABLE = TRUE).
- It **requires a warehouse and therefore costs money**, which is why it is optional rather than automatic.
- **Internal stages are always encrypted by Snowflake.** For external stages, you manage encryption and compression.

> ON THE EXAM
>
> Directory tables are how unstructured data becomes queryable: file names, sizes and URLs, addressable in SQL.

File formats Snowflake loads

FORMAT	KIND
CSV, and delimited text generally	Structured
JSON	Semi-structured
Avro	Semi-structured
ORC	Semi-structured, columnar
Parquet	Semi-structured, columnar
XML	Semi-structured

ON THE EXAM

Six formats. One structured, five semi-structured. That split is the answer to a surprising number of questions.

File format objects and compression

- A **file format object** stores the settings once: type, delimiter, header handling, null handling, date formats.
- Reference it from a stage or from a COPY statement, instead of repeating options on every load.
- Supported compression: **GZIP, BZ2, BROTLI, ZSTD, SNAPPY, DEFLATE and RAW_DEFLATE**.
- Snowflake **decompresses automatically**. You do not uncompress files before loading them.

> ON THE EXAM
>
> A file format object is the same pattern as a policy: define once, reference many times, change in one place.

File sizing, and the sentence that explains it

- **The number of parallel load operations cannot exceed the number of data files.** Everything else follows from this.
- Aim for roughly **100 to 250 MB compressed**, or larger. This applies to bulk loads *and* to Snowpipe.
- **Very large files, 100 GB or more, are not recommended.** One file cannot be split across threads.
- Files under about **10 MB** should be aggregated. Per-file overhead starts to dominate.
- Use a **path or prefix** so COPY lists only the relevant files rather than traversing a whole bucket.

> ON THE EXAM
>
> One 200 GB file on a 2X-Large loads no faster than on an X-Small. There is nothing for the extra threads to do.

Which stage for which job

THE SITUATION	THE STAGE
A quick personal load nobody else needs	User stage, @~
One table, loaded by its owner, no transformation	Table stage, @%mytable
Several people, several tables, or transformation	Named internal stage
The data already lives in your own cloud storage	External stage plus a storage integration
Files must be listed and queried in SQL	A directory table on the stage

ON THE EXAM

Row three is the default in real accounts. The automatic stages run out of room quickly.

A team receives a nightly set of CSV files containing order data. The same files must be loaded into three tables: a raw table, a cleaned table where several columns are cast and reordered during load, and an archive table.

Four engineers run the pipeline on rotation. An engineer proposes staging the files on the raw table's table stage.

What is wrong with the proposal?

A Table stages cannot serve multiple tables, cannot be granted to other users, and do not support transformation during load.

B Table stages only accept a single file per load, so the nightly set would need combining first.

C Nothing is wrong, provided each engineer is granted USAGE on the table stage.

D Table stages cannot hold CSV files. Only named stages support structured formats.

◆ PAUSE AND DECIDE

What is wrong with the proposal?

A Three restrictions, all of which this scenario hits.

Multiple tables, multiple users, and transformation during load. Every one of those is something a table stage cannot do.

B One file per load.

Invented. Any stage type holds many files, and loading many at once is exactly what you want.

C Grant USAGE to each engineer.

A table stage is not a database object and has no grantable privileges. There is nothing to grant.

D Table stages cannot hold CSV.

All stage types hold any supported format. Format is a property of the file and the COPY, not the stage.

THE SIGNAL

Three independent blockers in one scenario. Any one alone would force a named internal stage.

A nightly bulk load reads a single compressed file of about 180 GB from an external stage. It takes just over five hours on a Large warehouse.

The team resizes to 2X-Large. Runtime is essentially unchanged. Query Profile shows no spilling and no queueing, and the warehouse appears largely idle during the load.

Why did resizing not help?

A COPY INTO ignores warehouse size and always runs on a fixed serverless pool.

B The file must be decompressed before loading, and decompression is single-threaded regardless of size.

C Parallel load operations cannot exceed the number of files. One file means one thread, whatever the size of the warehouse.

D External stages cap load parallelism at four threads. An internal stage would have scaled.

◆ PAUSE AND DECIDE

Why did resizing not help?

A. COPY runs on a serverless pool.
COPY INTO runs on the warehouse you name. Snowpipe is the serverless loader, and it is a different thing.

B. Decompression is single-threaded.
Sounds plausible and is not the documented reason. Snowflake decompresses automatically, and the constraint is file count.

C. Parallelism cannot exceed the number of files.
Documented directly. One file gives one load operation, so the extra threads on a 2X-Large have nothing to do.

D. External stages cap at four threads.
Invented. Stage type does not limit parallelism; the number of files does.

THE SIGNAL

Split the file. Roughly a thousand files of 180 MB would saturate the warehouse and cost less than the 2X-Large did.

> A security review finds that a CREATE STAGE statement in a version-controlled deployment script contains an AWS access key and secret key in plain text. Several analysts have been granted the privilege to create stages.
>
> The team needs Snowflake to keep reading the same S3 bucket without exposing credentials to anyone who can read the script or run SHOW STAGES.

What should be implemented?

A. Move the credentials into a Snowflake SECRET object and reference it from the stage.

B. Create a storage integration holding the IAM configuration, and reference it from the external stage.

C. Copy the files into a named internal stage, which removes the need for cloud credentials.

D. Apply a masking policy to the stage definition so the keys are hidden from SHOW STAGES.

◆ PAUSE AND DECIDE

What should be implemented?

A. A SECRET object referenced from the stage.

Secrets serve API integrations and external access, notably Git. Cloud storage authentication is what storage integrations are for.

B. A storage integration referenced from the external stage.

Exactly its purpose. An administrator creates it once, stage creators never see credentials, and rotation happens in one place.

C. Copy files into an internal stage.

Something must still read the bucket to copy them, needing the same credentials. It also duplicates storage.

D. Mask the stage definition.

Masking policies protect column values in tables and views. They do not apply to object definitions.

THE SIGNAL

Four integration types, and picking the wrong one is the commonest error here. Storage for buckets, API for HTTPS endpoints.

A team stores several hundred thousand scanned invoices as PDFs in an external stage. They want to build a table listing every file with its name, size and last-modified time, join that to an internal register, and generate URLs so an application can retrieve individual documents.

An engineer proposes running LIST on the stage each morning and parsing the output into a table.

What is the better approach?

A Load the PDFs into a VARIANT column, which makes their metadata queryable in SQL.

B Create an external table over the stage, which exposes file metadata as columns.

C Nothing better exists. LIST is the only way to see stage contents, so parsing its output is correct.

D Enable a directory table on the stage, which exposes file metadata and URLs as queryable columns.

◆ PAUSE AND DECIDE

What is the better approach?

A Load the PDFs into a VARIANT column.

VARIANT holds semi-structured data and is capped at 16 MB per row. A PDF is unstructured; this is the wrong container.

B An external table over the stage.

External tables read the contents of structured and semi-structured files. They do not catalogue unstructured documents.

C LIST is the only option.

Directory tables exist for exactly this. Parsing command output into a table is the workaround they replace.

D Enable a directory table on the stage.

Purpose-built: file names, sizes, last-modified times and URLs as queryable columns, joinable like any table.

THE SIGNAL

Directory tables need a warehouse and so cost money. That is the trade, and it is why they are not on by default.

What to carry into the next module

- **@~ is the user stage, @%table is the table stage, @name is a named stage.** Free marks.
- **Table stages are not objects.** No grantable privileges, no ALTER or DROP, one table, no transformation on load.
- **Parallel loads cannot exceed the number of files.** One huge file gets one thread. Aim for 100 to 250 MB compressed.
- **Storage integration** for external stages, so credentials never appear in a stage definition.
- **Directory tables** make stage contents queryable. They need a warehouse, so they cost money.

> ON THE EXAM
>
> Next: COPY INTO itself. Error handling, load metadata, transformation on load, and unloading back out.

MODULE 14 · DOMAIN 3 - DATA LOADING, UNLOADING, AND CONNECTIVITY

COPY INTO, error handling and unloading

Step two of the load path, and the same command run backwards.

DOMAIN 3 of 5 WEIGHT 18% MODULE 14 of 31

admin@sthithapragna.com

COPY is **idempotent by default**. Run it twice and you do not get the data twice.

Load metadata, and the 64-day boundary

- Snowflake records which files have been loaded into each table. **This load metadata expires after 64 days.**
- Within 64 days, COPY knows a file's status and **skips it** rather than duplicating the data.
- After 64 days the status is **unknown**, and COPY **skips the file by default** to avoid an accidental reload.
- **LOAD_UNCERTAIN_FILES = TRUE** loads files with expired metadata, while still using metadata where it exists.
- **FORCE = TRUE** ignores load metadata entirely and loads everything. Default is FALSE.

> ON THE EXAM
>
> Note the asymmetry: inside 64 days a file is skipped because it WAS loaded. Outside 64 days it is skipped because Snowflake cannot tell.

PURGE, and where Snowpipe differs

- **PURGE = TRUE** removes files from the stage after a successful load, which keeps storage costs down.
- **If the purge fails, no error is returned.** Confirm with LIST rather than assuming.
- **Snowpipe does not support PURGE.** Use REMOVE periodically, or cloud storage lifecycle rules.
- Snowpipe's load metadata lives on the **pipe object, not the table**, and is retained **14 days**, not 64.

> ON THE EXAM
>
> Recreating a pipe wipes its load history. Files already loaded can then be reloaded, which is a real duplication risk.

ON_ERROR, and two different defaults

VALUE	WHAT IT DOES
ABORT_STATEMENT	Stop the entire load if any error is found. The default for bulk COPY.
CONTINUE	Load every row that can be loaded, and skip the bad ones.
SKIP_FILE	Skip the whole file if any error is found. The default for Snowpipe.
SKIP_FILE_n	Skip the file once n error rows have been found.
SKIP_FILE_n%	Skip the file once the proportion of error rows reaches n percent.

ON THE EXAM

Bulk COPY defaults to ABORT_STATEMENT. Snowpipe defaults to SKIP_FILE. Same option, different default, and questions exploit that.

Validating before you load

- **VALIDATION_MODE** checks the files and returns results. **No data is loaded** when it is specified.
- **RETURN_n_ROWS** returns n rows if no errors are found. A quick sanity check on parsing.
- **RETURN_ERRORS** returns errors across the files. **RETURN_ALL_ERRORS** includes errors from previously failed loads.
- The **VALIDATE** table function inspects a load that has *already run*, rather than a hypothetical one.
- **XML is not supported** with VALIDATION_MODE.

> ON THE EXAM
>
> VALIDATION_MODE looks forward at files you have not loaded. VALIDATE looks backward at a COPY you already ran.

Transforming during load

SUPPORTED

Simple, row-by-row work

- Reorder columns
- Omit columns you do not want
- Cast data types
- Simple SQL expressions on staged columns

NOT SUPPORTED

Anything needing other rows

- Joins to other tables
- Aggregations and GROUP BY
- Filters and WHERE clauses
- FLATTEN

THE TIEBREAKER

The rule underneath: a COPY transformation can only look at the row in front of it. Anything needing another row is out.

Matching columns by name

- By default, staged columns map to target columns **by position**. The first column goes to the first column.
- **MATCH_BY_COLUMN_NAME** maps by name instead: **CASE_SENSITIVE**, **CASE_INSENSITIVE**, or **NONE**.
- That makes loads resilient when a source system reorders its columns or adds new ones.
- **INCLUDE_METADATA** maps METADATA$ columns, such as source filename and row number, into target columns.

> ON THE EXAM
>
> Positional matching fails silently when a source adds a column. Everything shifts one place and the load still succeeds.

Unloading is the same command backwards, and it is **less** restricted than loading.

Unloading options worth knowing

OPTION	WHAT IT DOES
SINGLE	Write one file instead of many. Default is FALSE.
MAX_FILE_SIZE	Maximum size of each output file. Default is 16 MB.
PARTITION BY	An expression splitting rows into separate files, producing a directory structure.
HEADER	Include column names as the first row. Default is FALSE.
OVERWRITE	Replace files already at the target location.

ON THE EXAM

The 16 MB default surprises people. Unload a large table without raising MAX_FILE_SIZE and you get a great many small files.

How this module gets tested

THE QUESTION SHAPE	THE ANSWER
'We re-ran COPY and nothing loaded'	Load metadata. Use FORCE to reload deliberately.
'One bad row killed a 5,000-file load'	ABORT_STATEMENT is the bulk default. Use CONTINUE or SKIP_FILE.
'Check the files without loading them'	VALIDATION_MODE. No data is loaded.
'Join to a lookup table during load'	Not supported. Load first, transform after.
'The source added a column and data went wrong'	Positional matching. Use MATCH_BY_COLUMN_NAME.
'The partner needs one CSV with headers'	SINGLE = TRUE, HEADER = TRUE, and raise MAX_FILE_SIZE.

ON THE EXAM

Six shapes. The first two are the ones that appear most often, and both are about a default nobody chose.

A pipeline loads daily files from an external stage. A bug is found in a downstream transformation. The team corrects the target table, then re-runs the original COPY statement to reload the last week of source files.

The statement completes successfully, reports zero rows loaded, and no error appears. The files are still present in the stage and were staged four days ago.

What is happening, and how should they reload?

A The files exceed the 64-day metadata window. Set LOAD_UNCERTAIN_FILES = TRUE.

B Load metadata records these files as already loaded. Use FORCE = TRUE to reload them.

C PURGE removed the files after the first load, so there is nothing left to read.

D The COPY was aborted by ON_ERROR. Set ON_ERROR = CONTINUE and re-run.

◆ PAUSE AND DECIDE

What is happening, and how should they reload?

A Exceeds the 64-day window. LOAD_UNCERTAIN_FILES.

The files were staged four days ago, so their status is known rather than uncertain. Right option, wrong situation.

B Already loaded. Use FORCE = TRUE.

COPY is idempotent by default. Within 64 days it knows these files were loaded and skips them. FORCE ignores that metadata.

C PURGE removed the files.

The scenario states the files are still present in the stage. It also would have produced an error about missing files, not zero rows.

D Aborted by ON_ERROR.

An abort produces an error, not a clean success. The statement completed successfully with zero rows.

THE SIGNAL

'Completes successfully, zero rows, no error' is the signature of load metadata. A failure would look quite different.

> A nightly bulk load reads about 5,000 files. One night it fails after roughly forty minutes with an error about an unparseable date in a single row. Nothing is loaded at all.
>
> The team wants the load to complete, and to be told afterwards which rows could not be parsed. Losing the whole night's data to one bad record is unacceptable.

What should they change?

A. Nothing in COPY. Add a Snowpipe pipe, which skips bad files automatically.

B. Set VALIDATION_MODE = RETURN_ALL_ERRORS on the nightly load.

C. Set FORCE = TRUE so the load ignores the failed rows and continues.

D. The default is ABORT_STATEMENT. Set ON_ERROR = CONTINUE and review COPY_HISTORY afterwards.

◆ PAUSE AND DECIDE

What should they change?

A Add a Snowpipe pipe.

Snowpipe does default to SKIP_FILE, and redesigning the ingestion architecture to change an error-handling default is disproportionate.

B VALIDATION_MODE = RETURN_ALL_ERRORS.

Validation mode loads no data at all. The nightly load would report errors and never populate the table.

C FORCE = TRUE.

FORCE governs load metadata and reloading. It has nothing to do with error handling.

D ON_ERROR = CONTINUE, then review COPY_HISTORY.

Loads every row that parses, skips the rest, and the rejected rows are visible afterwards. Exactly the stated requirement.

THE SIGNAL

ABORT_STATEMENT is the bulk default nobody chooses. Note that the failed run will not appear in COPY_HISTORY, because nothing was ingested.

> An engineer wants a single COPY statement to load staged CSV files into a target table while looking up a region code from an existing dimension table, and while summing several line items into a total column.
>
> They have written a SELECT from the stage with a join and a GROUP BY, and it fails.

Why does it fail, and what is the right pattern?

A. The SELECT must reference columns as $1, $2. Rewriting it positionally will allow the join.

B. MATCH_BY_COLUMN_NAME must be set before a query can be used as the source of a COPY.

C. COPY transformations do not support joins or aggregations. Load the raw data first, then transform.

D. Joins are supported, but only against tables in the same schema as the stage.

◆ PAUSE AND DECIDE

Why does it fail, and what is the right pattern?

A Rewrite the SELECT positionally.

Positional references are genuinely required, and they do not unlock joins. The restriction is on the operation, not the syntax.

B Set MATCH_BY_COLUMN_NAME first.

That controls how staged columns map to target columns. It is unrelated to whether a join is permitted.

C No joins or aggregations. Load raw, then transform.

Documented. A COPY transformation sees one row at a time, so anything needing another row or table is out.

D Joins work within the same schema.

Invented qualification. Joins are not supported in a COPY transformation at any scope.

THE SIGNAL

Two separate violations here: a join and an aggregation. Either alone would fail. Both need the load-then-transform pattern.

> A partner requires a single CSV file containing about 900 MB of data, with a header row, delivered to an external stage each week. The partner's loader cannot handle multiple files or headerless data.
>
> An engineer runs COPY INTO with a file format of type CSV and finds dozens of small files in the stage, none with headers.

Which options are needed?

A. SINGLE = TRUE, HEADER = TRUE, and MAX_FILE_SIZE raised above the 16 MB default.

B. PARTITION BY on a constant expression, which forces all rows into one partition and one file.

C. OVERWRITE = TRUE, which replaces the many files with one combined file.

D. Nothing. Unload to an internal stage instead, which always produces a single file with headers.

◆ PAUSE AND DECIDE

Which options are needed?

A SINGLE, HEADER, and a raised MAX_FILE_SIZE.

Three defaults, three symptoms. SINGLE is FALSE, HEADER is FALSE, and MAX_FILE_SIZE is 16 MB, which 900 MB comfortably exceeds.

B PARTITION BY a constant.

Partitioning splits output into more files, not fewer. It also cannot omit the partition column from the files.

C OVERWRITE = TRUE.

Controls whether existing files at the target are replaced. It does not combine output into one file.

D Unload to an internal stage.

Stage type does not change file count or headers. The same defaults apply wherever you unload to.

THE SIGNAL

Every symptom traces to a default: SINGLE FALSE, HEADER FALSE, MAX_FILE_SIZE 16 MB. None of them is what a partner usually wants.

What to carry into the next module

- **Load metadata expires after 64 days.** FORCE reloads deliberately; LOAD_UNCERTAIN_FILES handles expired status.
- **ABORT_STATEMENT is the bulk default. SKIP_FILE is the Snowpipe default.** Same option, different defaults.
- **VALIDATION_MODE loads nothing.** It looks forward at files; VALIDATE looks backward at a COPY already run.
- **No joins, aggregations, filters or FLATTEN during load.** Unloading has no such limit, because the source can be any SELECT.
- **Unload defaults:** SINGLE FALSE, HEADER FALSE, MAX_FILE_SIZE 16 MB.

> ON THE EXAM
>
> Next: continuous ingestion. Snowpipe, Snowpipe Streaming, streams, tasks and dynamic tables. The densest module in the course.

MODULE 15 · DOMAIN 3 - DATA LOADING, UNLOADING, AND CONNECTIVITY

Snowpipe, Streaming, streams, tasks and dynamic tables

The densest module in the course. Five mechanisms, and one question that separates them.

DOMAIN 3 of 5 WEIGHT 18% MODULE 15 of 31

admin@sthithapragna.com

Batch loading is a **statement you run**. Continuous ingestion is a **thing that runs itself**.

Snowpipe

- **Serverless.** Snowflake provides the compute. No warehouse of yours, and it bills as serverless credits.
- Two ways to trigger it: **auto-ingest** from cloud event notifications, or calls to the **Snowpipe REST API**.
- Target latency is roughly **one minute** after a file lands. It loads files, not rows.
- A **PIPE** object wraps a COPY statement. Snowpipe is essentially COPY that runs itself when files appear.

> ON THE EXAM
>
> Auto-ingest needs a notification integration, which is the third of the four integration types from module 5.

Snowpipe details that get tested

- **Load metadata lives on the PIPE, not the table**, and is retained **14 days**, not 64.
- **Recreating a pipe wipes its load history**, so files already loaded can be loaded again. A real duplication risk.
- **Snowpipe does not support PURGE.** Use REMOVE periodically, or cloud storage lifecycle rules.
- **ON_ERROR defaults to SKIP_FILE**, not ABORT_STATEMENT.
- **ALTER PIPE … REFRESH** picks up files staged in the **previous 7 days**. Older files need a COPY statement.

> ON THE EXAM
>
> Three different retention numbers now: 64 days for COPY load metadata, 14 for a pipe, 7 for REFRESH. Keep them apart.

Snowpipe Streaming has two architectures

CLASSIC

Channels open against the table

- SDK: snowflake-ingest-sdk, Java 8 or later
- Schema validation happens client-side
- Credit-based billing
- Supported now, deprecation planned

HIGH-PERFORMANCE

Channels open against a PIPE

- SDK: snowpipe-streaming, Java 11 or Python 3.9
- Schema validation is server-side, from the PIPE
- Throughput-based billing, up to 10 GB/s per table
- Latency of 5 to 10 seconds. ON_ERROR supports only CONTINUE

THE DISTINCTION THAT MATTERS

Streaming writes rows, not files. That is the whole point, and it is what separates it from Snowpipe.

Which architecture, and what is changing

- Classic is **fully supported today**, and Snowflake has stated it is **planned for deprecation**.
- A formal deprecation announcement is intended for **mid-2026**, followed by an **18-month migration window**.
- Snowflake's recommendation is to use the **high-performance architecture for new work**.
- The January 2026 study guide predates this split. **Expect questions about Snowpipe Streaming generally**, not about which architecture.

> ON THE EXAM
>
> Know that two exist and how they differ. Do not expect a question that turns on the SDK version.

Streams: an offset, not a copy

- A stream stores an **offset** in the source table's version history and returns rows changed since that point.
- **It does not store the data.** It is a bookmark, which is why creating one costs almost nothing.
- **Consuming a stream in a DML transaction advances the offset.** The change records are then gone from it.
- Two consumers sharing one stream will not both see the changes. **Create a separate stream per consumer.**
- **SYSTEM$STREAM_HAS_DATA** checks whether there is anything to process, which is how tasks skip empty runs.

> ON THE EXAM
>
> 'A bookmark, not a copy' explains the cost, the staleness behaviour, and why two consumers interfere.

The three stream types

TYPE	WHAT IT TRACKS
Standard	Inserts, updates and deletes.
Append-only	Inserts only. Updates, deletes and truncates are not recorded.
Insert-only	Inserts, on external tables only.

ON THE EXAM

An update appears as a DELETE and an INSERT pair, both with METADATA$ISUPDATE = TRUE. That is how you tell it apart from a real delete.

Where a stream can be created

- On **standard tables**, permanent or transient. Also on **directory tables** and **external tables**.
- On **views**, including secure and shared views, provided **change tracking is enabled** on the view and all underlying tables.
- Only the **object owner** can enable change tracking. Creating a stream on a view as the view owner enables it.
- The retention extension does **not** apply to directory tables or external tables, because they have no retention period.

> ON THE EXAM
>
> A stream on a view is the usual way to capture changes across a join without duplicating logic in every consumer.

Staleness, and the extension that saves you

- A stream goes **stale** when its offset falls outside the source table's retention. **Unconsumed changes are lost.**
- That is **data loss, not an error**. Drop and recreate, which resets the offset to now.
- If retention is **under 14 days and the stream is unconsumed**, Snowflake extends retention to the stream's offset.
- Bounded by **MAX_DATA_EXTENSION_TIME_IN_DAYS, default 14, regardless of edition**.
- Check **STALE_AFTER** with DESCRIBE STREAM, not the retention setting.

> ON THE EXAM
>
> A stream on a 1-day table typically survives about fourteen days unconsumed, not one. CREATE OR REPLACE TABLE makes it stale immediately.

Tasks: serverless or your warehouse

- **Omit the WAREHOUSE parameter and the task is serverless.** Name a warehouse and it is user-managed.
- WAREHOUSE and USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE are **mutually exclusive**. Unset WAREHOUSE to convert to serverless.
- For serverless tasks, Snowflake sizes the compute, **learning the right size** from completed runs after the first few.
- Tasks are **created suspended**. They must be resumed before anything happens.
- **SUSPEND_TASK_AFTER_NUM_FAILURES** auto-suspends a task that keeps failing.

> ON THE EXAM
>
> 'Created suspended' catches people constantly. A perfectly correct task that nobody resumed does nothing at all.

Scheduling and task graphs

- Schedule with a **cron expression including a time zone**, or a fixed interval.
- **A schedule is required for a standalone task or the root task. A child task cannot have one** – it uses AFTER.
- With no schedule, a task runs only via **EXECUTE TASK**.
- A **task graph** is a DAG with exactly **one root task**. A **finalizer** runs at the end, set with FINALIZE.
- **OVERLAP_POLICY**: NO_OVERLAP, ALLOW_CHILD_OVERLAP, or ALLOW_ALL_OVERLAP. Under NO_OVERLAP an overrun **skips** the next slot.

> ON THE EXAM
>
> OVERLAP_POLICY replaces the deprecated ALLOW_OVERLAPPING_EXECUTION. Recognise both names.

Dynamic tables against streams and tasks

STREAM PLUS TASK

You write the logic

- Full control: MERGE, compound keys, custom rules
- You manage the schedule and the dependencies
- Sub-minute freshness is possible
- More moving parts to operate

DYNAMIC TABLE

Snowflake writes the logic

- You give a SELECT and a TARGET_LAG
- Snowflake infers dependencies and refresh order
- Minimum target lag of 1 minute
- Query definition is immutable: drop and recreate

A LIMIT WORTH REMEMBERING

Dynamic tables cannot be downstream of a materialized view, an external table, or a stream. That rules them out sometimes.

Monitoring things that run themselves

WHAT YOU WANT TO KNOW	WHERE TO LOOK
Is this pipe running, and is it behind?	SYSTEM$PIPE_STATUS
What has Snowpipe cost, and how many files?	PIPE_USAGE_HISTORY
Did these files load, and what went wrong?	COPY_HISTORY. Note that ABORT_STATEMENT runs never appear
Did my task run, and did it fail?	TASK_HISTORY. Set error notifications for failures
Is my stream about to go stale?	STALE_AFTER, via DESCRIBE STREAM or SHOW STREAMS
Is there anything to process right now?	SYSTEM$STREAM_HAS_DATA

ON THE EXAM

Continuous ingestion fails silently by design. Nobody is watching a screen, so these are how you find out.

Five mechanisms, one decision

THE REQUIREMENT	THE MECHANISM
Files land in a bucket and should load within about a minute	Snowpipe with auto-ingest
An application writes rows directly, seconds of latency	Snowpipe Streaming
A single scheduled bulk load, full control over timing	COPY INTO in a task
Process only what changed, with custom logic such as MERGE	A stream plus a task
Keep a derived result fresh, joins and aggregations, minutes of lag	A dynamic table
Always-current derived data, one table, no joins	A materialized view

ON THE EXAM

This extends the three-way tiebreaker from module 4 across the whole ingestion surface. Freshness and control decide it.

Openflow: an honest position

- **Openflow** is Snowflake's data integration service, built on **Apache NiFi**. Connectors move data in from external systems.
- Two deployment models: **Snowflake Deployments** on Container Services, and **BYOC** in your own cloud account.
- The **January 2026 study guide lists it but excludes it from scored questions** 'until it is globally GA'.
- Snowflake Deployments reached GA on **AWS, Azure and GCP in November 2025**, so that exclusion may already have lifted.
- **Know what it is and move on.** Check the current study guide PDF before you sit.

> ON THE EXAM
>
> This is the one genuinely uncertain area on the paper. Recognise the name, do not memorise connector details.

How this module gets tested

THE QUESTION SHAPE	THE ANSWER
'Seconds of latency, application writes rows'	Snowpipe Streaming. Snowpipe is about a minute
'Files loaded twice after a change to the pipe'	Recreating a pipe wipes its 14-day load history
'The stream shows nothing after a long weekend'	Check STALE_AFTER. Retention is extended up to 14 days
'Two processes read the same stream'	Consuming advances the offset. One stream per consumer
'The child task will not accept a schedule'	Only the root task has one. Children use AFTER
'Nothing happens after creating the task'	Tasks are created suspended. Resume it

ON THE EXAM

Six shapes covering the module. Four of the six are about behaviour when something goes wrong, not about configuration.

A stream sits on a table whose DATA_RETENTION_TIME_IN_DAYS is 1. The task that consumes it was suspended by accident on a Friday and nobody noticed until the following Thursday, six days later.

An engineer assumes the stream must be stale and the six days of changes lost. They prepare to drop and recreate it, plus run a manual backfill.

Is the engineer right?

A Yes. With 1-day retention the stream went stale on Saturday and the changes are unrecoverable.

B Yes, but only because the task was suspended. A running task would have kept the offset alive.

C Probably not. Retention is extended for an unconsumed stream up to MAX_DATA_EXTENSION_TIME_IN_DAYS, 14 by default. Check STALE_AFTER.

D No. Streams never go stale on permanent tables, because Fail-safe preserves the change records for 7 days.

◆ PAUSE AND DECIDE

Is the engineer right?

A. Stale on Saturday. Changes lost.

Reasons from the retention setting alone, which is exactly what the extension exists to prevent.

B. Only because the task was suspended.

Backwards. The extension applies precisely because the stream was unconsumed. A consuming task would have advanced the offset instead.

C. Extension up to 14 days. Check STALE_AFTER.

Documented: retention under 14 days is temporarily extended to the stream's offset, bounded by MAX_DATA_EXTENSION_TIME_IN_DAYS.

D. Fail-safe preserves change records.

Fail-safe is Snowflake-only disaster recovery. It has nothing to do with stream offsets.

THE SIGNAL

The extension is 14 days by default regardless of edition. Six days is comfortably inside it, and STALE_AFTER confirms.

> A trading application produces individual events that must be queryable in Snowflake within about ten seconds. The application can call a Snowflake SDK directly. Volumes are high and steady.
>
> An architect proposes having the application write small JSON files to a stage every few seconds, with Snowpipe auto-ingest loading them.

What is wrong with the proposal?

A. Snowpipe targets about a minute of latency and charges per file. Snowpipe Streaming writes rows directly and hits seconds.

B. Snowpipe cannot load JSON. The application must write Parquet for auto-ingest to work.

C. Nothing is wrong, provided ON_ERROR is set to CONTINUE on the pipe.

D. Auto-ingest requires a storage integration, which the application cannot create at runtime.

◆ PAUSE AND DECIDE

What is wrong with the proposal?

A Snowpipe is about a minute and per-file. Streaming writes rows.

Ten seconds is below Snowpipe's target, and tiny files every few seconds is the pattern the file-sizing guidance warns against.

B Snowpipe cannot load JSON.

It loads JSON perfectly well. Format is not the problem here.

C Nothing wrong with ON_ERROR = CONTINUE.

A sensible setting that does nothing about latency or per-file overhead.

D Auto-ingest needs a storage integration.

It needs a notification integration, created once by an administrator. Not something the application does at runtime.

THE SIGNAL

'The application can call an SDK directly' is the planted detail. It rules out the file-based route entirely.

> An engineer builds a three-stage pipeline as a task graph. The root task runs hourly on cron. They give each of the two child tasks a cron schedule as well, five and ten minutes past the hour, to stagger the work.
>
> The statements creating the child tasks fail.

Why, and how should the pipeline be built?

A Cron schedules cannot include minute precision. Use an interval schedule on the children instead.

B Each task in a graph needs its own warehouse. Assign warehouses and the schedules will be accepted.

C A task graph may contain at most two tasks. Split the pipeline into two graphs.

D Child tasks cannot have a schedule. Only the root task does; children are chained with AFTER.

◆ PAUSE AND DECIDE

Why, and how should the pipeline be built?

A Cron lacks minute precision.

Cron expressions include a minute field. Precision is not the problem.

B Each task needs its own warehouse.

Tasks can share a warehouse, or be serverless with no warehouse at all. Unrelated to the failure.

C At most two tasks per graph.

Invented. A task graph is a DAG with one root and as many children as the work needs.

D Only the root task has a schedule. Children use AFTER.

Documented directly, and it is also better design: children run when their predecessor finishes, not at a guessed clock time.

THE SIGNAL

Staggering by clock time is the instinct this rule removes. Dependency ordering is what a task graph exists to provide.

> A pipe has been auto-ingesting files for eight months. To change its COPY statement, an engineer runs CREATE OR REPLACE PIPE with the new definition, then resumes it.
>
> Within an hour, downstream reports show roughly doubled row counts for several recent days. The staged files were never removed, because Snowpipe does not support PURGE.

What happened?

A. The pipe's ON_ERROR reverted to CONTINUE on recreation, so previously rejected rows were loaded.

B. Recreating the pipe wiped its load history, so files still in the stage were treated as new and loaded again.

C. CREATE OR REPLACE PIPE triggers a full REFRESH of every file ever staged, regardless of age.

D. The target table's 64-day load metadata expired, allowing the reload.

◆ PAUSE AND DECIDE

What happened?

A ON_ERROR reverted to CONTINUE.

Would explain a few extra rows, not doubled counts across several days. Rejected rows are a different scale of problem.

B Recreating the pipe wiped its load history.

Load metadata lives on the pipe, not the table. Destroy the pipe and every staged file looks new.

C CREATE OR REPLACE triggers a full REFRESH.

REFRESH is a separate ALTER PIPE operation, and it only covers files staged in the previous 7 days.

D The table's 64-day metadata expired.

Snowpipe does not use the table's 64-day load metadata. Its history is on the pipe and retained 14 days.

THE SIGNAL

Two facts combine: history lives on the pipe, and PURGE is unsupported so the files were still there to reload.

What to carry into the next module

- **Snowpipe loads files, about a minute. Snowpipe Streaming writes rows, seconds.** That is the distinction that gets tested.
- **Pipe metadata is 14 days and lives on the pipe.** Recreating a pipe wipes it. No PURGE. ON_ERROR defaults to SKIP_FILE.
- **A stream is an offset, not a copy.** Consuming advances it, so use one stream per consumer.
- **Staleness is data loss**, but retention is extended up to MAX_DATA_EXTENSION_TIME_IN_DAYS, 14 by default. Check STALE_AFTER.
- **Only the root task has a schedule.** Children use AFTER. Tasks are created suspended.

> ON THE EXAM
>
> Next: connectors and integrations, then five scenarios closing Domain 3.

MODULE 16 · DOMAIN 3 - DATA LOADING, UNLOADING, AND CONNECTIVITY

Connectors, integrations, and the Domain 3 drill

The last pieces of the ingestion surface, then five scenarios across modules 13 to 16.

DOMAIN 3 of 5 WEIGHT 18% MODULE 16 of 31

admin@sthithapragna.com

The connector landscape

CONNECTOR	WHAT IT IS FOR
Kafka connector	Streams Kafka topics into tables. Two ingestion methods, covered next.
Spark connector	Reads and writes Snowflake from an existing Spark cluster.
JDBC and ODBC	Native drivers. Third-party BI and ETL tools, Java and C applications.
Python, Node.js, Go, .NET, PHP	Programmatic interfaces for application code.
SQL API	A REST API. No driver to install, so it suits serverless functions.

ON THE EXAM

All Standard Edition. The choice is about the workload, never the edition.

The Kafka connector has two modes

- **snowflake.ingestion.method** is either **SNOWPIPE**, which is file-based, or **SNOWPIPE_STREAMING**, which writes rows.
- By default the target table has just **two VARIANT columns: RECORD_CONTENT and RECORD_METADATA**.
- **Schema detection and evolution works only with SNOWPIPE_STREAMING.** It is not supported with file-based Snowpipe.
- Enable it with **snowflake.enable.schematization**, plus **ENABLE_SCHEMA_EVOLUTION** on the table, which needs OWNERSHIP.

> ON THE EXAM
>
> The same files-versus-rows split as module 15, surfacing again. Only the row-based mode can shape a table to the data.

Reaching outward from Snowflake

- An **external function** calls code hosted outside Snowflake, through a proxy such as API Gateway. It needs an **API integration**.
- External functions are **not** the same as external tables. One calls a remote service; the other reads remote files.
- **External network access** lets a UDF or stored procedure reach an external endpoint, using a **network rule** with MODE = EGRESS.
- On Business Critical, Amazon API Gateway **private endpoints** are supported for external functions.

> ON THE EXAM
>
> Note the network rule reappearing here in EGRESS mode. Same object as module 9, pointed the other way.

The four integration types, finally side by side

INTEGRATION	WHAT IT CONNECTS TO
Storage	Cloud object storage. S3, Azure Blob, GCS. Used by external stages.
API	An HTTPS endpoint. Used by external functions and by Git repositories.
Notification	Cloud messaging. Used by Snowpipe auto-ingest and by error notifications.
Catalog	An external Iceberg catalog such as AWS Glue or a REST catalog.

ON THE EXAM

Four types, four purposes, never interchangeable. Picking the wrong one is the commonest integration error on this exam.

A team configures the Kafka connector with snowflake.ingestion.method set to SNOWPIPE, and sets snowflake.enable.schematization to TRUE, expecting their JSON messages to arrive as proper typed columns.

Data arrives correctly but every table has only two columns, both VARIANT, named RECORD_CONTENT and RECORD_METADATA.

Why is schematization not working?

A ENABLE_SCHEMA_EVOLUTION defaults to TRUE and must be explicitly disabled before schematization takes effect.

B Schema detection and evolution works only with SNOWPIPE_STREAMING, not with file-based Snowpipe.

C The connector needs a storage integration before it can create columns in a target table.

D JSON cannot be schematized. Only Avro messages carry enough type information.

◆ PAUSE AND DECIDE

Why is schematization not working?

A ENABLE_SCHEMA_EVOLUTION must be disabled.

Backwards, and it defaults to FALSE. It must be set to TRUE, by a role with OWNERSHIP on the table.

B It works only with SNOWPIPE_STREAMING.

Documented explicitly. The file-based Snowpipe method does not support schema detection and evolution at all.

C A storage integration is needed.

Storage integrations serve external stages. Nothing here is reading cloud object storage.

D Only Avro can be schematized.

JSON schematizes perfectly well. The ingestion method is the constraint, not the message format.

WHERE THIS CAME FROM

Module 16, following module 15. The files-versus-rows split decides more than latency: only the row-based path can shape a table.

> A daily COPY loads a partner's CSV export into a wide table. For six weeks it works. Then the partner adds a new column in the middle of their export without telling anybody.
>
> The COPY continues to succeed every night with no errors. Two weeks later an analyst notices that postcodes are appearing in the city column and dates are shifted by one field.

What went wrong, and what prevents it?

A The file format's delimiter changed. Set MATCH_BY_COLUMN_NAME to detect delimiter drift.

B Load metadata expired after 64 days, so files were reloaded against a changed schema.

C ON_ERROR was set to CONTINUE, which silently skipped the malformed rows.

D COPY maps columns by position by default. Use MATCH_BY_COLUMN_NAME to map by name instead.

◆ PAUSE AND DECIDE

What went wrong, and what prevents it?

A. Delimiter drift.

A changed delimiter would produce parse errors, not neatly shifted data. MATCH_BY_COLUMN_NAME does not detect delimiters.

B. 64-day metadata expiry.

That governs whether a file is reloaded. It cannot move values between columns.

C. ON_ERROR = CONTINUE hid it.

CONTINUE skips rows that error. These rows did not error; they loaded successfully into the wrong columns.

D. Positional mapping. Use MATCH_BY_COLUMN_NAME.

Default mapping is by position, so an inserted column shifts everything after it. The load succeeds, which is what makes it dangerous.

WHERE THIS CAME FROM

Module 14. This is the failure mode that produces no error at all, which is why it ran for two weeks before anyone noticed.

> An IoT platform writes one small JSON file per device per minute to an external stage. Around 40,000 files arrive each hour, averaging 15 KB each. Snowpipe auto-ingest loads them.
>
> The data lands correctly and on time, but the Snowpipe line on the bill is far larger than the volume of data suggests it should be.

What is the cause, and what should change?

A. Snowpipe has a per-file overhead charge. Aggregate into fewer, larger files, or use Snowpipe Streaming.

B. Snowpipe is billing warehouse credits. Assign a smaller warehouse to the pipe.

C. Auto-ingest is more expensive than the REST API. Switch the platform to calling the REST API.

D. The pipe needs PURGE = TRUE, because retained staged files are being re-scanned and re-billed.

◆ PAUSE AND DECIDE

What is the cause, and what should change?

A Per-file overhead. Aggregate, or use Streaming.

40,000 files an hour at 15 KB each is the pattern the sizing guidance warns against. Overhead dominates the actual data cost.

B Assign a smaller warehouse.

Snowpipe is serverless. You never assign it a warehouse, so there is nothing to resize.

C The REST API is cheaper.

Both trigger the same loading work. The trigger mechanism is not where the cost is.

D Set PURGE = TRUE.

Snowpipe does not support PURGE at all. Loaded files are also not re-scanned.

WHERE THIS CAME FROM

Modules 13 and 15. The mirror of the one-huge-file scenario: too many tiny files rather than too few large ones.

A pipeline uses an append-only stream on a source table, consumed hourly by a task that inserts new rows into a reporting table. It has run reliably for months.

Auditors find that customer records deleted from the source table are still present in the reporting table, and that corrections applied as updates never reached it.

What is the cause?

A The stream went stale, so deletes and updates were lost when the offset was reset.

B An append-only stream tracks inserts only. Updates and deletes are never recorded.

C The task ran with insufficient privileges to apply deletes, and failed silently.

D Deletes are only visible in a stream if the source table has Time Travel of at least 7 days.

◆ PAUSE AND DECIDE

What is the cause?

A. The stream went stale.

Staleness loses everything unconsumed, not selectively deletes and updates. The pipeline also ran reliably throughout.

B. Append-only tracks inserts only.

Documented. Updates, deletes and truncates are simply not recorded. A standard stream is needed to see them.

C. Insufficient privileges.

The task only inserts, and it never attempted a delete. There was nothing to be denied.

D. Deletes need 7 days of Time Travel.

Invented. Retention governs how long a stream stays valid, not which operations appear in it.

WHERE THIS CAME FROM

Module 15. The pipeline never broke. It was doing precisely what an append-only stream and an insert-only task were built to do.

> A team wants a Snowflake stored procedure to call an internal pricing service over HTTPS, and separately wants Snowflake to read Apache Iceberg tables from a data lake whose catalog is AWS Glue.
>
> An engineer proposes creating one storage integration and using it for both, since both reach outside Snowflake.

What is actually required?

A. One API integration covers both, since Glue is reached over an HTTPS API.

B. A notification integration for the pricing service, and a storage integration for the Iceberg tables.

C. External network access with an EGRESS network rule for the pricing service, and a catalog integration for Glue.

D. The storage integration works for both, provided its allowed locations list includes the pricing service URL.

◆ PAUSE AND DECIDE

What is actually required?

A One API integration for both.

API integrations do serve HTTPS endpoints, and an external Iceberg catalog needs a catalog integration specifically.

B Notification plus storage.

Notification integrations carry cloud messaging events. Neither requirement involves messaging.

C External network access, plus a catalog integration.

A procedure reaching an endpoint uses external network access with an EGRESS network rule. An external Iceberg catalog needs a catalog integration.

D Storage integration with an added URL.

The engineer's proposal. Storage integrations address object storage locations, not service endpoints.

WHERE THIS CAME FROM

Modules 4, 9, 13 and 16 together. Two requirements that both 'reach outside Snowflake' and need entirely different objects.

Where each question came from

QUESTION	MODULE TO REVISIT IF YOU MISSED IT
1. Kafka schematization did nothing	Module 16 - the Kafka connector's two modes
2. Columns silently shifted	Module 14 - MATCH_BY_COLUMN_NAME
3. Snowpipe bill far larger than the data	Modules 13 and 15 - file sizing and per-file overhead
4. Deletes never reached the reporting table	Module 15 - the three stream types
5. Two ways of reaching outside Snowflake	Modules 4, 9, 13 and 16 - the four integration types

ON THE EXAM

Questions 3 and 5 span several modules. Those are the ones worth re-reading even if you got them right.

Domain 3 in five sentences

- **Loading is two steps.** Files reach a stage, then COPY INTO makes them rows. Snowpipe automates step two.
- **File count decides parallelism.** Too few files wastes the warehouse; too many wastes money on overhead. Aim for 100 to 250 MB.
- **Defaults nobody chose:** ABORT_STATEMENT for bulk, SKIP_FILE for Snowpipe, positional column matching, MAX_FILE_SIZE 16 MB.
- **Files or rows** separates Snowpipe from Snowpipe Streaming, and decides Kafka schematization too.
- **Four integration types**, never interchangeable. Storage, API, notification, catalog.

> ON THE EXAM
>
> Next: Domain 4. Performance, querying and transformation at 21%, beginning with warehouse sizing and scaling.

MODULE 17 · DOMAIN 4 - PERFORMANCE OPTIMIZATION, QUERYING, AND TRANSFORMATION

Warehouse sizing, scaling and what it costs

Domain 4 begins. Scale up or scale out is the decision, and the exam asks it repeatedly.

DOMAIN 4 of 5 WEIGHT 21% MODULE 17 of 31

admin@sthithapragna.com

Scale **up** for one slow query. Scale **out** for many queued queries. They fix different problems.

Auto-suspend and auto-resume

- **Auto-suspend** stops a warehouse after a period of inactivity. **Auto-resume** restarts it when a query arrives.
- Every resume bills a **60-second minimum**. A warehouse that resumes twenty times pays that minimum twenty times.
- Suspending **clears the warehouse local disk cache**. The next queries run cold and are slower.
- So the setting is a trade: **short suspend saves idle credits, long suspend preserves the cache**.
- For a warehouse serving frequent short queries, a longer auto-suspend is often **cheaper**, not more expensive.

> ON THE EXAM
>
> That last line is counter-intuitive and it was a Domain 1 drill question. Frequent resumes cost more than staying warm.

Scale up or scale out?

SCALE UP

A larger warehouse size

- One query is slow
- More memory, so less spilling to disk
- More local storage for the cache
- Doubles the credits per hour, and often halves the time

SCALE OUT

More clusters

- Many queries are queuing
- More queries run concurrently
- Each query still runs in one cluster
- Does nothing for a single slow query

THE TIEBREAKER

Read the Query Profile. Spilling means scale up. Queuing means scale out. Module 19 reads both off the profile.

Multi-cluster: two modes

- **Maximized mode:** minimum equals maximum. **All clusters run concurrently** from the moment the warehouse resumes.
- **Auto-scale mode:** maximum is greater than minimum. Clusters start and stop between those bounds.
- **The scaling policy does not apply in Maximized mode**, because nothing ever starts or stops.
- Maximized suits steady, predictable high concurrency. Auto-scale suits workloads that fluctuate.
- Multi-cluster warehouses require **Enterprise Edition** or higher.

> ON THE EXAM
>
> The mode is decided entirely by whether min equals max. There is no separate setting to choose it.

Standard or Economy scaling policy?

STANDARD - THE DEFAULT

Minimise queuing

- Starts a cluster immediately when a query queues
- Subsequent clusters wait 20 seconds each
- Shuts down after 2 to 3 consecutive checks
- Favours starting clusters over conserving credits

ECONOMY

Conserve credits

- Starts a cluster only if load will keep it busy 6 minutes
- Accepts queuing during that evaluation
- Shuts down after 5 to 6 consecutive checks
- Favours keeping running clusters fully loaded

HOW IT ACTUALLY WORKS

Checks run every minute against the least loaded cluster. With Standard and 5 clusters, starting all five takes at least 80 seconds.

What a multi-cluster warehouse costs

- **Credits equal warehouse size, times running clusters, times time.** A Large with 3 clusters is 24 credits an hour.
- The **60-second minimum applies per cluster**, each time that cluster starts.
- **Maximized mode pays for every cluster all the time** the warehouse is running, whether busy or not.
- Setting the **minimum above 1** keeps that many clusters running permanently. Useful for availability, and never free.

> ON THE EXAM
>
> A common accident: setting minimum to 3 for 'reliability' and quietly paying triple for a warehouse that needed one cluster.

Queuing and timeouts

- When a warehouse is at capacity, further queries **queue** rather than failing. Query Profile shows the queued time.
- **STATEMENT_TIMEOUT_IN_SECONDS** aborts a running statement. Default is **172800 seconds, which is 48 hours**.
- **STATEMENT_QUEUED_TIMEOUT_IN_SECONDS** aborts a statement that has waited too long in the queue. **Default is 0, meaning no limit.**
- Both can be set at account, warehouse, user or session level. The **most specific** setting wins.

> ON THE EXAM
>
> Those defaults are why a runaway query can burn credits for two days. Setting a sensible STATEMENT_TIMEOUT is basic hygiene.

Generation 2 warehouses

- **Gen2 standard warehouses are generally available**, set with GENERATION = '2'. Snowflake cites roughly 2.1x faster core analytics.
- **Gen2 tops out at 4X-Large.** 5X-Large and 6X-Large are Gen1-only sizes.
- GENERATION applies to **standard warehouses only**, not to Snowpark-optimized ones.
- Converting a running Gen1 warehouse charges for **both** until in-flight queries finish, and it will not auto-suspend meanwhile.
- Behaviour change bundle **2026_03 makes Gen2 the default** for new standard warehouses where available.

> ON THE EXAM
>
> The exam is written against the classic sizing model. Learn that as the answer, and know Gen2 exists.

Specialised warehouses

- **Snowpark-optimized warehouses** provide more memory per node. That is what they are for.
- **Standard warehouse memory does not increase with size.** An out-of-memory Snowpark job is not fixed by resizing.
- **Snowpark-optimized memory does not increase above Medium** either. Bigger buys cores, not headroom.
- **Snowflake Adaptive Compute** selects size, cluster count and auto-suspend for you. Newer, and awareness only.

> ON THE EXAM
>
> Module 6 built a scenario on the memory rule. It is worth repeating because it contradicts everyone's instinct.

How this module gets tested

THE SYMPTOM	THE ANSWER
One query is slow, and spilling to disk	Scale up. A larger warehouse size
Many queries queuing at peak	Scale out. Multi-cluster, Enterprise Edition
Queuing continues despite multi-cluster	Check the scaling policy. Economy waits 6 minutes
Clusters never start despite the policy	Maximized mode. The policy does not apply
Credits high but the warehouse is mostly idle	Minimum cluster count above 1, or a long auto-suspend
Frequent short queries costing more than expected	The 60-second minimum on every resume

ON THE EXAM

Six symptoms. Notice that three of them are about a setting behaving exactly as documented rather than about a fault.

> A single nightly aggregation over a 12 TB table runs on a Medium warehouse and takes four hours. Query Profile shows 340 GB spilled to local storage and 80 GB spilled to remote storage. There is no queuing.
>
> An engineer proposes converting the warehouse to multi-cluster with a maximum of six clusters.

Will that help, and what should be done?

A Yes. Six clusters give six times the compute, so the query will finish roughly six times faster.

B Yes, but only with the Standard scaling policy. Economy would leave the query queued.

C No. Add a clustering key to the source table, since spilling indicates poor pruning.

D No. A single query runs in one cluster. Scale up instead, which gives more memory and less spilling.

◆ PAUSE AND DECIDE

Will that help, and what should be done?

A Six clusters, six times faster.

The instinct the module exists to correct. A single query cannot be spread across clusters.

B Yes, with the Standard policy.

Right that the policies differ, and no policy makes one query use two clusters.

C Add a clustering key.

Spilling means insufficient memory, not poor pruning. Pruning shows as partitions scanned, which the scenario does not mention.

D One cluster per query. Scale up.

Spilling to local and remote storage is the classic scale-up signal. A larger size gives more memory and more local disk.

THE SIGNAL

Remote spilling especially is a memory shortage. It is the single clearest scale-up signal in the Query Profile.

> A BI warehouse is multi-cluster with a minimum of 1 and a maximum of 8, using the Economy scaling policy. Between 9 and 10 each morning, dashboards are slow and Query Profile shows queries queuing for two to four minutes.
>
> Monitoring shows the warehouse rarely runs more than two clusters, even at the busiest point.

What is the most likely cause?

A. The maximum of 8 is too low for the load. Raise it to 16.

B. The warehouse is in Maximized mode, so it cannot scale beyond its minimum.

C. Economy starts a cluster only if load will keep it busy 6 minutes. A one-hour peak of short queries rarely qualifies.

D. Queuing is unrelated to clusters. Increase STATEMENT_QUEUED_TIMEOUT_IN_SECONDS.

◆ PAUSE AND DECIDE

What is the most likely cause?

A Raise the maximum to 16.

It is only using two of the eight it already has. More headroom it will not use changes nothing.

B It is in Maximized mode.

Minimum 1 and maximum 8 are different, so it is auto-scale. Maximized needs them equal.

C Economy needs 6 minutes of sustained load.

Documented. Short dashboard queries rarely justify a cluster under that rule, so queuing is tolerated by design.

D Raise the queued timeout.

That aborts queries that wait too long. It makes the symptom worse rather than fixing the cause.

THE SIGNAL

'Rarely more than two clusters, with 8 available' is the tell. The ceiling is not the constraint; the policy is.

A team creates a multi-cluster warehouse with MIN_CLUSTER_COUNT = 4 and MAX_CLUSTER_COUNT = 4, and sets SCALING_POLICY = ECONOMY expecting to save credits during quiet periods.

The monthly bill is four times what they expected, and monitoring shows four clusters running whenever the warehouse is up, including overnight when almost nothing runs.

What is happening?

A Economy is misconfigured. It must be set before the clusters are created to take effect.

B Minimum equals maximum, so this is Maximized mode. All clusters always run and the scaling policy does not apply.

C Economy only reduces clusters after 5 to 6 checks, so overnight the checks never complete.

D Four clusters are the minimum billable unit above X-Small. Reduce the warehouse size instead.

◆ PAUSE AND DECIDE

What is happening?

A. Economy set too late.

Scaling policy can be altered at any time. Ordering is not the issue.

B. Min equals max, so Maximized. The policy does not apply.

The mode is decided by the counts. With them equal, all clusters run concurrently and nothing ever starts or stops.

C. Overnight checks never complete.

Plausible-sounding, and the checks are not running at all. In Maximized mode there is no scale-down process.

D. Four clusters is a minimum billable unit.

Invented. A single-cluster warehouse of any size is perfectly valid.

THE SIGNAL

There is no setting that selects the mode. Equal counts mean Maximized, and the scaling policy is then silently ignored.

> A team runs a very large nightly transformation on a 5X-Large standard warehouse. Having read about the performance improvement, they attempt to convert it to Generation 2 by setting GENERATION = '2'.
>
> The statement fails. They ask whether Gen2 is unavailable in their region.

What is the most likely explanation?

A Gen2 tops out at 4X-Large. 5X-Large and 6X-Large are Gen1-only sizes.

B Gen2 requires Business Critical Edition, which the account does not have.

C GENERATION can only be set at creation. Recreate the warehouse with the clause included.

D Gen2 requires the warehouse to be suspended, and theirs was running.

◆ PAUSE AND DECIDE

What is the most likely explanation?

A Gen2 tops out at 4X-Large.

Documented. 5X-Large and 6X-Large exist only as Gen1 sizes, so a 5X-Large cannot be converted.

B Gen2 needs Business Critical.

It is not edition-gated. Warehouse generation is unrelated to the edition matrix.

C GENERATION is creation-only.

An existing warehouse can be converted, though converting a running one charges for both until in-flight queries finish.

D It must be suspended first.

Conversion works on a running warehouse. The documented consequence is double billing, not a failure.

THE SIGNAL

To use Gen2 they must drop to 4X-Large. Whether that is a net gain depends on whether the 2.1x claim offsets the smaller size.

What to carry into the next module

- **Scale up for one slow query. Scale out for many queued queries.** A single query runs in one cluster.
- **Min equals max is Maximized**, and the scaling policy is then ignored. Max above min is auto-scale.
- **Standard starts immediately, Economy waits 6 minutes.** Shutdown after 2 to 3 checks, or 5 to 6.
- **Credits equal size times clusters times time.** The 60-second minimum applies per cluster.
- **Gen2 stops at 4X-Large.** Standard warehouse memory does not grow with size at all.

> ON THE EXAM
>
> Next: the three caches, and a decision tree for why a query is slow.

MODULE 18 · DOMAIN 4 - PERFORMANCE OPTIMIZATION, QUERYING, AND TRANSFORMATION

The three caches, and why a query is slow

Three caches in two layers, and a decision tree for the commonest question in this domain.

DOMAIN 4 of 5 WEIGHT 21% MODULE 18 of 31

admin@sthithapragna.com

Two caches live in **cloud services**. One lives **on the warehouse**. That single fact sorts almost everything.

The three caches

CACHE	WHERE IT LIVES	WHAT IT HOLDS
Result cache	Cloud services	The complete result of a previous query.
Metadata cache	Cloud services	Micro-partition statistics: row counts, value ranges, distinct counts.
Warehouse local disk	On the warehouse	Micro-partition data recently read by that warehouse, on local SSD.

ON THE EXAM

Only the third needs a running warehouse. Only the third is lost when the warehouse suspends.

The result cache: how long it lasts

- A result persists for **24 hours from its last use**. Every reuse **resets** that 24-hour window.
- There is an absolute ceiling of **31 days from first execution**. After that it is purged and regenerated.
- Reuse consumes **no warehouse compute at all**, because the result comes from cloud services.
- Controlled by **USE_CACHED_RESULT**, settable at **account, user or session** level. Enabled by default.

> ON THE EXAM
>
> Candidates remember the 24 hours and drop the 31-day ceiling. A harder question targets exactly that gap.

The result cache: when it will not be reused

- **The query text must match exactly.** A change of case, a table alias, or extra whitespace prevents reuse.
- **The data must be unchanged.** Any DML on a referenced table invalidates it.
- **The micro-partitions must be unchanged**, so **reclustering also invalidates it**, even with no DML.
- The role must hold the **required privileges on every table** involved.
- No functions evaluated at execution time, such as **CURRENT_TIMESTAMP or RANDOM**. No UDFs or external functions.

> ON THE EXAM
>
> Five ways to miss the cache, and the first is the one people never suspect. Formatting a query differently is enough.

The other use of persisted results

- **RESULT_SCAN** queries the result of a previous statement as if it were a table, using its query ID.
- **RESULT_SCAN(LAST_QUERY_ID())** is the common form. Build a query step by step without recomputing earlier stages.
- It is the standard way to filter or join the output of **SHOW**, **DESCRIBE** and **CALL**, which return results that are otherwise hard to reuse.
- It reads the **same persisted results** as the result cache, so the same 24-hour window applies.

ON THE EXAM

Retrieval optimisation and post-processing are two uses of one mechanism. RESULT_SCAN is the second.

The metadata cache

- Answers from **micro-partition statistics** rather than from data. Row counts, value ranges, distinct counts.
- **SELECT COUNT(*)** on a large table can return instantly with **no warehouse running** and no credits consumed.
- Some MIN and MAX queries are answered the same way, where the recorded statistics are sufficient.
- This is the same metadata that drives **pruning** in module 3. One store, two uses.

> ON THE EXAM
>
> If a scenario says a query on a huge table returned in milliseconds with nothing running, this is usually the answer.

The warehouse local disk cache

- Holds micro-partition data that **this warehouse** has recently read, on its local SSD.
- **Dropped when the warehouse suspends.** The next queries run cold and re-read from storage.
- **Specific to one warehouse.** Another warehouse reading the same table builds its own cache from scratch.
- **Larger warehouses have more local storage**, so they hold more of the working set.

> ON THE EXAM
>
> This is the other half of the auto-suspend trade from module 17. Suspending saves credits and throws away the cache.

Why is this query slow?

01

Queued?

Waiting for compute. Scale out, or reduce concurrency.

02

Scanning too much?

Partitions scanned near total. Clustering, or a better filter.

03

Spilling?

Bytes spilled local or remote. Scale up for more memory.

04

Exploding join?

Rows out far exceed rows in. Fix the join condition.

THE DECISION TREE

Four questions in order. Each has a different remedy, and the Query Profile answers all four. Module 19 reads them.

How this module gets tested

THE OBSERVATION	THE EXPLANATION
Instant result, no warehouse running, no credits	Result cache, or metadata cache for COUNT(*)
Same query, same data, but no cache hit	Query text differs. Case, alias or whitespace
Cache hit stopped after weeks of daily use	The 31-day ceiling from first execution
Cache lost after a maintenance job, with no DML	Reclustering changed the micro-partitions
First queries slow after every idle period	Warehouse cache dropped on suspend
Benchmarking shows impossibly fast repeat runs	Set USE_CACHED_RESULT = FALSE for the session

ON THE EXAM

Six observations. Four of them are the result cache, which tells you where to spend revision time.

A dashboard runs SELECT region, SUM(amount) FROM sales GROUP BY region every ten minutes and always hits the result cache. An analyst copies the query into a worksheet, adds a table alias and reformats the keywords in lowercase, then runs it.

It takes eleven seconds and consumes warehouse credits. The data has not changed and the analyst holds the same role.

Why was the cached result not reused?

A Result cache reuse requires the same session. A new worksheet always starts a new session.

B Aggregations are never cached, because GROUP BY results are recomputed each time.

C The query text must match exactly. A table alias or a change of case prevents reuse.

D The dashboard tool holds an exclusive lock on the cached result while it is scheduled.

◆ PAUSE AND DECIDE

Why was the cached result not reused?

A Reuse requires the same session.

The cache is in cloud services and is account-wide. A different session, user or warehouse can reuse it.

B Aggregations are never cached.

Invented. Aggregations cache like anything else, and they are the queries that benefit most.

C The text must match exactly.

Documented, naming case and table aliases specifically. The analyst made two changes, either of which alone would suffice.

D An exclusive lock.

No such mechanism. Cached results are read-only and shareable.

THE SIGNAL

The scenario removes every other explanation: same data, same role. Only the text changed, and that was enough.

> An operations dashboard refreshes every five minutes with an identical query. The underlying table is loaded continuously by Snowpipe, roughly once a minute.
>
> A team lead expects most refreshes to be free from the result cache and is surprised to find that essentially every refresh consumes warehouse credits.

Why is the cache almost never hit?

A Any DML on a referenced table invalidates the cached result, and Snowpipe is loading every minute.

B The result cache only applies to queries run manually, not to queries issued by a BI tool.

C Five minutes is shorter than the minimum cache retention, so results have not persisted yet.

D Snowpipe writes bypass the cache invalidation mechanism, so the cache is serving stale results instead.

◆ PAUSE AND DECIDE

Why is the cache almost never hit?

A DML invalidates it, and Snowpipe loads every minute.

The data must be unchanged for reuse. A load a minute means almost every five-minute refresh sees changed data.

B Only manual queries are cached.

Invented. The cache does not know or care which client submitted the query.

C Five minutes is below a minimum retention.

There is no minimum. A result is available immediately and lasts 24 hours from last use.

D Snowpipe bypasses invalidation.

Would mean stale results, which is the opposite of the symptom. Invalidation is not optional.

THE SIGNAL

The result cache and continuous ingestion pull against each other. Fresh data is exactly what stops a result being reusable.

> An analytics team reports that the first two or three queries after any quiet period take four to five times longer than usual. Then everything is fast again. The queries are varied, so the result cache does not apply.
>
> The warehouse has AUTO_SUSPEND set to 60 seconds. Finance set it that way to eliminate idle credits.

What is happening, and what is the trade?

A. Auto-resume takes several minutes, so the first queries wait for the warehouse to become available.

B. The metadata cache is rebuilt on resume, which is what the first queries are waiting for.

C. The 60-second minimum billing period throttles the first queries after a resume.

D. Suspending drops the warehouse local disk cache. A longer auto-suspend preserves it, at the cost of idle credits.

◆ PAUSE AND DECIDE

What is happening, and what is the trade?

A Auto-resume takes minutes.

Resume is typically a second or two. It would also affect only the first query, not the first two or three.

B The metadata cache is rebuilt.

The metadata cache is in cloud services and survives suspension entirely. Only the warehouse cache is lost.

C The 60-second minimum throttles queries.

That is a billing rule, not a performance one. It affects what you pay, never how fast a query runs.

D The local disk cache is dropped. Longer suspend preserves it.

Documented. The next queries re-read from storage until the cache warms again.

THE SIGNAL

A 60-second auto-suspend can cost more than it saves: repeated 60-second minimums plus a permanently cold cache.

> A monthly board report runs the same query on the first working day of each month. For over a year it returned instantly at no compute cost. This month it took ninety seconds and consumed credits.
>
> The query text is unchanged, the source table is append-only and was last written to eight months ago, and the role is the same.

What is the most likely explanation?

A USE_CACHED_RESULT must have been set to FALSE at account level.

B The result cache is capped at 31 days from first execution, so a monthly query can never reuse a result.

C Monthly queries fall outside the 24-hour window, so this has always been recomputed and the timing was coincidental.

D Fail-safe expired on the source table, so the cached result could no longer be validated.

◆ PAUSE AND DECIDE

What is the most likely explanation?

A USE_CACHED_RESULT set to FALSE.

Possible, and it would have affected every query in the account, not one monthly report. Nothing suggests a global change.

B The 31-day ceiling makes monthly reuse impossible.

24 hours from last use, capped at 31 days from first execution. A gap of a month exceeds both, so a monthly query always regenerates.

C It has always been recomputed. Coincidence.

Contradicts the scenario, which says it returned instantly at no compute cost for over a year.

D Fail-safe expired.

Fail-safe is disaster recovery for dropped data. It has no connection to result cache validity.

THE SIGNAL

Both limits fail here. 24 hours from last use is exceeded, and so is 31 days from first execution.

What to carry into the next module

- **Result and metadata caches are in cloud services.** No warehouse needed, and they survive suspension.
- **The warehouse local disk cache dies with the warehouse.** That is the other half of the auto-suspend trade.
- **24 hours from last use, 31 days from first execution.** Learn the ceiling, not just the window.
- **Exact text, unchanged data, unchanged partitions, right privileges, no volatile functions.** Five ways to miss.
- **Queued, scanning, spilling, exploding join.** Four questions, four remedies.

> ON THE EXAM
>
> Next: reading a Query Profile. The module that turns this decision tree into something you can actually do.

MODULE 19 · DOMAIN 4 - PERFORMANCE OPTIMIZATION, QUERYING, AND TRANSFORMATION

Reading a Query Profile

The module most likely to move a borderline score. Four numbers answer almost every question.

DOMAIN 4 of 5 WEIGHT 21% MODULE 19 of 31

admin@sthithapragna.com

The profile does not tell you the query is slow. It tells you **which of four things** is wrong.

admin@sthithapragna.com

Finding it, and what is in it

- In Snowsight: **Activity, then Query History**, select a query, then the **Query Profile** tab.
- From a worksheet, there is a **Query Profile link** beside the results.
- Four elements: **Steps**, the **operator tree**, the **node list**, and the **overview**.
- The operator tree is a **DAG**. Darker nodes consumed more resources.

> ON THE EXAM
>
> The profile is available for any query in history, not just one you just ran. That matters for investigating last night's job.

How to read the tree

- Read it **bottom-up**. Leaf scans feed joins, joins feed aggregates, and the top feeds **Result**.
- **Time in a node includes its children**, so the percentages do not sum to 100. Read them as which subtree dominates.
- **Arrows carry row counts.** Look for an operator emitting far more rows than it consumes.
- **Most Expensive Nodes** lists operators taking at least **1%** of execution time. Start there.

> ON THE EXAM
>
> Most Expensive Nodes is the first thing to look at. If the top node is a TableScan, the problem is data volume or pruning.

Execution time breakdown

CATEGORY	WHAT DOMINANCE MEANS
Processing	CPU-bound. The query may simply be big. Filter earlier, or pre-aggregate.
Local Disk I/O	The working set did not fit in memory and paid a disk penalty.
Remote Disk I/O	Blocked on cloud storage. A huge scan, or spilling past local disk. The alarm bell.
Network Communication	Data movement between nodes.
Synchronization	Workers waiting on each other. Often the fingerprint of skew.
Initialization	Overhead of starting the query on the warehouse.

ON THE EXAM

For most queries the time is dominated by Processing or IO. Large Network or Initialization suggests a non-obvious problem.

The statistics that answer the four questions

STATISTIC	WHAT IT TELLS YOU
Partitions scanned, partitions total	Pruning. Close to equal means you are reading almost everything.
Bytes spilled to local storage	The query exceeded memory. Bad.
Bytes spilled to remote storage	It exceeded even local disk. Dramatically worse.
Percentage scanned from cache	How much came from the warehouse local disk cache.
Rows in and rows out on each arrow	Join behaviour. Out far exceeding in is the exploding join signal.

ON THE EXAM

Five numbers. If you learn to find these quickly, you can diagnose almost any profile question on the paper.

Problem one: inefficient pruning

- **Partitions scanned close to partitions total**, on a query that filters to a small slice of the table.
- The most expensive node is usually a **TableScan**, and Remote Disk I/O is high.
- Remedies: a **clustering key** on the filter column, a more selective filter, or fewer columns selected.
- **A bigger warehouse does not fix this.** It reads the same excess data faster and costs more.

> ON THE EXAM
>
> Watch for JoinFilter. Snowflake can derive a range from one side of a join to prune the other, so good pruning may appear without a WHERE clause.

Problem two: queries too large to fit in memory

- **Bytes spilled to local storage**, and worse, **bytes spilled to remote storage**. Local Disk I/O or Remote Disk I/O dominant.
- The expensive node is often a **Sort**, a **Join** or an **Aggregate**. Those are the memory-hungry operators.
- Documented remedies, in order: **size up** for more memory, **optimise the query**, **reduce data scanned** with clustering.
- Snowflake's own guidance: **resizing should come after optimising the workload**, not instead of it.

> ON THE EXAM
>
> Remote spilling is the single clearest scale-up signal in the whole profile. Local spilling is a warning; remote is an emergency.

Problem three: exploding joins

- A **Join node emitting far more rows than it consumed**, and usually topping the expense list.
- Cause: a **missing join condition**, producing a Cartesian product, or one that is **too loose**.
- Check the arrows. Two inputs of a few million rows producing billions is not a compute problem.
- **No warehouse size fixes this.** The query is producing rows that should never have existed.

> ON THE EXAM
>
> This is the one where scaling up is most tempting and most wasteful. The row counts on the arrows give it away immediately.

Problem four: UNION without ALL

- **UNION** eliminates duplicates. **UNION ALL** does not, and is therefore much cheaper.
- In the profile it appears as a **UnionAll operator with an unexpected Aggregate on top**. That Aggregate is doing the deduplication.
- If duplicates are impossible by construction, **UNION ALL** removes that work entirely.
- The cost is invisible in the SQL. Three characters separate a cheap query from an expensive one.

> ON THE EXAM
>
> The clue is an Aggregate operator you did not ask for. Nothing in the SQL says GROUP BY, and there it is.

Symptom, node, remedy

WHAT YOU SEE	MOST LIKELY NODE	WHAT TO DO
Partitions scanned near total	TableScan	Clustering key, or a more selective filter
Bytes spilled to remote storage	Sort, Join or Aggregate	Scale up. Then optimise the query
Rows out far exceeding rows in	Join	Fix the join condition. Do not resize
An Aggregate nobody asked for	UnionAll	Use UNION ALL if duplicates are impossible
Queued time before execution	None. It had not started	Scale out, or reduce concurrency
High Synchronization	Any	Skew. Workers waiting on one overloaded colleague

ON THE EXAM

Six rows covering the module. Rows 1 to 4 are Snowflake's four documented problems, in order.

> A dashboard query filtering to a single account_id over a 9 TB events table takes 90 seconds. Query Profile shows: TableScan is 81% of execution time; partitions scanned 118,400 of 121,000; bytes spilled to local storage 0; bytes spilled to remote storage 0; no queued time.
>
> Remote Disk I/O dominates the time breakdown.

What is the problem, and what should be done?

A. Pruning is failing. The query reads almost every partition to return one account. Cluster on account_id, or filter more selectively.

B. The warehouse is too small. Remote Disk I/O dominance means it needs more memory.

C. An exploding join. TableScan at 81% indicates rows being multiplied downstream.

D. The result cache is not being used. Enable USE_CACHED_RESULT for the dashboard's role.

◆ PAUSE AND DECIDE

What is the problem, and what should be done?

A Pruning is failing. Cluster on account_id.

118,400 of 121,000 partitions is 98% of the table, to return one account. TableScan dominant, and both spill figures are zero.

B Too small. Remote Disk I/O means more memory.

Remote Disk I/O has two causes: spilling, or a huge scan. Both spill figures are zero, so this is the scan.

C An exploding join.

A TableScan is not a join. The scenario gives no row counts suggesting multiplication, and mentions no join at all.

D Enable the result cache.

It is on by default, and a dashboard filtering by varying account_id would rarely repeat a query exactly anyway.

THE SIGNAL

Both spill figures being zero is what separates A from B. Remote Disk I/O without spilling means the query is reading too much.

> A nightly aggregation on a Medium warehouse takes 3 hours. Query Profile shows: Sort is 64% of execution time; bytes spilled to local storage 890 GB; bytes spilled to remote storage 210 GB; partitions scanned 4,100 of 96,000; no queued time.
>
> Local Disk I/O and Remote Disk I/O together dominate the time breakdown.

What is the problem?

A. Inefficient pruning. Add a clustering key to reduce the partitions scanned.

B. Concurrency. Convert the warehouse to multi-cluster with a maximum of four.

C. The query exceeded memory and spilled, including to remote storage. Scale up, then optimise the query.

D. UNION without ALL. The Sort operator is performing duplicate elimination.

◆ PAUSE AND DECIDE

What is the problem?

A Inefficient pruning. Add a clustering key.

4,100 of 96,000 is about 4%. Pruning is working well. This is the number that rules the option out.

B Concurrency. Go multi-cluster.

No queued time, and one query cannot use two clusters. Nothing here is a concurrency problem.

C Memory exceeded, spilling to remote. Scale up, then optimise.

890 GB local and 210 GB remote spill, with Sort dominant and IO dominating the breakdown. The textbook signature.

D UNION without ALL.

That appears as an Aggregate above a UnionAll, not as a Sort. No union is mentioned.

THE SIGNAL

The pruning figure is the deliberate distractor. It looks like a big number until you notice it is 4% of the table.

> A report joining an orders table to a shipments table has started taking hours. Query Profile shows a Join node at 91% of execution time. The arrows show 2.4 million rows entering from one side, 6.1 million from the other, and 14.6 billion rows leaving the join.
>
> Partitions scanned is 6% of total. Bytes spilled to remote storage is large.

What is the root cause?

A. Memory. The remote spilling is the primary problem, so the warehouse should be scaled up.

B. Pruning. 6% is too low, so the join is reading stale partitions.

C. Skew. One worker received most of the rows, which is why the Join node dominates.

D. An exploding join. 2.4 million by 6.1 million producing 14.6 billion means the join condition is missing or too loose.

◆ PAUSE AND DECIDE

What is the root cause?

A Memory. Scale up.

The spilling is real and it is a symptom. 14.6 billion rows will exhaust any warehouse. Fix the cause, not the consequence.

B Pruning. 6% is too low.

6% is excellent pruning. Low is good here, and the option inverts the meaning of the ratio.

C Skew.

Skew shows as high Synchronization time, not as row multiplication. The row counts point elsewhere.

D An exploding join. The condition is missing or too loose.

Rows out exceed rows in by roughly a thousandfold. That is the documented signature, and no warehouse size fixes it.

THE SIGNAL

The spilling is a consequence, not the cause. Scale up and you will spill less while still producing 14.6 billion unwanted rows.

> A query combines twelve monthly partitions of the same table with UNION between each pair. Each month's rows are distinct by construction; no row can appear in two months.
>
> Query Profile shows UnionAll operators, and above them an Aggregate operator taking 47% of execution time. The SQL contains no GROUP BY and no DISTINCT.

What is the Aggregate doing, and what should change?

A It is computing statistics for the optimiser. This is normal overhead and cannot be avoided.

B UNION eliminates duplicates, and that Aggregate is doing it. Use UNION ALL, since duplicates are impossible here.

C It is the result cache being populated. Set USE_CACHED_RESULT = FALSE to remove the overhead.

D It is performing the partition pruning for all twelve scans. Add a clustering key to remove it.

◆ PAUSE AND DECIDE

What is the Aggregate doing, and what should change?

A. Optimiser statistics. Unavoidable.

Statistics come from micro-partition metadata, not from a runtime operator taking 47% of the query.

B. UNION deduplicates. Use UNION ALL.

The documented signature: a UnionAll with an unexpected Aggregate above it doing duplicate elimination. Three characters remove 47% of the work.

C. The result cache being populated.

Populating the cache is not an operator in the tree, and disabling it would make repeat runs slower, not faster.

D. Pruning for the twelve scans.

Pruning happens at the TableScan using metadata, before any data is read. It never appears as an Aggregate.

THE SIGNAL

'No GROUP BY and no DISTINCT' is the whole clue. An Aggregate you did not ask for is doing work you did not know you requested.

What to carry into the next module

- **Start at Most Expensive Nodes.** TableScan means volume or pruning. Join means the join. Sort means memory.
- **Partitions scanned against total** is pruning. Low is good. **Bytes spilled remote** is the clearest scale-up signal.
- **Rows out far exceeding rows in** at a Join is an exploding join. No warehouse size fixes it.
- **An Aggregate you did not ask for**, above a UnionAll, is UNION deduplicating. Use UNION ALL.
- **Time in a node includes its children**, so percentages do not sum to 100.

> ON THE EXAM
>
> Next: clustering, materialized views, search optimization and query acceleration. Four features that all make queries faster.

MODULE 20 · DOMAIN 4 - PERFORMANCE OPTIMIZATION, QUERYING, AND TRANSFORMATION

Clustering, materialized views, search optimization, QAS

Four features that all make queries faster. The whole game is knowing which one fits.

DOMAIN 4 of 5 WEIGHT 21% MODULE 20 of 31

admin@sthithapragna.com

They are not competitors. Each one fixes a **different shape of query**, and two of them can be used together.

The four at a glance

FEATURE	WHAT IT ACTUALLY DOES	EDITION
Clustering key	Reorganises the physical layout so related rows share partitions	Standard
Materialized view	Precomputes and maintains a result over one table	Enterprise
Search optimization	Builds a search access path beside the table for point lookups	Enterprise
Query acceleration	Temporarily leases extra serverless compute for scan and filter work	Enterprise

ON THE EXAM

Only clustering is Standard Edition. If a scenario names Standard, three of these four are already ruled out.

Clustering and materialized views, briefly

- **Clustering** suits **range scans** and equality on **medium-cardinality** columns. Large tables only, and not write-heavy ones.
- Ordered **lowest cardinality first**, three or four columns at most. A unique identifier is the classic bad key.
- **Materialized views** suit a **repeated aggregation** that must always be current, with **no lag**.
- **One table, no joins, no window functions.** That restriction rules them out more often than anything else.

> ON THE EXAM
>
> Modules 3 and 4 in four lines. If either restriction bites, the answer is one of the other two features.

Search Optimization: what it accelerates

- **Selective point lookups** returning one or a few rows from a very large table. High-cardinality columns.
- **Substring and regular expression** searches: LIKE, ILIKE, RLIKE.
- Predicates on **VARIANT, OBJECT and ARRAY**: equality, IN, ARRAY_CONTAINS, ARRAYS_OVERLAP, full-text SEARCH, and NULL checks.
- Selected **geospatial functions** on GEOGRAPHY values.
- Individually supported predicates joined by **AND** remain supported.

> ON THE EXAM
>
> The natural sentence is 'find me the one row with this exact value'. That is a needle in a haystack, not a range scan.

Search Optimization: enabling and cost

- **ALTER TABLE ... ADD SEARCH OPTIMIZATION**, optionally **ON EQUALITY(col)**, **ON SUBSTRING(col)** or **ON GEO(col)**.
- **Enabling at table level does NOT enable it for semi-structured columns.** VARIANT, OBJECT and ARRAY need an explicit ON clause.
- Costs **storage** for the access path, typically about **a quarter of the table's size**, plus **serverless compute** for maintenance.
- Estimate first with **SYSTEM$ESTIMATE_SEARCH_OPTIMIZATION_COSTS**. Building the path can take significant time on a large table.

> ON THE EXAM
>
> That second point is a favourite. A team enables it table-wide, sees no improvement on their JSON queries, and cannot work out why.

Query Acceleration: borrowed compute

- Leases **extra serverless compute temporarily** for one query, then releases it. Your warehouse stays the size it was.
- Enabled per warehouse: **ENABLE_QUERY_ACCELERATION = TRUE**, with **QUERY_ACCELERATION_MAX_SCALE_FACTOR** capping how much it may lease.
- It accelerates the **scan and filter** portions only. **Aggregation and sorting are not accelerated.**
- The shape it suits: **large scans with an aggregation or a selective filter**, especially unpredictable ad hoc ones.

> ON THE EXAM
>
> This is the answer when occasional huge queries would otherwise force you to oversize a warehouse permanently.

Query Acceleration: eligibility and monitoring

- **Ineligible** when: filters are not selective enough, GROUP BY cardinality is too high, certain LIMIT clauses, or non-deterministic functions such as **SEQ or RANDOM**.
- **SYSTEM$ESTIMATE_QUERY_ACCELERATION(query_id)** returns eligible or ineligible, plus estimated times per scale factor.
- **ACCOUNT_USAGE.QUERY_ACCELERATION_ELIGIBLE** lists eligible queries. Latency up to **180 minutes**.
- That view **only shows queries not yet accelerated**. Once QAS is on and working, they disappear from it.
- Now **enabled by default** in Gen2 standard warehouses and multi-cluster warehouses.

> ON THE EXAM
>
> The 'only shows not-yet-accelerated queries' detail explains an empty result that looks like a problem and is actually success.

Snowflake Optima: the free one

- **Standard Edition, no additional cost**, but requires a **Gen2 or Adaptive warehouse**.
- **Optima Indexing** creates hidden indexes for recurring point lookups. **Optima Metadata** improves pruning. **Optima Planning** improves plans.
- **Best-effort.** Search Optimization costs credits and gives **consistent** index freshness for workloads that need guaranteed performance.
- Too new to be scored heavily. **Answer Search Optimization** when a question asks for guaranteed point-lookup performance.

> ON THE EXAM
>
> Recorded here because it looks like a free alternative to Search Optimization. It is not a replacement, and it is not on the exam yet.

The tiebreaker table

THE SYMPTOM	THE FEATURE	WHY
Range scan, poor pruning, medium-cardinality filter	Clustering key	Moves the data so pruning works
Point lookup, high-cardinality column, huge table	Search optimization	An access path for needles
Substring, regex, VARIANT or geospatial	Search optimization	The only one supporting these
Repeated aggregation, one table, always current	Materialized view	Precomputed, no lag
Repeated aggregation across a join, lag acceptable	Dynamic table	Materialized views cannot join
Occasional huge ad hoc scans	Query acceleration	Borrows compute for one query
The account is on Standard Edition	Clustering key	The other three need Enterprise

ON THE EXAM

Seven rows. If you memorise one table from this domain, memorise this one.

> A support application looks up individual customer records by a UUID from a 40 TB table, one row at a time, thousands of times an hour. Query Profile shows partitions scanned close to partitions total on every lookup.
>
> An engineer proposes a clustering key on the UUID column. The account is on Enterprise Edition.

What should be done?

A Proceed with the clustering key on the UUID. Poor pruning always means clustering.

B Enable search optimization on the UUID column. Point lookups on a unique column are exactly what it is for.

C Create a materialized view keyed on the UUID so lookups read the view instead.

D Enable query acceleration on the warehouse, which will accelerate the scans.

◆ PAUSE AND DECIDE

What should be done?

A Clustering key on the UUID.

A unique identifier is the classic bad clustering key. ruinous reclustering cost, and it prunes nothing useful for a range.

B Search optimization on the UUID column.

Selective point lookups on a high-cardinality column are its headline use. It builds an access path rather than moving data.

C A materialized view keyed on the UUID.

A materialized view precomputes an aggregate. It does not act as an index for arbitrary single-row lookups.

D Query acceleration.

QAS speeds up large scans. These are point lookups, and the fix is to avoid scanning rather than to scan faster.

THE SIGNAL

Poor pruning does not always mean clustering. The question is the shape of the query: a needle, not a range.

> An analytics warehouse is a Medium, correctly sized for its normal workload. Two or three times a week an analyst runs an ad hoc query scanning several terabytes with a selective filter. It takes forty minutes and slows everything else.
>
> The queries are unpredictable, filtering on different columns each time. Finance will not approve permanently running a Large warehouse.

What is the appropriate remedy?

A. Add clustering keys to the large tables so the ad hoc scans prune better.

B. Enable search optimization on the tables, which accelerates large scans.

C. Resize to Large anyway. Ad hoc analysis is a legitimate cost.

D. Enable query acceleration on the warehouse, which leases extra compute only for the eligible queries.

◆ PAUSE AND DECIDE

What is the appropriate remedy?

A Clustering keys on the large tables.

'Filtering on different columns each time' rules it out. A clustering key helps the columns it is defined on, and you cannot cover them all.

B Search optimization for large scans.

SOS is for point lookups and substring, VARIANT or geospatial predicates. It is not a large-scan accelerator.

C Resize to Large permanently.

Pays double for the normal workload all week to help a query that runs three times. Finance has already refused.

D Query acceleration on the warehouse.

Large scans with a selective filter are its eligible shape, and it leases compute only when needed rather than permanently.

THE SIGNAL

'Unpredictable, different columns each time' is the planted detail. It eliminates anything you have to configure in advance.

> A finance dashboard runs the same aggregation over a single 3 TB transactions table roughly two hundred times a day. The result must reflect every transaction the instant it is committed, with no staleness whatsoever.
>
> The account is on Enterprise Edition. The query involves no joins and no window functions.

Which feature fits?

A. A dynamic table with TARGET_LAG = '1 minute', which is the minimum available.

B. Query acceleration, which will speed up the aggregation each time it runs.

C. A materialized view, which is maintained automatically and has no lag.

D. Search optimization on the transactions table, which accelerates aggregate queries.

◆ PAUSE AND DECIDE

Which feature fits?

A. A dynamic table at 1 minute lag.

Correct that 1 minute is the minimum, and 'no staleness whatsoever' rules out any lag at all.

B. Query acceleration.

QAS accelerates scan and filter work. Aggregation is explicitly not accelerated, and it recomputes every time regardless.

C. A materialized view.

One table, no joins, no window functions, repeated aggregation, always current. Every condition is satisfied.

D. Search optimization.

Built for point lookups and substring, VARIANT or geospatial predicates. It does not accelerate aggregations.

THE SIGNAL

The scenario states 'no joins and no window functions' deliberately. That sentence exists to confirm a materialized view is legal here.

admin@sthithapragna.com

A team runs ALTER TABLE events ADD SEARCH OPTIMIZATION on a large table. Point lookups on the ordinary id column improve substantially, and the storage bill rises as expected.

However, queries filtering on payload:user.email, a path inside a VARIANT column, are exactly as slow as before. The team believes search optimization is broken.

What is happening?

A Enabling at table level does not cover semi-structured columns. VARIANT needs an explicit ON EQUALITY clause.

B Search optimization does not support VARIANT columns at all. Flatten the payload into typed columns instead.

C The search access path is still building. VARIANT columns are always optimised last.

D The query must cast the VARIANT path explicitly before search optimization will apply.

◆ PAUSE AND DECIDE

What is happening?

A Table-level does not cover semi-structured. Use ON EQUALITY.

Documented explicitly. Queries against VARIANT, OBJECT and ARRAY are not optimised if you omit the ON clause.

B VARIANT is not supported.

It is well supported: equality, IN, ARRAY_CONTAINS, ARRAYS_OVERLAP, SEARCH, substring and NULL checks. It just needs naming.

C Still building, VARIANT last.

Building does take significant time, and there is no documented ordering that leaves VARIANT until the end.

D The path must be cast explicitly.

Casts are supported and sometimes useful, and matching a VARIANT element against a constant works without one.

THE SIGNAL

The other columns improving is the clue. The feature is working; it was simply never switched on for the column that matters.

What to carry into the next module

- **Clustering for ranges, search optimization for needles.** Medium cardinality against high cardinality.
- **Materialized view for zero lag on one table. Dynamic table when a join is involved.**
- **Query acceleration for occasional huge scans** on a warehouse that is otherwise the right size.
- **Table-level search optimization skips semi-structured columns.** VARIANT, OBJECT and ARRAY need an ON clause.
- **Only clustering is Standard Edition.** A scenario naming Standard eliminates the other three.

> ON THE EXAM
>
> Next: SQL features, semi-structured and unstructured data. QUALIFY, PIVOT, MERGE, VARIANT and FLATTEN.

MODULE 21 · DOMAIN 4 - PERFORMANCE OPTIMIZATION, QUERYING, AND TRANSFORMATION

SQL features, semi-structured and unstructured data

Three kinds of data, one SQL dialect. QUALIFY, MERGE, VARIANT, FLATTEN and file URLs.

DOMAIN 4 of 5 WEIGHT 21% MODULE 21 of 31

admin@sthithapragna.com

QUALIFY

- **QUALIFY filters on the result of a window function**, the way HAVING filters on the result of an aggregate.
- You cannot put a window function in a WHERE clause. Without QUALIFY you need a subquery or a CTE.
- The classic use: **keep only the latest row per key**, with ROW_NUMBER partitioned by the key and ordered by timestamp.
- Evaluated **after** window functions are computed, which is why it can see them.

> ON THE EXAM
>
> If a scenario describes deduplicating to the most recent record and offers QUALIFY, that is almost always the intended answer.

Reshaping and merging

- **PIVOT** turns rows into columns; **UNPIVOT** turns columns into rows. Both need the value set specified, or ANY.
- **MERGE** combines insert, update and delete against a target using a source and an ON condition. The upsert.
- MERGE is what a **stream plus task** pipeline uses to apply captured changes, from module 15.
- **Multi-table INSERT** writes one source into several targets in a single statement, conditionally or unconditionally.

> ON THE EXAM
>
> MERGE is the reason streams and tasks beat dynamic tables when the logic is complex. Dynamic tables do not support MERGE.

Sampling and a few others

- **SAMPLE** or **TABLESAMPLE** returns a subset. By fraction of rows, by a fixed number of rows, or by block.
- Row sampling is more accurate; **block sampling is faster** because it selects whole micro-partitions.
- **Common table expressions** with WITH, including **recursive** CTEs for hierarchies.
- **Window functions**: ROW_NUMBER, RANK, LAG, LEAD, and running totals with OVER.

> ON THE EXAM
>
> Sampling is how you test a transformation against a large table without paying to scan all of it.

VARIANT, OBJECT and ARRAY

- **VARIANT** holds any semi-structured value. **OBJECT** is a set of key-value pairs. **ARRAY** is an ordered list.
- **VARIANT is limited to 16 MB per row**, compressed. That bounds how large one document can be.
- **PARSE_JSON** converts a string to a VARIANT. **TO_VARIANT** converts any value. **TO_JSON** goes back to a string.
- In a VARIANT, a JSON null is stored as the string 'null', not a SQL NULL. **STRIP_NULL_VALUES** handles that on load.

> ON THE EXAM
>
> The 16 MB limit is a real constraint, not trivia. A scenario about very large JSON documents is usually pointing at it.

Getting at the values

- **Colon** for the first level: **src:user**. **Dot** or **brackets** below that: **src:user.email** or **src['user']['email']**.
- **Always cast.** src:user.email returns a VARIANT; src:user.email::STRING returns a string.
- Without the cast, values come back **with quotes** and comparisons behave oddly.
- Paths are **case-sensitive**. Bracket notation is required when a key contains a space or a special character.

> ON THE EXAM
>
> 'Why does my value have quotes around it' is the commonest semi-structured complaint, and the answer is a missing cast.

FLATTEN parameters

PARAMETER	MEANING AND DEFAULT
INPUT	Required. The expression to flatten. Must be VARIANT, OBJECT or ARRAY.
PATH	The element to flatten. An empty string flattens the outermost element.
OUTER	Default FALSE. Rows that cannot be expanded are omitted entirely. TRUE keeps one row with NULLs.
RECURSIVE	Default FALSE. Only the element at PATH is expanded. TRUE expands all sub-elements.
MODE	Default BOTH. Or restrict to 'OBJECT' or 'ARRAY'.

ON THE EXAM

OUTER is effectively an outer join. With the default FALSE, a row whose array is empty disappears from your results entirely.

FLATTEN output columns

COLUMN	WHAT IT HOLDS
SEQ	A sequence number per input record. Not guaranteed gap-free or ordered.
KEY	The key, for maps and objects. NULL for arrays.
PATH	The path to the element within the structure.
INDEX	The array index, or NULL if it is not an array.
VALUE	The value of the flattened element. This is the one you usually select.
THIS	The element being flattened. Useful in recursive flattening.

ON THE EXAM

Six fixed columns, always the same. Columns of the correlated source table remain accessible alongside them.

LATERAL FLATTEN

- **LATERAL** lets a table function see columns from the row preceding it in the FROM clause.
- Without LATERAL, FLATTEN has no way to know *which* row's JSON to expand.
- **For single-level arrays, TABLE(FLATTEN(...)) and LATERAL FLATTEN(...) give the same result.**
- The usual shape: **FROM raw, LATERAL FLATTEN(input => raw.payload:events)**.

> ON THE EXAM
>
> Think of LATERAL as 'for each row on the left, run this function using that row's data'. It is a correlated join.

Schema detection and evolution

- **INFER_SCHEMA** inspects staged files and returns the column definitions it detects.
- **CREATE TABLE ... USING TEMPLATE** builds a table from that inferred schema, so you need not write the DDL.
- **ENABLE_SCHEMA_EVOLUTION** on a table lets loads add new columns, and drop NOT NULL from columns missing in new files.
- Setting that parameter requires **OWNERSHIP** on the table. It is what the Kafka connector needed in module 16.

> ON THE EXAM
>
> These are the tools for data whose shape you do not control. Standard Edition, and they work with Parquet, Avro, ORC, JSON and CSV.

The three file URL types

TYPE	FUNCTION	EXPIRY	WHO CAN USE IT
File URL	BUILD_STAGE_FILE_URL	Never expires	Authenticated, with read privileges on the stage
Scoped URL	BUILD_SCOPED_FILE_URL	24 hours	Authenticated, but no stage privileges needed
Pre-signed URL	GET_PRESIGNED_URL	Configurable. Default 3600s, max 7 days	Anyone. The token is in the URL

ON THE EXAM

Scoped URLs are created on demand when a consumer queries a secure view. That is how unstructured data is shared.

The encryption trap, and the FILE type

- Files on an internal stage are **client-side encrypted by default**, with keys owned by Snowflake.
- **Client-side encrypted files are unreadable** through pre-signed, file or scoped URLs. They simply will not open.
- Create the stage with **ENCRYPTION = (TYPE = 'SNOWFLAKE_SSE')** for server-side encryption. GET_PRESIGNED_URL requires it.
- The **FILE data type** holds a reference to a staged file, carrying STAGE, RELATIVE_PATH, both URL forms, CONTENT_TYPE, SIZE and more.

> ON THE EXAM
>
> This has to be decided when the stage is created. Discovering it after loading a million documents is an expensive morning.

How this module gets tested

THE REQUIREMENT	THE ANSWER
Keep only the latest row per key	QUALIFY with ROW_NUMBER
Apply captured changes: insert, update and delete	MERGE
Turn a JSON array into one row per element	LATERAL FLATTEN
Rows with an empty array are disappearing	OUTER => TRUE on FLATTEN
Values come back with quotes around them	Missing ::cast on the path expression
A partner with no Snowflake account must open a file	Pre-signed URL, and server-side encryption on the stage

ON THE EXAM

Six shapes. The last one needs two facts together, which is why it makes a good question.

> A table holds one row per customer update, with many rows per customer. An analyst needs a result containing only the most recent row for each customer, judged by an updated_at timestamp.
>
> They have written a query using ROW_NUMBER partitioned by customer_id and ordered by updated_at descending, and are filtering with WHERE rn = 1. The statement fails.

Why does it fail, and what is the cleanest fix?

A ROW_NUMBER cannot be partitioned. Use RANK instead and filter on that.

B The ORDER BY must be ascending inside a window function. Reverse it and take the last row.

C Deduplication requires a MERGE into a separate table. A SELECT cannot express it.

D A window function cannot be used in WHERE. Use QUALIFY, which filters on window function results.

◆ PAUSE AND DECIDE

Why does it fail, and what is the cleanest fix?

A ROW_NUMBER cannot be partitioned.

Partitioning is its normal use. The problem is where the filter sits, not the function.

B ORDER BY must be ascending.

Invented. Descending is standard for taking the most recent row.

C Deduplication needs a MERGE.

A SELECT expresses this fine. MERGE writes changes to a target; nothing here is being written.

D Window functions cannot be used in WHERE. Use QUALIFY.

WHERE is evaluated before window functions are computed. QUALIFY runs after, which is exactly why it exists.

THE SIGNAL

A subquery or CTE also works and is more verbose. 'Cleanest fix' is what points at QUALIFY.

A raw table has a VARIANT column named payload, containing an array at payload:items. An engineer writes a query selecting from the raw table and from FLATTEN(input => payload:items), expecting one row per item.

The statement fails with an error about an invalid identifier. Colleagues report that a nearly identical query in another schema works.

What is missing?

A PARSE_JSON must wrap the column, because FLATTEN cannot read a VARIANT directly.

B LATERAL. Without it, FLATTEN cannot see the columns of the row preceding it in the FROM clause.

C The MODE parameter must be set to 'ARRAY' before an array can be flattened.

D The array must first be cast with ::ARRAY, since FLATTEN only accepts the ARRAY type.

◆ PAUSE AND DECIDE

What is missing?

A PARSE_JSON must wrap the column.

PARSE_JSON converts a string to a VARIANT. The column is already a VARIANT, so there is nothing to parse.

B LATERAL, so FLATTEN can see the preceding row.

A table function has no connection to preceding rows without LATERAL. Hence the invalid identifier on payload.

C MODE must be 'ARRAY'.

MODE defaults to BOTH, which already covers arrays. It restricts rather than enables.

D Cast with ::ARRAY first.

FLATTEN accepts VARIANT, OBJECT or ARRAY. The input type is not the problem.

THE SIGNAL

The error names an identifier, not a type. That is the clue: FLATTEN cannot resolve 'payload' because it cannot see that row.

A team stores scanned contracts on a named internal stage created with default settings. They generate pre-signed URLs with GET_PRESIGNED_URL and send one to an external auditor who has no Snowflake account.

The URL resolves, but the downloaded file is unreadable. Opening it in Snowsight from the stage works perfectly.

What is happening, and what is required?

A Internal stage files are client-side encrypted by default. The stage needs server-side encryption, SNOWFLAKE_SSE.

B Pre-signed URLs only work on external stages. Move the contracts to S3.

C The auditor needs a Snowflake account. Pre-signed URLs still require authentication.

D The expiration was too short and the token had lapsed. Increase it to the 7-day maximum.

◆ PAUSE AND DECIDE

What is happening, and what is required?

A Client-side encrypted by default. Needs SNOWFLAKE_SSE.

Documented: client-side encrypted files are unreadable through pre-signed, file or scoped URLs.
GET_PRESIGNED_URL requires server-side encryption.

B Pre-signed URLs need an external stage.

They work on internal stages, provided server-side encryption is configured.

C The auditor needs an account.

The defining property of a pre-signed URL is that no authentication is needed. The token is in the URL.

D The token had lapsed.

An expired token fails to resolve. Here it resolved and delivered a file, which was simply unreadable.

THE SIGNAL

'The URL resolves but the file is unreadable' is the exact fingerprint. An auth failure would not deliver a file at all.

> A pipeline loads device telemetry as JSON into a VARIANT column. Most documents load without issue. A small number of devices produce documents that fail to load, and those devices report very long event histories in a single message.
>
> The team has checked the file format and the JSON validates correctly.

What is the most likely cause?

A Deeply nested JSON requires ENABLE_SCHEMA_EVOLUTION on the target table.

B PARSE_JSON cannot handle arrays with more than 1,000 elements. Use TO_VARIANT instead.

C The documents exceed the VARIANT limit. Split each message into several rows, or store only the events array.

D The COPY statement needs MATCH_BY_COLUMN_NAME, because VARIANT columns match by name.

◆ PAUSE AND DECIDE

What is the most likely cause?

A Nesting requires schema evolution.

Schema evolution adds columns as sources change. It has nothing to do with the size or depth of one document.

B PARSE_JSON caps arrays at 1,000 elements.

Invented. There is no element-count limit, only the overall size limit.

C They exceed the 16 MB VARIANT limit.

VARIANT is capped at 16 MB per row. Long event histories in one message is exactly how that limit is reached.

D MATCH_BY_COLUMN_NAME is needed.

That maps staged columns to target columns by name. It does not affect whether a document fits in a VARIANT.

THE SIGNAL

'Most load fine, a few fail, and those few are the largest' is the shape of a size limit rather than a format problem.

What to carry into the next module

- **QUALIFY filters on window functions**, as HAVING does on aggregates. WHERE cannot see them.
- **LATERAL lets FLATTEN see the current row.** OUTER defaults to FALSE, so unexpandable rows vanish.
- **FLATTEN returns SEQ, KEY, PATH, INDEX, VALUE, THIS.** Always cast path expressions with ::type.
- **VARIANT is 16 MB per row.** A JSON null is stored as the string 'null'.
- **File URL never expires, scoped is 24 hours, pre-signed needs no auth.** Internal stages need SNOWFLAKE_SSE for any of them.

> ON THE EXAM
>
> Next: UDFs, stored procedures and Snowpark, then six scenarios closing Domain 4.

MODULE 22 · DOMAIN 4 - PERFORMANCE OPTIMIZATION, QUERYING, AND TRANSFORMATION

UDFs, procedures, Snowpark, and the Domain 4 drill

The last of your own code, then six scenarios across modules 17 to 22.

DOMAIN 4 of 5 WEIGHT 21% MODULE 22 of 31

admin@sthithapragna.com

UDF or stored procedure?

USER-DEFINED FUNCTION

Returns a value into a query

- Always returns a value, explicitly
- Called as part of a SQL expression
- Many can appear in one statement
- The return value is usable directly in SQL

STORED PROCEDURE

Performs operations

- May or may not return a value
- Called with CALL, as its own statement
- One per CALL statement
- CALL provides nowhere to put the return value

A DETAIL WORTH HAVING

Every CREATE PROCEDURE needs a RETURNS clause even if it returns nothing. Without an explicit return it gives NULL.

Getting a procedure's value back anyway

- You **cannot** write x = my_procedure(y). CALL has nowhere to put it.
- Inside a **Snowflake Scripting** block, you can capture the return value in a variable.
- Call the procedure **inside another procedure**, where the handler language can hold the result.
- A procedure returning **tabular data** can be called in the **FROM clause** of a SELECT.
- Call it, then use **RESULT_SCAN** with its statement ID. The same function from module 18.

> ON THE EXAM
>
> Four documented workarounds. If a scenario needs the value inline in ordinary SQL, though, the answer is a UDF.

Owner's rights and caller's rights

- **Owner's rights is the default.** The procedure runs with the **owner's** privileges.
- That is how an owner **delegates an operation** to users who could not otherwise perform it.
- **Owner's rights procedures cannot change session state.** No setting session variables or parameters.
- **Caller's rights** runs with the caller's privileges, and inherits the caller's warehouse, database and schema.
- A caller's rights procedure **can view, set and unset session variables**, and those changes **persist after the CALL**.

> ON THE EXAM
>
> Owner's rights is the security answer: give somebody the ability to do one specific thing without granting the underlying privilege.

Languages and UDF variants

WHAT	DETAIL
Handler languages	SQL, JavaScript, Python, Java and Scala. The same set for both UDFs and procedures.
Scalar UDF	Returns one value per input row. The default kind.
UDTF	A table function. Returns a set of rows rather than one value.
Secure UDF	Hides the definition. Required if the function is to be shared.
Vectorized Python UDF	Processes batches of rows rather than one at a time. Faster for numeric work.
Snowflake Scripting UDF	A SQL UDF containing procedural code. Returns a scalar, and is callable in a SELECT.

ON THE EXAM

UDTF is the one to remember. If a scenario needs a function to return several rows per input, a scalar UDF cannot do it.

Snowpark, in one slide

- A **DataFrame API** for Python, Java and Scala. Operations are **lazy** and translate into SQL that runs on a warehouse.
- The point is **pushdown**. Work happens in the engine rather than pulling rows to a client.
- It is also how you author **UDFs, UDTFs and stored procedures** in those languages.
- **Snowpark-optimized warehouses** give more memory per node. Standard warehouse memory does not grow with size.

> ON THE EXAM
>
> Module 6 covered this properly. The line worth carrying is that a Snowpark DataFrame chain appears as ordinary SQL in Query History.

> A team has written a stored procedure that calculates a risk score from several inputs. Analysts now want to use that score inside ordinary queries, as in SELECT customer_id, risk_score(...) FROM customers.
>
> The statement fails. An engineer suggests wrapping the procedure in a view.

What is the correct approach?

A Rewrite the logic as a UDF. A UDF returns a value into a SQL expression; a procedure cannot.

B Grant USAGE on the procedure to the analyst role, which allows inline invocation.

C Wrap the procedure in a view as suggested. Views can call procedures per row.

D Change the procedure to caller's rights, which permits use inside a SELECT.

◆ PAUSE AND DECIDE

What is the correct approach?

A Rewrite it as a UDF.

A UDF is called as part of a SQL expression and returns its value directly. That is precisely what the analysts need.

B Grant USAGE on the procedure.

USAGE lets a role call the procedure. It does not change how a procedure may be invoked.

C Wrap it in a view.

A view is a stored query. It cannot invoke a procedure per row, so the restriction is unchanged.

D Switch to caller's rights.

The rights model governs whose privileges apply, not where the object can be called.

WHERE THIS CAME FROM

Module 22. Procedures are invoked with CALL as their own statement. No privilege or wrapper changes that.

A compliance team must let analysts archive records into a protected audit table. Analysts must not be able to write to that table directly, or read it. A stored procedure has been written to perform the archive correctly.

The procedure runs fine for its owner. When an analyst calls it, it fails with an insufficient privileges error on the audit table.

What is wrong?

A Analysts need INSERT on the audit table. There is no way to avoid granting it.

B The procedure must be recreated as a secure UDF, which masks the underlying object.

C The procedure was created with EXECUTE AS CALLER. Owner's rights runs with the owner's privileges instead.

D Stored procedures cannot write to tables the caller lacks access to, under either rights model.

◆ PAUSE AND DECIDE

What is wrong?

A Grant INSERT to analysts.

Defeats the requirement. They would then be able to write to the audit table directly, which is what must be prevented.

B Recreate as a secure UDF.

Secure UDFs hide a definition. They do not elevate privileges, and a UDF cannot perform the DML anyway.

C It uses caller's rights. Owner's rights delegates the privilege.

Owner's rights runs with the owner's privileges, which is exactly how an operation is delegated without granting the underlying access.

D Neither rights model permits it.

Owner's rights exists precisely to permit it. It is the documented delegation mechanism.

WHERE THIS CAME FROM

Modules 8 and 22. Owner's rights is the default, so somebody explicitly chose caller's rights here.

> A malformed analytical query was submitted on a Friday evening and nobody noticed. It ran continuously until Sunday, consuming credits the whole time on a Large warehouse, before finally being cancelled by hand.
>
> The account has no resource monitor on that warehouse. A team lead asks what would have stopped it automatically.

What is the most direct control?

A AUTO_SUSPEND. A shorter setting would have suspended the warehouse mid-query.

B STATEMENT_TIMEOUT_IN_SECONDS. Its default is 172800 seconds, which is 48 hours.

C STATEMENT_QUEUED_TIMEOUT_IN_SECONDS, which aborts long-running statements.

D Query acceleration, which would have finished the query sooner.

◆ PAUSE AND DECIDE

What is the most direct control?

A. A shorter AUTO_SUSPEND.

Auto-suspend triggers on inactivity. A warehouse running a query is not inactive, so it never fires.

B. STATEMENT_TIMEOUT_IN_SECONDS, default 48 hours.

It aborts a running statement. The 48-hour default is exactly why a runaway query can burn a weekend.

C. STATEMENT_QUEUED_TIMEOUT_IN_SECONDS.

That governs time spent waiting in the queue, not time spent running. Its default is 0, meaning no limit.

D. Query acceleration.

Speeds up eligible queries. A malformed query finishing sooner is not the goal; stopping it is.

WHERE THIS CAME FROM

Module 17. A resource monitor would also have capped the spend, and the question asks for the most direct control on the statement itself.

> A join between a large fact table and a dimension table is slow. Query Profile shows no spilling, partitions scanned at 4% of total, and rows out roughly equal to rows in at the join.
>
> The dominant category in the execution time breakdown is Synchronization, at 58%.

What does that most likely indicate?

A An exploding join. Synchronization time is how row multiplication appears in the profile.

B Inefficient pruning. 4% is too low and the workers are waiting for data.

C The result cache is being populated, which blocks the workers until it completes.

D Data skew. Parallel workers are waiting on one overloaded colleague.

◆ PAUSE AND DECIDE

What does that most likely indicate?

A An exploding join.

That shows as rows out far exceeding rows in. The scenario says they are roughly equal, which rules it out.

B Inefficient pruning at 4%.

Inverts the ratio. 4% of partitions scanned is excellent pruning; low is good.

C The result cache being populated.

Populating the cache is not an operator and does not block workers. It is not in the time breakdown at all.

D Data skew. Workers waiting on one overloaded colleague.

High Synchronization is the documented fingerprint of skew: parallel workers idle while one finishes.

WHERE THIS CAME FROM

Module 19. The scenario rules out the other three documented problems one by one, leaving the category itself as the answer.

A team needs a function that takes a customer id and returns every address that customer has ever used, as multiple rows, so it can be joined into other queries.

They have written a scalar Python UDF that returns an ARRAY of addresses, and are now flattening the array in every query that calls it.

What would be cleaner?

A. A stored procedure returning tabular data, called with CALL before each query.

B. A UDTF, which is a table function returning a set of rows rather than one value.

C. A vectorized Python UDF, which returns batches of rows instead of single values.

D. A secure UDF, which permits multiple return values when the definition is hidden.

◆ PAUSE AND DECIDE

What would be cleaner?

A. A tabular stored procedure.
It can be called in a FROM clause, and it still needs a CALL and cannot be joined per row like a table function.

B. A UDTF.
A table function returns a set of rows per input, which is exactly the requirement. It joins in naturally with LATERAL.

C. A vectorized Python UDF.
Vectorization is about processing input in batches for speed. It still returns one value per row.

D. A secure UDF.
Secure hides the definition. It has no effect on how many values a function returns.

WHERE THIS CAME FROM

Modules 21 and 22. The array-then-flatten pattern is the signal: it is a UDTF written the long way round.

> A team on Standard Edition has a 20 TB table queried with selective range filters on an order_date column. Query Profile shows partitions scanned close to partitions total.
>
> An architect recommends enabling the Search Optimization Service, and separately suggests a materialized view over the filtered result.

What should actually be done?

A Enable search optimization as recommended. Poor pruning is exactly what it fixes.

B Build the materialized view. It precomputes the filtered result and avoids the scan.

C Upgrade to Enterprise, then enable both, since range filters need search optimization.

D Add a clustering key on order_date. Both recommendations require Enterprise Edition.

◆ PAUSE AND DECIDE

What should actually be done?

A Enable search optimization.

Enterprise only, and it targets point lookups. These are range filters, which is clustering's shape.

B Build the materialized view.

Also Enterprise only, and a view filtered to one range answers nothing outside it.

C Upgrade and enable both.

Spends money to reach a worse answer. Even on Enterprise, range filters point at clustering rather than search optimization.

D Clustering key on order_date. The others need Enterprise.

Standard Edition, and a date column with range filters is the textbook clustering case. It fixes the pruning directly.

WHERE THIS CAME FROM

Modules 2, 3 and 20. Two independent reasons for D: the edition, and the shape of the query. Either alone would decide it.

Where each question came from

QUESTION	MODULE TO REVISIT IF YOU MISSED IT
1. Procedure value needed inside a SELECT	Module 22 - UDF or stored procedure
2. Analyst cannot write to the audit table	Modules 8 and 22 - owner's rights
3. Query ran all weekend	Module 17 - STATEMENT_TIMEOUT_IN_SECONDS
4. High Synchronization time	Module 19 - the execution time breakdown
5. Array returned then flattened every time	Modules 21 and 22 - UDTFs
6. Standard Edition and poor pruning	Modules 2, 3 and 20 - the tiebreaker table

ON THE EXAM

Questions 2, 5 and 6 span more than one module. Those are worth re-reading even if you got them right.

Domain 4 in five sentences

- **Scale up for one slow query, scale out for many queued ones.** One query runs in one cluster.
- **Read the profile before spending money.** Expensive nodes, partitions scanned, bytes spilled, rows in and out.
- **Clustering for ranges, search optimization for needles, MVs for zero lag, QAS for occasional giants.**
- **Defaults nobody chose:** 48-hour statement timeout, OUTER FALSE, owner's rights, memory that never grows.
- **A UDF returns a value into a query. A procedure performs work.**

ON THE EXAM

Next: Domain 5. Data collaboration at 10%, beginning with Time Travel, Fail-safe and zero-copy cloning.

MODULE 23 · DOMAIN 5 - DATA COLLABORATION

Time Travel, Fail-safe and zero-copy cloning

Domain 5 begins. Three features built on one idea: micro-partitions are immutable.

DOMAIN 5 of 5 WEIGHT 10% MODULE 23 of 31

admin@sthithapragna.com

Because micro-partitions are **immutable**, the old version of your data still exists. These three features are what you can do with it.

The retention grid

TABLE TYPE	TIME TRAVEL	FAIL-SAFE
Permanent, Standard Edition	0 or 1 day. Default 1	7 days
Permanent, Enterprise or higher	0 to 90 days. Default 1	7 days
Transient	0 or 1 day	None
Temporary	0 or 1 day	None

ON THE EXAM

90 days is Enterprise and permanent. Both conditions must hold, which is why Domain 1's drill closed on exactly that.

Time Travel

- Query data as it was, using **AT** or **BEFORE** with **TIMESTAMP**, **OFFSET** in seconds, or **STATEMENT** with a query ID.
- **STATEMENT is the precise one.** Point at the query ID of the statement that caused the damage and read the state just before it.
- Works in a **SELECT**, in **CREATE … CLONE**, and with **UNDROP**.
- **UNDROP** restores a dropped table, schema or database, provided you are still inside the window. It restores the most recent version.
- Set with **DATA_RETENTION_TIME_IN_DAYS**, which can be set at account, database, schema or table level.

> ON THE EXAM
>
> Setting retention to 0 disables Time Travel entirely, and therefore UNDROP too. That is a deliberate cost decision, not an accident.

Time Travel or Fail-safe?

TIME TRAVEL

Yours

- 0 to 90 days, depending on edition and table type
- You query it yourself with ordinary SQL
- Immediate. AT, BEFORE, UNDROP, CLONE
- You choose the retention, and pay for it

FAIL-SAFE

Snowflake's

- A further 7 days after Time Travel expires
- Only Snowflake can recover from it, via support
- Best-effort, and may take hours or days
- Not configurable, and permanent objects only

THE TIEBREAKER

Fail-safe is a last resort, not a backup. If a scenario needs self-service recovery, Fail-safe is always the wrong answer.

Zero-copy cloning

- **A metadata operation.** The clone points at the same micro-partitions. No data is copied and it is near-instant.
- **No storage cost until it diverges.** Storage grows only as rows are added, deleted or modified in either object.
- The clone is **fully independent**. Changes to one do not affect the other.
- Clone a **table, schema or database**, and with **AT or BEFORE** to clone it as it was at a point in time.
- A clone's **history begins at the moment of cloning**. You cannot Time Travel a clone back to before it existed.

> ON THE EXAM
>
> Cloning a 40 TB production database for testing takes seconds and costs nothing on day one. The cost arrives as the clone diverges.

What a clone does not inherit

- **Load history.** A cloned table has none, so **files already loaded into the source can be loaded again into the clone.**
- **Automatic Clustering starts suspended** on the clone, even if it is active on the source. The clustering key itself is replicated.
- **Cloned tasks are created suspended** and must be resumed.
- **Grants.** CREATE … CLONE does not copy grants for most objects. Use **COPY GRANTS**.
- **Unconsumed records in cloned streams are inaccessible**, consistent with how Time Travel behaves for tables.

> ON THE EXAM
>
> The load history one causes real duplication. A team clones prod to dev, re-runs the pipeline, and loads the same files twice.

The container subtlety with grants

- When you clone a **database or schema**, the clone **inherits granted privileges on the clones of all child objects**.
- But **the clone of the container itself does not inherit** the privileges granted on the source container.
- So the cloned tables keep their grants, and nobody can reach them, because the schema grant is missing.
- Clones also inherit any **future grants** defined for that object type in the schema.

> ON THE EXAM
>
> This is why a cloned database looks correctly permissioned and is still inaccessible. Recall module 8: you need USAGE on the container too.

How this module gets tested

THE REQUIREMENT	THE ANSWER
Undo a bad UPDATE from twenty minutes ago	Time Travel. AT or BEFORE with STATEMENT or OFFSET
Recover a table somebody dropped this morning	UNDROP, if still inside the retention window
Recover data from six weeks ago on Standard Edition	Not possible. Standard permanent tops out at 1 day
Give the test team a copy of production today	Zero-copy clone. Near-instant, no storage until it diverges
Recover after Time Travel has expired	Fail-safe, via Snowflake Support. Permanent objects only
The cloned pipeline loaded everything twice	A clone has no load history

ON THE EXAM

Six shapes. The third and the fifth are the ones candidates get wrong, in opposite directions.

> A team clones the production database into a development database so testers can work against realistic data. The development environment runs the same nightly COPY pipeline against the same external stage, which still holds the last thirty days of files.
>
> The next morning, several tables in the development clone hold roughly double the expected rows. The pipeline reported no errors.

What happened?

A The clone shares micro-partitions with the source, so rows loaded into production appeared in the clone as well.

B FORCE = TRUE must have been set on the COPY statement, overriding load metadata.

C A cloned table does not inherit load history, so files already loaded into the source were loaded again.

D Time Travel on the clone replayed the previous day's DML into the new tables.

◆ PAUSE AND DECIDE

What happened?

A The clone shares partitions, so production rows appeared.

Sharing partitions is how cloning works, and the clone is fully independent from the moment it is created. New production rows do not appear in it.

B FORCE = TRUE was set.

Would produce the same symptom, and the scenario says the same pipeline ran unchanged. The clone explains it without anyone changing anything.

C A clone has no load history, so the files loaded again.

Documented. The clone's load metadata is empty, so every file in the stage looks new to it.

D Time Travel replayed DML.

Time Travel lets you read past states. It never replays anything into a table.

THE SIGNAL

Modules 14 and 23 together. The 64-day load metadata belongs to the source table, and the clone starts with none of it.

> A permanent table on Enterprise Edition has DATA_RETENTION_TIME_IN_DAYS set to 3. Eleven days ago a bulk delete removed several million rows in error, and nobody noticed until an audit this week.
>
> A team lead proposes querying the table with AT(OFFSET => -950400) to read the state before the delete.

Will that work, and what are the options?

A. No. Time Travel expired after 3 days. The only remaining route is Fail-safe, via Snowflake Support.

B. Yes. OFFSET is not bounded by DATA_RETENTION_TIME_IN_DAYS, only TIMESTAMP is.

C. No, but raising DATA_RETENTION_TIME_IN_DAYS to 90 now will restore access to the older data.

D. Yes, because Enterprise Edition provides 90 days regardless of the retention setting.

◆ PAUSE AND DECIDE

Will that work, and what are the options?

A No. Fail-safe, via Snowflake Support.

Retention was 3 days, so day 11 is well past it. Fail-safe covers 7 days beyond that, and it is a best-effort support request.

B OFFSET is not bounded.

All three forms of AT and BEFORE are bounded by the retention period. The argument type does not change the window.

C Raise retention now.

Changing the setting affects data going forward. It cannot resurrect history already outside the previous window.

D Enterprise gives 90 days regardless.

Enterprise raises the maximum you may configure. The default is still 1 day, and this table was set to 3.

THE SIGNAL

Enterprise permits up to 90 days. It does not provide them. Somebody has to set the parameter, and here they set 3.

> An analyst ran an UPDATE without a WHERE clause forty minutes ago, overwriting a status column across an entire 8 TB table. Other teams have written legitimate rows to the table since, which must be preserved.
>
> The account is on Enterprise Edition with 7 days of retention. The analyst has the query ID of the bad UPDATE.

What is the cleanest recovery?

A. UNDROP the table, which restores the most recent version before the UPDATE.

B. Raise DATA_RETENTION_TIME_IN_DAYS to 90, then query the table as it was.

C. Contact Snowflake Support to recover from Fail-safe, since the data has been overwritten.

D. CLONE the table BEFORE(STATEMENT => the bad query ID), then merge the correct status values back.

◆ PAUSE AND DECIDE

What is the cleanest recovery?

A UNDROP the table.

UNDROP restores a dropped object. Nothing was dropped here; the table exists and holds wrong values.

B Raise retention to 90 first.

Unnecessary. The event was forty minutes ago and 7 days of retention already covers it comfortably.

C Fail-safe, via Support.

Fail-safe applies only after Time Travel expires. Time Travel is available and immediate here.

D CLONE BEFORE(STATEMENT), then merge back.

STATEMENT points precisely at the bad UPDATE, the clone is instant and free, and merging preserves the legitimate writes since.

THE SIGNAL

'Other teams have written since' rules out any full restore. You need the old values beside the current table, not instead of it.

> An administrator clones a production database to create a staging environment, then tells the QA team it is ready. The QA role held SELECT on every table in production.
>
> QA reports that they cannot see the staging database at all. Checking SHOW GRANTS on the cloned tables shows their SELECT grants are intact.

Why can they not reach the data?

A Cloned objects require COPY GRANTS, without which no grants are inherited at any level.

B The clone of the container does not inherit privileges granted on the source container. USAGE on the database and schema is missing.

C Clones are owned by the cloning role and cannot be granted to other roles until ownership is transferred.

D The QA role needs its secondary roles enabled before grants on cloned objects take effect.

◆ PAUSE AND DECIDE

Why can they not reach the data?

A COPY GRANTS needed, or nothing is inherited.

True for most single objects, and the scenario says the table grants ARE intact. Cloning a database does propagate child grants.

B The container clone inherits nothing. USAGE is missing.

Documented: child object grants are inherited, but the cloned database and schema do not inherit grants on their sources.

C Ownership must be transferred first.

The cloning role owns the clone, and ownership does not prevent granting. Nothing needs transferring.

D Secondary roles must be enabled.

That applies to privileges granted directly to a user under UBAC. These were granted to a role.

THE SIGNAL

Modules 8 and 23. The tables are correctly permissioned and unreachable, because containment still applies to a clone.

What to carry into the next module

- **All three features exist because micro-partitions are immutable.** The old version is still there.
- **Time Travel is yours, Fail-safe is Snowflake's.** 7 days, permanent objects only, support request, best-effort.
- **90 days needs Enterprise AND permanent AND somebody to set the parameter.** The default is 1 day.
- **AT or BEFORE with TIMESTAMP, OFFSET or STATEMENT.** STATEMENT is the precise one. Cloning accepts them too.
- **A clone has no load history**, starts with clustering and tasks suspended, and its container inherits no grants.

> ON THE EXAM
>
> Next: secure sharing, listings, clean rooms and native apps. Sharing without copying, built on the same immutability.

MODULE 24 · DOMAIN 5 - DATA COLLABORATION

Secure sharing, listings, clean rooms and native apps

Four ways to collaborate, and none of them copy the data.

DOMAIN 5 of 5 WEIGHT 10% MODULE 24 of 31

admin@sthithapragna.com

The data never moves. The consumer reads your micro-partitions, pays for their own compute, and can never write.

What can and cannot be shared

SHAREABLE	NOT SHAREABLE
Tables	Non-secure views
External tables and dynamic tables	Stored procedures
Secure views	File stages
Secure materialized views	Tasks
Secure UDFs	Streams
Iceberg tables	

ON THE EXAM

Secure is the theme on the left. A non-secure view leaks its definition and can leak the rows it filters out, which is why it is barred.

Building a share, and what the consumer does

- **CREATE SHARE**, then grant to it: **USAGE on the database**, **USAGE on each schema**, and **SELECT on the objects**.
- Requires **ACCOUNTADMIN** or the **CREATE SHARE** global privilege.
- The consumer creates a **database from the share**. Their administrator grants **IMPORTED PRIVILEGES**, which opens everything in it.
- For finer control, the provider puts **database roles** in the share. The consumer then grants each one to their own account roles.
- Object identifiers must be **uppercase and alphanumeric**, so consumers need no double-quoted identifiers.

> ON THE EXAM
>
> The grant pattern is module 8's containment rule again: USAGE on the database, USAGE on the schema, then the object privilege.

Direct share or listing?

DIRECT SHARE

Same region, same cloud

- Shared straight to named consumer accounts
- Only within one region and cloud platform
- No metadata, no discovery, no usage metrics
- Immediate. Nothing is replicated

LISTING

Any region, any cloud

- Cross-Cloud Auto-Fulfillment replicates for you
- Private to named accounts, or public on Marketplace
- Carries metadata, sample queries, usage metrics
- The provider pays storage and replication

THE TIEBREAKER

This is the distinction that decides most sharing questions. A direct share cannot cross a region boundary, full stop.

Reader accounts, exchanges and the Marketplace

- A **reader account** shares with a consumer who has **no Snowflake account**. The **provider creates and pays for it**.
- **Snowflake Marketplace** is the public catalogue. Listings can be free, trial, or paid.
- A **Data Exchange** is a private, managed group where you control who may publish and who may consume.
- To share across regions *without* a listing you need **replication**, which is **blocked if the database contains external tables**.
- **VPS accounts cannot use the Marketplace or public listings**, from module 2. VPS is not a strict superset.

> ON THE EXAM
>
> Reader accounts are the answer to 'they are not a Snowflake customer'. The provider pays, which is the detail people forget.

Data Clean Rooms

- Two parties analyse **combined** data without either seeing the other's rows. The classic use is audience overlap.
- The provider links tables and defines **templates**, which are the SQL queries that may be run.
- **Join policies** and **column policies** control which columns may be joined on and which may appear in results.
- **Data in a clean room cannot be queried or viewed directly by either party.** It is reachable only through a template.
- Clean rooms are **versioned native applications**, built on the Native Application Framework.

> ON THE EXAM
>
> The signal is 'neither party may see the other's records'. A share cannot do that, because a share grants read access.

The Native Application Framework

- Package logic built from Snowflake primitives, stored procedures, UDFs and shares, and publish it as an **application**.
- The consumer installs it, and **it runs inside their own account**. The logic travels to the data, not the data to the logic.
- Distributed **privately** or on the **Marketplace**, and **versioned**, so consumers can be upgraded.
- The provider's code can be hidden while still operating on the consumer's data.

> ON THE EXAM
>
> That inversion is the point. A share moves read access outward; a native app moves executable logic inward.

How this module gets tested

THE REQUIREMENT	THE ANSWER
Share with an account in the same region	Direct share
Share with consumers in other regions or clouds	A listing, with Cross-Cloud Auto-Fulfillment
Share with someone who has no Snowflake account	A reader account, created and paid for by the provider
Share a filtered slice of a table	A secure view. Non-secure views cannot be shared
Neither party may see the other's records	A data clean room
Run our logic on the customer's data, in their account	A native application

ON THE EXAM

Six shapes. The first two are the pair that decides most questions, and the difference is a region boundary.

> A provider in AWS eu-west-1 has a working direct share used by three consumer accounts in the same region. A fourth consumer, in Azure East US, needs the same data.
>
> An engineer adds the fourth account to the existing share. The statement is accepted but the consumer cannot see anything.

What is required?

A The consumer must grant IMPORTED PRIVILEGES before the share becomes visible.

B A direct share works only within one region and cloud. Use a listing with Cross-Cloud Auto-Fulfillment, or set up replication.

C Cross-cloud sharing requires the consumer to upgrade to Business Critical Edition.

D The share must be recreated with the GLOBAL keyword to span cloud platforms.

◆ PAUSE AND DECIDE

What is required?

A Grant IMPORTED PRIVILEGES.

Genuinely required for a consumer to use a share, and it cannot make a share visible across a region boundary.

B Direct shares are same-region. Use a listing or replication.

Documented. Listings with auto-fulfillment replicate the data product to the consumer's region automatically.

C The consumer needs Business Critical.

Sharing is Standard Edition. Business Critical adds failover and private connectivity, not cross-region sharing.

D Recreate the share with GLOBAL.

Invented keyword. There is no syntax that makes a direct share cross regions.

THE SIGNAL

The statement being accepted is the realistic part. Adding the account succeeds; the data simply never reaches another region.

> A provider wants to share a curated dataset with a partner. Their design includes a view that joins three internal tables, a stream on the main fact table so the partner can process changes incrementally, and a task that refreshes a summary.
>
> Only some of these objects can be added to the share.

Which parts fail, and what should be done instead?

A. Only the task fails. Views and streams are both shareable if the provider owns them.

B. All three fail. Only base tables can ever be added to a share.

C. The view fails only if it is not secure, and the stream and task are shareable objects.

D. The view must be secure; the stream and task cannot be shared. Let the partner create their own stream on the shared objects.

◆ PAUSE AND DECIDE

Which parts fail, and what should be done instead?

A Only the task fails.

Streams cannot be shared either, and a non-secure view cannot be shared at all.

B All three fail. Only base tables.

Too restrictive. Secure views, secure materialized views, secure UDFs, external and dynamic tables are all shareable.

C Only the view is a problem.

Half right about the view, and streams and tasks are both on the not-shareable list.

D Secure the view; the partner makes their own stream.

The documented answer, including the specific guidance not to work around it with a secure view on a stream.

THE SIGNAL

The workaround matters. Snowflake explicitly warns against sharing a secure view built on a stream; the consumer makes their own.

> A retailer and a media company want to know how many customers they have in common, and how those customers responded to a campaign. Neither is willing to send the other any customer records, and neither may see the other's identifiers.
>
> An architect proposes that each creates a secure view over hashed email addresses and shares it with the other.

Why is that insufficient, and what fits?

A. A data clean room. Data linked into it cannot be viewed directly by either party, only queried through approved templates.

B. Nothing is wrong. Hashing the identifiers means neither party sees personal data.

C. A reader account for each party, which isolates the data from both organisations.

D. A native application, which is the only way to run analysis across two accounts.

◆ PAUSE AND DECIDE

Why is that insufficient, and what fits?

A A data clean room.

Purpose-built for this. Neither party can query the linked data directly; join and column policies constrain what may be matched and returned.

B Hashing is enough.

A share still grants read access to every row. Hashes can be matched against a known list, so identifiers remain exposed.

C A reader account each.

Reader accounts serve consumers with no Snowflake account. Both parties here are Snowflake customers, and it isolates nothing.

D A native application.

Clean rooms are built on that framework, and the answer named in a scenario like this is the clean room itself.

THE SIGNAL

'Neither may see the other's identifiers' rules out any share, because a share grants read access to rows by definition.

> A consumer has been given a share and can query the tables in it. They now want to enrich the shared customer table with their own internal segment codes and keep the result for reporting.
>
> An engineer attempts an UPDATE against the shared table to add the codes, and it fails.

What should they do?

A. Ask the provider to grant INSERT and UPDATE on the shared objects.

B. Clone the shared database, which produces a writable copy at no storage cost.

C. Create a table in their own account from a query joining the shared table to their internal data.

D. Nothing. Shared data cannot be combined with local data in the same query.

◆ PAUSE AND DECIDE

What should they do?

A. Ask for INSERT and UPDATE.

Shared access is read-only by design. There is no grant that makes a consumer able to write to a provider's table.

B. Clone the shared database.

Cloning requires ownership-level access to the source. A consumer cannot clone objects in a share.

C. Build their own table from a join.

Shared objects can be joined to local data freely. The result is written to the consumer's own table, which they own and pay to store.

D. Shared data cannot be joined locally.

Joining shared and local data is one of the main reasons to consume a share at all.

THE SIGNAL

Read-only is not a limitation to route around. Query out, write locally, and the consumer pays only for what they store.

What to carry into the next module

- **Nothing is copied.** The consumer reads your partitions, pays their own compute, and can only read.
- **Direct share is same region and cloud. A listing crosses both**, via Cross-Cloud Auto-Fulfillment.
- **Non-secure views, stored procedures, stages, tasks and streams cannot be shared.** Let consumers make their own streams.
- **Reader accounts** serve consumers with no Snowflake account, and the **provider pays**.
- **Clean room** when neither party may see the other's rows. **Native app** when your logic must run in their account.

> ON THE EXAM
>
> Next: replication and failover, then five scenarios closing Domain 5.

MODULE 25 · DOMAIN 5 - DATA COLLABORATION

Replication, failover, and the Domain 5 drill

Business continuity, then five scenarios closing the last domain.

DOMAIN 5 of 5 WEIGHT 10% MODULE 25 of 31

admin@sthithapragna.com

Disaster recovery is **three** features, not one. Replicate, fail over, redirect the clients.

The edition boundary

FEATURE	EDITION
Database replication	Standard. Available to all editions
Replication of other account objects: warehouses, users, roles	Business Critical or higher
Failover and failback	Business Critical or higher
Client Redirect	Business Critical or higher

ON THE EXAM

This one line decides most questions here. You can replicate a database on Standard. You cannot fail over to it.

Replication groups and failover groups

- **A failover group is a replication group that can also fail over.** That is the entire difference.
- A **secondary** group in a target account is **read-only**. Promoting it to primary makes it read-write.
- You may promote **any target account** listed in the group's allowed accounts. Both provide **point-in-time consistency**.
- **Refresh applies only the changes since the previous refresh**, so cost scales with change rather than with size.
- Use **several groups** when some databases need refreshing more often than others.

> ON THE EXAM
>
> Read-only until promoted is the fact that generates scenarios. A perfectly healthy secondary that rejects every write.

Practical constraints worth knowing

- After upgrading to Business Critical, **failover capabilities can take up to 12 hours** to become available.
- **Replicating to a lower-edition account raises an error by default**, so sensitive data cannot land somewhere weaker.
- Replication of a database is **blocked if it contains external tables**.
- Replication works across regions **within a Region Group**. Crossing Region Groups needs Snowflake Support.
- **Optimized refresh** scales refresh time with rate of change rather than object count. Newer, and awareness only.

> ON THE EXAM
>
> The 12-hour delay is the one that catches people mid-incident. You cannot upgrade your way out of an outage this afternoon.

> A company on Standard Edition has been replicating a production database to a second account in another region for six months, and queries it successfully for reporting.
>
> A regional outage takes the primary account offline. The team attempts to promote the secondary database so writes can continue, and the statement fails.

Why, and what was needed?

A Failover requires Business Critical or higher. Database replication alone is Standard, and gives a read-only copy.

B The secondary must be refreshed immediately before promotion, and the primary is unreachable.

C Promotion requires the ORGADMIN role, which the team was not using.

D Secondary databases can never be promoted. A new primary must be built from a clone.

◆ PAUSE AND DECIDE

Why, and what was needed?

A Failover needs Business Critical. Replication alone is Standard.

The documented boundary. Standard replicates a database and gives a read-only secondary; promoting it is a Business Critical capability.

B A refresh is required before promotion.

Refreshing first is good practice for a planned failover. It is not what blocks this, and an unplanned failover accepts data loss up to the last refresh.

C Promotion requires ORGADMIN.

Account administrators manage failover groups. ORGADMIN handles organization-level tasks such as creating accounts.

D Secondaries can never be promoted.

Promotion is exactly what failover is. It simply requires the right edition.

WHERE THIS CAME FROM

Modules 2 and 25. Replicating for six months proves nothing about failover. They are separate features on opposite sides of the line.

> A team on Business Critical has a failover group replicating to a second region, refreshing every fifteen minutes. An analyst in the secondary region reports that they can query everything but every INSERT and UPDATE is rejected.
>
> There is no outage. The primary is healthy and serving production traffic normally.

What is happening?

A The refresh interval is too short, so the secondary is locked during each refresh.

B The analyst's role lacks INSERT and UPDATE on the replicated objects. Grants do not replicate.

C A secondary failover group is read-only. It becomes writable only when promoted to primary.

D Business Critical requires Tri-Secret Secure before secondary accounts accept writes.

◆ PAUSE AND DECIDE

What is happening?

A Locked during each refresh.

Refreshes do not lock the secondary against reads, and a 15-minute interval would not reject writes continuously.

B Missing grants. Grants do not replicate.

Grants on database objects are synchronised when roles are replicated. And the analyst can read everything, so their privileges are working.

C A secondary is read-only until promoted.

Documented. Read-only access is the defining property of a secondary; promotion is what makes it read-write.

D Tri-Secret Secure is required first.

Tri-Secret Secure concerns encryption keys. It has nothing to do with write access on a secondary.

WHERE THIS CAME FROM

Module 25. Nothing is broken. A secondary is a working read-only replica, and that is exactly what it is meant to be.

> A provider needs to give a consumer in another cloud region access to a database. They decide against a listing and set up database replication to a second account in the consumer's region, intending to share from there.
>
> The replication configuration fails. The database contains twelve tables, three secure views, and two external tables over an S3 data lake.

What is the most likely cause?

A. Secure views cannot be replicated, only shared. Convert them to tables first.

B. Replication is blocked when the database contains external tables.

C. Replication cannot be used to feed a share. Only listings support onward sharing.

D. A database can only be replicated if it contains fewer than ten objects per schema.

◆ PAUSE AND DECIDE

What is the most likely cause?

A. Secure views cannot be replicated.
They replicate normally. Secure views are also the standard way to share a filtered slice, so barring them would break sharing generally.

B. External tables block replication.
Documented: replicating a primary database is blocked if one or more external tables exist in it.

C. Replication cannot feed a share.
Replicate-then-share is the documented alternative to a listing for cross-region sharing.

D. A ten-object limit per schema.
Invented. There is no such limit.

WHERE THIS CAME FROM

Modules 24 and 25. The two external tables are the whole question. Twelve tables and three secure views are there to look like the problem.

A team creates a transient table to hold intermediate results, sets DATA_RETENTION_TIME_IN_DAYS to 0 to save cost, and drops it at the end of the pipeline. The next morning they discover the final step failed and they need the intermediate data back.

Someone suggests UNDROP, and failing that, contacting Snowflake Support for Fail-safe recovery.

What can be recovered?

A UNDROP will work, because it operates independently of the retention setting.

B The data is recoverable from the query result cache, since the pipeline ran within 24 hours.

C Fail-safe will recover it after 7 days, as it does for any dropped table.

D Nothing. Retention of 0 means no Time Travel, and transient tables have no Fail-safe.

◆ PAUSE AND DECIDE

What can be recovered?

A UNDROP works regardless of retention.

UNDROP relies on Time Travel. With retention at 0 there is no window, so there is nothing to restore.

B Recover from the result cache.

The result cache holds query results for reuse, not table contents for restoration. There is no way to restore a table from it.

C Fail-safe recovers it.

Fail-safe covers permanent objects only. Its absence is exactly why transient tables are cheaper.

D Nothing. No Time Travel, and transient has no Fail-safe.

Two independent reasons, and either alone is fatal. This is the cost of transient plus zero retention.

WHERE THIS CAME FROM

Module 23. Both choices were deliberate cost savings, and together they removed every recovery route. That is the trade, made explicit.

> A provider agrees to share data with a small partner who is not a Snowflake customer. They create a reader account, load the partner's users into it, and the partner begins running large analytical queries against the shared data.
>
> At the end of the month the provider receives an unexpectedly large bill and asks how to pass the compute cost to the partner.

What is the position?

A The provider creates and pays for reader accounts, including their compute. Costs cannot be passed through Snowflake billing.

B Reader account compute is billed to the consumer automatically once they accept the terms.

C The bill must be a Snowflake error, since reader accounts are read-only and read-only access is free.

D The provider should convert the reader account into a listing, which transfers compute cost to the consumer.

◆ PAUSE AND DECIDE

What is the position?

A The provider pays, and it cannot be passed through.

Reader accounts are created, owned and paid for by the provider. Resource monitors on the reader account are the practical control.

B Billed to the consumer.

That is the ordinary share model, where the consumer has their own account. A reader account belongs to the provider.

C Read-only access is free.

Read-only concerns writes, not cost. Every query consumes warehouse credits regardless of direction.

D Convert to a listing.

A listing needs the consumer to have a Snowflake account. This partner does not, which is why a reader account was chosen.

WHERE THIS CAME FROM

Module 24. The usual billing split does not apply here, because there is no consumer account. The provider owns both sides.

Where each question came from

QUESTION	MODULE TO REVISIT IF YOU MISSED IT
1. Promotion failed on Standard Edition	Modules 2 and 25 - the edition boundary
2. Secondary rejects every write	Module 25 - secondaries are read-only until promoted
3. Replication blocked by external tables	Modules 24 and 25 - replication constraints
4. Transient table with 0 retention, dropped	Module 23 - the retention grid
5. Reader account bill	Modules 11 and 24 - the provider pays

ON THE EXAM

Every question here spans two modules. Domain 5 is small enough that its facts interact constantly.

Domain 5 in five sentences

- **Time Travel is yours, Fail-safe is Snowflake's.** 90 days needs Enterprise, permanent, and somebody to set it.
- **Nothing is copied in a share.** Read-only, provider pays storage, consumer pays compute.
- **Direct share is one region. A listing crosses regions and clouds.** Reader accounts serve non-customers, and the provider pays.
- **Replication is Standard. Failover, failback and Client Redirect are Business Critical.** A secondary is read-only until promoted.
- **Clean room when neither party may see the other's rows.** Native app when your logic must run in their account.

> ON THE EXAM
>
> Next: the confusion pairs module. Twenty things that look alike, gathered from all five domains.

MODULE 26 · EXAM STRATEGY - ALL DOMAINS

Confusion pairs – twenty things that look alike

No new facts. One question per pair, and the answer in a line.

DOMAIN All five WEIGHT 100% MODULE 26 of 31

admin@sthithapragna.com

Almost every wrong answer on this exam is the **other half of a pair**. Learn the pairs and you eliminate distractors on sight.

Pairs 1 to 4: tables and objects

THE PAIR	THE ONE QUESTION THAT SEPARATES THEM
Transient or temporary?	Must it outlive the session? Temporary dies with it. Neither has Fail-safe.
Materialized view or dynamic table?	Is any lag acceptable? No lag means MV. A join means dynamic table.
Secure view or ordinary view?	Does anyone outside see it? Only secure views can be shared.
Iceberg or external table?	Do you need to write? Iceberg can. External tables cannot.

ON THE EXAM

Modules 4 and 15. The materialized view versus dynamic table pair is the most frequently tested on this slide.

Pairs 5 to 8: access and security

THE PAIR	THE ONE QUESTION THAT SEPARATES THEM
SECURITYADMIN or SYSADMIN?	Is it a grant or an object? SECURITYADMIN grants. SYSADMIN creates. Neither does the other.
Account role or database role?	Can it be used with USE ROLE? Only account roles. Database roles must be granted to one.
Network rule or network policy?	Which is enforced? The policy. A rule is a list, and does nothing until a policy uses it.
Key rotation or rekeying?	Is existing data re-encrypted? Rotation, no. Rekeying, yes, and it needs Enterprise.

ON THE EXAM

Modules 8 and 9. The first row is the one candidates guess at most, and it is pure recall.

Pairs 9 to 12: monitoring and compute

THE PAIR	THE ONE QUESTION THAT SEPARATES THEM
ACCOUNT_USAGE or INFORMATION_SCHEMA?	How far back, how fresh? A year with lag, or 7 days with none.
Resource monitor or budget?	Is it serverless? Monitors cannot see serverless. Budgets can.
SUSPEND or SUSPEND_IMMEDIATE?	Do running statements finish? SUSPEND waits. IMMEDIATE cancels.
Maximized or auto-scale?	Does min equal max? If so, the scaling policy is ignored.

ON THE EXAM

Modules 11 and 17. Row two is the one that costs money in real accounts, not just marks.

Pairs 13 to 16: loading

THE PAIR	THE ONE QUESTION THAT SEPARATES THEM
Snowpipe or Snowpipe Streaming?	Files or rows? A minute, or seconds.
Standard or append-only stream?	Need deletes and updates? Append-only records inserts only.
VALIDATION_MODE or VALIDATE?	Forwards or backwards? Files not yet loaded, or a past COPY.
ABORT_STATEMENT or SKIP_FILE?	Which command? Bulk COPY default, or Snowpipe's.

ON THE EXAM

Modules 14 and 15. Row four is the same option with two different defaults, which is exactly why it gets tested.

Pairs 17 to 20: performance and collaboration

THE PAIR	THE ONE QUESTION THAT SEPARATES THEM
Clustering or search optimization?	A range or a needle? Ranges cluster. Point lookups need the access path.
Result cache or warehouse cache?	Does it survive suspension? Cloud services yes, warehouse no.
Time Travel or Fail-safe?	Can you do it yourself? Time Travel yes. Fail-safe is Support only.
Direct share or listing?	Is a region boundary crossed? A direct share cannot cross one.

ON THE EXAM

Modules 18, 20, 23 and 24. All four have appeared as full scenarios earlier.

Three-way confusions

THE THREE	HOW TO SEPARATE THEM
Masking, row access, projection	Values, rows, or presence in the output. Three different questions.
@~, @%table, @stage	User, table, named. Tilde, percent, bare name.
Result, metadata, warehouse cache	Two in cloud services, one on the warehouse. That decides everything else.
Storage, API, notification, catalog	Buckets, HTTPS endpoints, cloud messaging, Iceberg catalogs. Never interchangeable.

ON THE EXAM

Four groups where the exam offers all three or four members as options. Row one is the most frequently tested.

The same word, meaning different things

- **Secure** is a property, not an object type. Secure views, secure UDFs, and Tri-Secret Secure are three different things.
- **Refresh** means a materialized view updating, a dynamic table rebuilding, a pipe rescanning a stage, and a failover group syncing.
- **Stage** is a file location. A **staging table** is an ordinary table. They are unrelated.
- **Clustering** means both the automatic ordering of data and the CLUSTER BY key you define. Natural clustering exists without a key.
- **Share** is an object, a verb, and half of Secure Data Sharing. Read which one a question means.

> ON THE EXAM
>
> Where a term is overloaded, the question's other nouns tell you which meaning applies. Read them before choosing.

> A team needs a derived dataset joining two large tables, refreshed automatically, with no more than a few minutes of staleness. They also want a second derived dataset over a single table that must be exactly current at all times.
>
> Both accounts are Enterprise Edition.

Which objects fit the two requirements?

A Materialized views for both, since both are derived and automatically maintained.

B Dynamic tables for both, since dynamic tables support any query a materialized view does.

C A materialized view for the join, and a dynamic table for the single-table one.

D A dynamic table for the join, and a materialized view for the single-table one.

◆ PAUSE AND DECIDE

Which objects fit the two requirements?

A Materialized views for both.

A materialized view cannot join. The first requirement rules it out immediately.

B Dynamic tables for both.

Dynamic tables have a minimum target lag of 1 minute. 'Exactly current at all times' rules them out for the second.

C MV for the join, dynamic table for the single table.

Both the wrong way round. This is the option for someone who knows the pair exists and cannot remember which way it runs.

D Dynamic table for the join, MV for the single table.

A join forces the dynamic table; zero staleness forces the materialized view. Each requirement points at exactly one object.

THE SIGNAL

Pair 2. Two requirements, deliberately opposite, so you cannot answer with a single preference. C is the trap for half-knowledge.

> A security engineer is asked to stop connections from outside the corporate network, and separately to make sure a new analyst can be created and granted a role by the platform team without involving an account administrator.
>
> They create a network rule listing the office range, and grant SYSADMIN to the platform team's role.

What did they get wrong?

A The network rule needs no policy, but SYSADMIN cannot create users. USERADMIN is required.

B The rule must be added to a network policy and activated, and SYSADMIN cannot create users. USERADMIN or higher is needed.

C Both are correct. Network rules are self-enforcing and SYSADMIN includes user management.

D The network policy is right, but user creation requires ACCOUNTADMIN and cannot be delegated.

◆ PAUSE AND DECIDE

What did they get wrong?

A Rule needs no policy; SYSADMIN cannot create users.

Right about SYSADMIN, wrong about the rule. A rule is a list and enforces nothing on its own.

B Both wrong. Policy needed, and USERADMIN needed.

Two pairs, both got the wrong way round. A rule needs a policy to enforce it, and user creation lives on the USERADMIN branch.

C Both are correct.

Neither is. This is the option for someone who has not met either pair.

D User creation needs ACCOUNTADMIN.

USERADMIN exists precisely so that user management can be delegated without granting ACCOUNTADMIN.

THE SIGNAL

Pairs 5 and 7 together. Getting one right and one wrong lands you on A, which is why partial knowledge is dangerous here.

> An operations team wants an alert when any query exceeds ten minutes, raised within a few minutes of it happening. They also want a quarterly report of every query run against a table over the past nine months, including two tables since dropped.
>
> An architect proposes building both on the same view.

What should each use?

A ACCOUNT_USAGE for both, since it holds a full year and is the authoritative source.

B INFORMATION_SCHEMA for both, since it has no latency and QUERY_HISTORY exists in it.

C INFORMATION_SCHEMA for the alert, ACCOUNT_USAGE for the quarterly report.

D ACCOUNT_USAGE for the alert, INFORMATION_SCHEMA for the quarterly report.

◆ PAUSE AND DECIDE

What should each use?

A ACCOUNT_USAGE for both.

Latency of 45 minutes to 3 hours makes an alert within a few minutes impossible. Right for the report, wrong for the alert.

B INFORMATION_SCHEMA for both.

Its QUERY_HISTORY holds 7 days and excludes dropped objects. Both halves of the report fail.

C INFORMATION_SCHEMA for the alert, ACCOUNT_USAGE for the report.

No latency for the operational need; a year of history plus dropped objects for the audit. Each requirement matched to its strength.

D The other way round.

Both wrong. This is the option for someone who knows the pair and has it inverted.

THE SIGNAL

Pair 9. The architect's instinct to use one view for both is the real error. The pair exists because you cannot have both properties.

A pipeline uses a stream and a task to apply changes from a source table into a reporting table. Deletions in the source are not reaching the reporting table, while inserts arrive correctly.

Separately, the team's Snowpipe ingestion silently skips whole files when one row is malformed, and nobody has changed any settings.

What explains the two behaviours?

A. An append-only stream records inserts only, and Snowpipe's ON_ERROR defaults to SKIP_FILE.

B. A standard stream ignores deletes, and Snowpipe defaults to ABORT_STATEMENT.

C. The stream went stale, and Snowpipe's default is CONTINUE.

D. The task lacks DELETE privilege, and someone set ON_ERROR = SKIP_FILE manually.

◆ PAUSE AND DECIDE

What explains the two behaviours?

A. Append-only stream, and SKIP_FILE is Snowpipe's default.

Both behaviours are documented defaults working correctly. Inserts arriving while deletes do not is the append-only signature.

B. Standard stream ignores deletes; ABORT_STATEMENT default.

Inverted twice. A standard stream tracks deletes, and ABORT_STATEMENT is the bulk COPY default, not Snowpipe's.

C. The stream went stale; CONTINUE is the default.

Staleness loses everything unconsumed, not just deletes. CONTINUE is neither command's default.

D. Missing privilege, and someone set SKIP_FILE.

The scenario says nobody changed settings, and a privilege failure would raise an error rather than silently dropping deletes.

THE SIGNAL

Pairs 14 and 16. Neither behaviour is a fault. Both are defaults nobody chose, which is the recurring shape of this exam.

A warehouse is capped by a resource monitor with a suspend trigger. When it fires mid-afternoon, several long-running analytical queries are killed outright, losing hours of work.

Separately, the same account's Snowpipe and automatic clustering spending has grown steadily and the monitor has never reacted to it.

What explains both, and what should change?

A The monitor uses SUSPEND, which cancels running statements. Serverless spending is capped separately by lowering the quota.

B The monitor uses SUSPEND_IMMEDIATE, which cancels statements. Monitors cannot see serverless spending at all; use a budget.

C Both are the same cause: the quota is too low. Raising it fixes the killed queries and the serverless overspend.

D Neither is expected. Resource monitors never cancel statements and always include serverless credits.

◆ PAUSE AND DECIDE

What explains both, and what should change?

A SUSPEND cancels statements; lower the quota for serverless.

SUSPEND waits for running statements to finish. And no quota change makes a monitor see serverless spending.

B SUSPEND_IMMEDIATE cancels; use a budget for serverless.

Two pairs, both correct. SUSPEND_IMMEDIATE cancels running statements, and budgets are the documented tool for serverless and AI spend.

C One cause: the quota is too low.

Raising the quota delays the trigger and never makes serverless credits visible to a monitor.

D Neither behaviour is expected.

Both are documented. SUSPEND_IMMEDIATE cancels by design, and the serverless blind spot is stated explicitly.

THE SIGNAL

Pairs 10 and 11. Two independent behaviours in one scenario, and both are working as documented.

> A provider builds a view filtering their customer table to one partner's rows and adds it to a share. The share is created successfully but the view cannot be added.
>
> Once that is resolved, they discover the partner's account is in a different cloud region from theirs.

What is needed for each problem?

A. Make the view materialized, and enable Cross-Cloud Auto-Fulfillment on the direct share.

B. Make the view secure, and add the partner account to the existing direct share.

C. Nothing for the view; grant IMPORTED PRIVILEGES, and use replication for the region.

D. Make the view secure, and use a listing, since a direct share cannot cross a region boundary.

◆ PAUSE AND DECIDE

What is needed for each problem?

A Materialize the view; auto-fulfill the direct share.

Secure is the missing property, not materialized. And auto-fulfillment belongs to listings, not to direct shares.

B Secure the view; add the account to the direct share.

Right about the view. Adding an out-of-region account to a direct share is accepted and delivers nothing.

C The view is fine; grant IMPORTED PRIVILEGES.

A non-secure view cannot be added to a share at all, so the first half fails before privileges matter.

D Secure the view, and use a listing.

Only secure views can be shared, and only a listing crosses a region boundary. Each problem gets its own fix.

THE SIGNAL

Pairs 3 and 20. B is the near-miss: correct on the view, and quietly wrong on the region.

> A team uses a transient table for a nightly staging step to save on storage. A developer drops it by mistake at 09:00. At 11:00 they ask whether the data can be recovered.
>
> The table used the default retention setting. The account is on Enterprise Edition.

What is the position?

A. UNDROP should work. A transient table gets 1 day of Time Travel by default, and Fail-safe does not apply to it.

B. Nothing can be done. Transient tables have no Time Travel at all.

C. UNDROP will fail, but Snowflake Support can recover it from the 7-day Fail-safe window.

D. Enterprise gives 90 days of Time Travel, so the table can be recovered at any point in the next three months.

◆ PAUSE AND DECIDE

What is the position?

A UNDROP works. 1 day of Time Travel, no Fail-safe.

Transient tables support 0 or 1 day, defaulting to 1. Two hours is well inside it, and Fail-safe correctly does not apply.

B Transient tables have no Time Travel.

They have 0 or 1 day. It is Fail-safe they lack entirely, which is a different thing.

C Fail-safe can recover it.

Fail-safe covers permanent objects only. Its absence is exactly why transient storage is cheaper.

D Enterprise gives 90 days.

90 days applies to permanent tables. A transient table tops out at 1 day whatever the edition.

THE SIGNAL

Pairs 1 and 19. Both distractors confuse 'no Fail-safe' with 'no Time Travel'. Transient has one and not the other.

> A multi-cluster warehouse is configured with MIN_CLUSTER_COUNT and MAX_CLUSTER_COUNT both set to 3, and AUTO_SUSPEND set to 60 seconds. It serves a dashboard used in bursts through the day.
>
> The bill is high, and users report that the first queries after any quiet spell are noticeably slow.

What explains the two symptoms?

A The scaling policy is set to Economy, which keeps clusters running and delays new ones.

B Three clusters share one local disk cache, so cache misses are inevitable.

C Min equals max, so all three clusters run whenever it is up. And suspending drops the warehouse cache.

D The result cache is being cleared on suspend, so every burst recomputes from scratch.

◆ PAUSE AND DECIDE

What explains the two symptoms?

A The scaling policy is Economy.

With min equal to max the warehouse is Maximized, and the scaling policy is ignored entirely.

B Three clusters share one cache.

Invented. Each cluster has its own local storage; they do not contend over a shared cache.

C Maximized mode, and suspend drops the warehouse cache.

Both symptoms explained. Three clusters bill continuously, and a 60-second auto-suspend throws the local cache away repeatedly.

D The result cache is cleared on suspend.

The result cache lives in cloud services and survives suspension. It is the warehouse cache that is lost.

THE SIGNAL

Pairs 12 and 18. Two settings, two symptoms, and the result-cache option is there for anyone who has the caches the wrong way round.

How to use this module

- **Rewatch this one the day before.** It is deliberately dense and contains nothing you have not already met.
- **Cover the right-hand column and answer from the pair name alone.** If you cannot, that module needs another pass.
- **Watch for the inverted option.** Several of these scenarios offered the pair the wrong way round.
- **Two pairs in one question is the exam's favourite trick.** Knowing one and guessing the other lands you on a distractor.

> ON THE EXAM
>
> Next: how Snowflake writes questions. Then three practice exams under timed conditions.

MODULE 27 · EXAM STRATEGY - ALL DOMAINS

How Snowflake writes questions

Six recurring shapes, and what to do in the room. The last module before the practice exams.

DOMAIN Exam technique WEIGHT - MODULE 27 of 31

admin@sthithapragna.com

This exam rewards **recognition** over recall. The fact is usually easy. Working out **which** fact is being asked for is the hard part.

The format

- **Around 100 questions in 115 minutes. Multiple choice** and **multiple select**.
- **Scaled score, 750 out of 1000 to pass.** Not a raw percentage, so do not compute a target number of correct answers.
- Delivered through **Pearson VUE**, online-proctored or at a test centre. **$175**, and the credential is valid **two years**.
- **Confirm the current count, time and fee on Snowflake's own page before you book.** These do change.

> ON THE EXAM
>
> Scaled scoring means breadth matters. A candidate strong in three domains and blank in two is in more danger than the raw arithmetic suggests.

The timing arithmetic

- 115 minutes for about 100 questions is roughly **69 seconds each**. That is the number to internalise.
- Most questions take **30 seconds**. That is what buys you three or four minutes for the handful that are genuinely hard.
- **The scenario paragraphs are long.** Reading is most of the 69 seconds, which is why recognition beats recall.
- **Flag and move on.** A question you cannot place in 90 seconds will not improve by staring at it.

> ON THE EXAM
>
> Practising at 69 seconds a question is the point of the three practice exams that follow. Pace is a skill, and it is trainable.

The six shapes

THE SHAPE	HOW TO RECOGNISE IT
Constraints that stack	Two limits both bind. Fixing one leaves you no better off.
The true statement that answers nothing	An option is factually correct and irrelevant to what was asked.
Defaults nobody chose	Nothing is misconfigured. A documented default is doing its job.
The inverted pair	The right two concepts, with the halves swapped.
The phrase that decides it	One clause in the scenario eliminates three options at once.
Nothing is broken	The described behaviour is correct, and the premise is the error.

ON THE EXAM

Every scenario in this course used at least one of these. Recognising the shape narrows the options before you think about Snowflake at all.

Shape 1: constraints that stack

- Two independent limits both apply, and an option that fixes only one is offered as the answer.
- Example: 30 days of Time Travel on a **transient** table on **Standard**. Transient caps at 1 day; Standard caps at 1 day. Both must change.
- The half-right option is always present, and it is the commonest way to lose these marks.
- **Ask: have I checked every constraint the scenario stated?** Scenarios rarely mention a limit for decoration.

> ON THE EXAM
>
> Both Domain drills closed on this shape, in Domain 1 and Domain 2. It is the exam's favourite way to separate near-misses from passes.

Shape 2: the true statement that answers nothing

- An option states something entirely correct about Snowflake that has no bearing on the question asked.
- Example: 'SnowSQL automatic upgrades were disabled from version 1.3.0' offered against a connection timing out. True, and irrelevant.
- These are effective because your instinct is to verify each option's **accuracy** rather than its **relevance**.
- **Ask: does this option answer the question in the final sentence?** Not, is it true.

> ON THE EXAM
>
> The habit worth building is re-reading the actual question before choosing. Long scenarios make people answer a question they invented.

Shape 3: nothing is broken

- The behaviour described is **correct**, and the person in the scenario has misdiagnosed it. Sometimes the premise itself is the error.
- Example: a secondary failover group rejecting every write. That is its defining property, not a fault.
- Example: COPY reporting zero rows loaded with no error. Load metadata doing exactly its job.
- The wrong options describe plausible faults, which is what makes them attractive when you expect something to be broken.

> ON THE EXAM
>
> This shape is why 'defaults nobody chose' recurred through the course. Most Snowflake surprises are documented behaviour, not bugs.

Shape 4: the phrase that decides it

- One clause in the scenario eliminates most of the options, and the rest of the paragraph is scenery.
- Examples from this course: **'enforced by the database'** forces hybrid tables. **'shown the relevant passages'** forces Cortex Search.
- **'Filtering on different columns each time'** eliminates anything you configure in advance. **'Nobody changed anything'** points at a default.
- Scenarios are written backwards from the answer, so an oddly specific clause is **load-bearing**, not colour.
- **Ask: which single sentence here could not be removed without changing the answer?**

> ON THE EXAM
>
> This is the highest-value reading habit in the module. Finding the decisive clause often answers the question before you reason at all.

Shape 5: the inverted pair

- An option names the **right two concepts** with the halves the wrong way round. It exists to catch half-knowledge.
- Example: 'a materialized view for the join, a dynamic table for the single table'. Both correct objects, both misapplied.
- It is **more attractive than a plainly wrong option**, because it uses vocabulary you recognise and looks balanced.
- Defence: name the deciding property, not the pair. **'MV cannot join'** resolves it instantly; 'MV versus dynamic table' does not.

> ON THE EXAM
>
> Knowing a distinction exists is not the same as knowing its direction. This shape is built entirely on that gap.

Multiple-select questions

- The prompt tells you how many to pick. **Select two** or **select three** is an instruction, not a hint.
- There is normally **no partial credit**. Two right and one wrong usually scores the same as none.
- Work them as **separate true or false decisions**, one option at a time, rather than looking for a best answer.
- If you are confident about two and torn between two more, the count in the prompt resolves it.
- They take longer than single-answer questions. Budget for that rather than being surprised by it.

> ON THE EXAM
>
> The mode change is the point. Single-answer questions reward comparison; multiple-select rewards independent judgement on each option.

Elimination, and when to guess

- **There is no penalty for a wrong answer.** Never leave a question blank, ever.
- Most questions have **two** options you can eliminate quickly. Do that before thinking hard about the remaining two.
- Options containing **always**, **never** or **only** deserve extra scrutiny, though Snowflake does have genuine absolutes.
- If two options say the same thing differently, **neither is usually right**. The answer is one of the other two.
- An option that would be **disproportionate in real life**, like rebuilding an architecture to change a default, is rarely the answer.

> THE ONE RULE
>
> 'Never leave a blank' sounds obvious and people still do it when the clock runs out. Answer everything, then flag for review.

In the room

- **First pass:** answer everything you know in under a minute. Flag anything slower and keep moving.
- **Second pass:** return to the flagged ones with the clock understood and the pressure lower.
- **Change an answer only for a reason you can name.** A specific recalled fact, yes. A vague feeling, no.
- Read the **last sentence** of the scenario first if the paragraph is long. It tells you what to look for while reading.
- Online-proctored: test your setup the day before. Room checks eat into nobody's time but your own.

> ON THE EXAM
>
> Reading the question before the scenario is the single most useful habit here. It turns reading into searching.

> 'A team runs a nightly job on a Medium warehouse. Query Profile shows partitions scanned at 3% of total and bytes spilled to remote storage of 180 GB. An engineer proposes adding a clustering key to reduce the partitions scanned.'
>
> You are asked what should be done.

Which shape is this, and what does that tell you?

A Constraints that stack. Both the pruning and the spilling must be fixed together.

B The phrase that decides it. '3% of total' is excellent pruning, so the clustering option is eliminated and spilling is the real issue.

C Nothing is broken. Spilling to remote storage is normal for a nightly job.

D The inverted pair. Clustering and search optimization have been swapped.

◆ PAUSE AND DECIDE

Which shape is this, and what does that tell you?

A Constraints that stack.

There is only one problem here. The pruning figure is good, so it is not a second constraint but a distractor.

B The phrase that decides it. 3% eliminates clustering.

One number does all the work. Low partitions scanned means pruning is fine, so the engineer's proposal and every clustering option go.

C Nothing is broken.

180 GB spilled to remote storage is a genuine problem. Something really is wrong.

D The inverted pair.

No pair has been swapped. Only one acceleration feature is mentioned.

THE SIGNAL

Notice the engineer's proposal is wrong. A named character suggesting something is often the distractor, not a hint.

> 'An analyst reports that a query which normally returns instantly now takes forty seconds and consumes credits. The underlying table has not changed, the analyst holds the same role, and the query text is identical apart from being reformatted across several lines.'
>
> You are asked why the result was not reused.

Which shape is this?

A Constraints that stack. Both the reformatting and the role must be addressed.

B The inverted pair. The result cache and the warehouse cache have been swapped.

C The true statement that answers nothing. Every option is factually correct.

D The phrase that decides it, plus elimination by exclusion. Two conditions are ruled out explicitly, leaving the third.

◆ PAUSE AND DECIDE

Which shape is this?

A Constraints that stack.

Only one condition failed. The other two are stated as satisfied, which is the opposite of stacking.

B The inverted pair.

Only one cache is in play. Nothing has been swapped.

C A true statement that answers nothing.

Describes a property of one option, not the structure of the question.

D Exclusion. Two conditions ruled out, leaving the third.

'Table has not changed' and 'same role' are two of the five reuse conditions, stated as satisfied. Reformatting is the third.

THE SIGNAL

Scenarios that tell you what is NOT the problem are doing you a favour. Read those clauses as eliminations, not as background.

> 'A pipeline uses a stream and a task. Inserts reach the target table correctly, but deletions never do. Nobody has changed any configuration since the pipeline was built eight months ago.'
>
> You are asked what is happening.

Which shape is this, and what should you check first?

A Nothing is broken. 'Nobody changed anything' points at a default or a design choice made at build time.

B Constraints that stack. The stream type and the task privileges both need fixing.

C The true statement that answers nothing. Stream staleness is real but irrelevant.

D The phrase that decides it. 'Eight months ago' exceeds every retention window.

◆ PAUSE AND DECIDE

Which shape is this, and what should you check first?

A Nothing is broken. Check the design choice made at build time.

'Nobody changed anything' plus 'works for inserts' says the pipeline is doing what it was configured to do: an append-only stream.

B Constraints that stack.

One cause explains both observations. There is no second constraint.

C A true statement that answers nothing.

Names a property some option might have. It does not describe the question's structure.

D 'Eight months ago' decides it.

Tempting, and it is the wrong reading. Eight months is how long this has worked correctly, not how long data has been retained.

THE SIGNAL

'Nobody changed anything' is one of the most useful phrases on the paper. It almost always means a default or an original design choice.

What to do with the next three modules

- **Sit each practice exam in one go, timed.** Twenty questions in 23 minutes matches the real pace.
- **Score yourself, then read every explanation**, including the ones you got right. The reasoning is the point, not the tally.
- **Note which shape caught you**, not just which fact. A pattern of misses is more useful than a score.
- Leave a day between them if you can. Three exams in one evening measures stamina rather than knowledge.

> ON THE EXAM
>
> Next: three practice exams of twenty questions each, at exam pace, covering all five domains in proportion.

MODULE 28 · EXAM STRATEGY - ALL DOMAINS

Practice exam 1 – twenty questions

Weighted 6/4/4/4/2 across the five domains, at exam pace.

DOMAIN All five WEIGHT 100% MODULE 28 of 31

admin@sthithapragna.com

Before you start

- **Answer every question.** There is no penalty for a wrong answer, in this exam or the real one.
- **Write your answers down** as letters. Do not try to keep twenty of them in your head.
- **Note which shape caught you**, not just which fact. Module 27 named the six.
- Passing here is **15 or better**. Below 12 means go back to the domain the misses cluster in.

> ON THE EXAM
>
> The final two slides map every question to its domain and module, so a miss becomes a specific instruction.

> Every warehouse in an account is suspended. A user runs SELECT COUNT(*) FROM a 40 TB table and receives an answer in under a second, with no credits consumed.

What explains this?

A The query ran on the cloud services warehouse, which is always running.

B Snowflake auto-resumed a warehouse and suspended it again too quickly to bill.

C Row counts are held in micro-partition metadata in the cloud services layer, so no compute was needed.

D The result was served from the warehouse local disk cache.

◆ PAUSE AND DECIDE

What explains this?

A A cloud services warehouse.

There is no such object. Cloud services is a layer, not a warehouse you can run queries on.

B Auto-resume then suspend.

Any resume bills a 60-second minimum. Credits would have been consumed.

C Row counts come from metadata.

The metadata cache answers COUNT(*) from micro-partition statistics without touching data or compute.

D The warehouse local disk cache.

That cache lives on the warehouse and is dropped when it suspends. All warehouses are suspended.

ON THE EXAM

Modules 1 and 18. 'No credits consumed' is the decisive phrase: it rules out anything involving a warehouse.

> A company needs multi-cluster warehouses, materialized views, and 45 days of Time Travel. They do not handle PHI or payment card data, and do not need private connectivity.

What is the minimum edition?

A Enterprise

B Business Critical

C Standard

D Virtual Private Snowflake

◆ PAUSE AND DECIDE

What is the minimum edition?

A Enterprise.

All three requirements land exactly on the Enterprise tier, and nothing named requires more.

B Business Critical.

Adds Tri-Secret Secure, PrivateLink and failover. The scenario explicitly rules out the triggers for those.

C Standard.

Standard has none of the three: no multi-cluster, no materialized views, and 1 day of Time Travel.

D VPS.

Dedicated hardware, and it loses Marketplace access. Vastly more than is needed.

ON THE EXAM

Module 2. The question asks for the minimum, so find the highest requirement and stop. All three sit on the same tier here.

> A developer creates a temporary table in a worksheet session, loads data into it, and asks a colleague to query it from their own session. The colleague reports the table does not exist.

Why?

A The colleague lacks SELECT on the table. Grant it and they will see it.

B Temporary tables must be created in a shared schema to be visible to others.

C The table is still being created. Temporary tables are asynchronous.

D A temporary table exists only within the session that created it and is not visible to any other session.

◆ PAUSE AND DECIDE

Why?

A Grant SELECT.

No grant makes a temporary table visible outside its session. It does not exist there to be granted on.

B Create it in a shared schema.

Schema choice is irrelevant. Session scope is a property of the table type.

C Still being created.

Invented. Table creation is synchronous.

D Session-scoped, invisible elsewhere.

That is the defining property of a temporary table, and it is dropped when the session ends.

ON THE EXAM

Module 4. Transient persists across sessions; temporary does not. That is the whole distinction between them.

> An architect needs Snowflake to run a query and return results from an AWS Lambda function. No driver may be installed in the Lambda runtime, and the team wants the least operational overhead.

Which interface fits?

A The Python connector, packaged as a Lambda layer.

B The SQL API, a REST interface requiring no driver.

C SnowSQL, invoked as a subprocess.

D The JDBC driver via a Java Lambda.

◆ PAUSE AND DECIDE

Which interface fits?

A. Python connector as a layer.

Works, and it is a driver, which the scenario rules out. It also adds packaging overhead.

B. The SQL API.

A REST API with nothing to install, which is exactly why it suits serverless functions.

C. SnowSQL as a subprocess.

SnowSQL is an interactive client and is documented as legacy. Shelling out to it from Lambda is fragile.

D. JDBC via Java.

Still a driver, and it changes the runtime language to avoid a constraint that applies either way.

ON THE EXAM

Module 5. 'No driver' plus 'least operational overhead' is the SQL API's exact use case.

> A team wants to filter a table down to rows whose free-text feedback column expresses dissatisfaction, without writing a classifier or moving data out of Snowflake.

Which is the most direct approach?

A AI_FILTER in the WHERE clause, which returns true or false per row.

B Cortex Search, which retrieves the relevant rows.

C Cortex Analyst with a semantic model over the feedback column.

D Export the column and call an external sentiment API via an external function.

◆ PAUSE AND DECIDE

Which is the most direct approach?

A AI_FILTER in WHERE.

Returns true or false and is usable in SELECT, WHERE and JOIN. An LLM becomes part of the query plan.

B Cortex Search.

Retrieval over unstructured documents. This is a column in a table, and the requirement is filtering rather than retrieval.

C Cortex Analyst.

Translates natural language into SQL over structured tables. It does not evaluate free text row by row.

D An external sentiment API.

Moves data out of Snowflake, which the scenario rules out, and adds an integration nobody asked for.

ON THE EXAM

Module 6. AI_SENTIMENT would also work. AI_FILTER is the one that puts the decision directly in the WHERE clause.

> A 5 TB table is queried almost exclusively with filters on an order_date column spanning a few days at a time. Query Profile shows partitions scanned close to partitions total.

What is the most appropriate action?

A Enable the Search Optimization Service on order_date.

B Increase the warehouse size so the scans complete faster.

C Create a materialized view filtered to the last 30 days.

D Define a clustering key on order_date.

◆ PAUSE AND DECIDE

What is the most appropriate action?

A Search optimization on order_date.

Built for selective point lookups, substring and semi-structured predicates. These are range filters.

B A bigger warehouse.

Reads the same excess data faster and costs more. It does not improve pruning.

C A materialized view for 30 days.

Answers nothing outside that window, and the queries span arbitrary date ranges.

D A clustering key on order_date.

A date column with range filters on a large table is the textbook clustering case. It fixes the pruning directly.

ON THE EXAM

Modules 3 and 20. Ranges cluster; needles need the access path. That single distinction decides this question.

> A platform team must be able to create users and roles, and to grant privileges on objects they do not own. They must not be able to create warehouses or change account parameters.

Which role should they be granted?

A SYSADMIN

B SECURITYADMIN

C ACCOUNTADMIN

D USERADMIN

◆ PAUSE AND DECIDE

Which role should they be granted?

A SYSADMIN.

Creates warehouses and objects, which is explicitly forbidden, and cannot create users.

B SECURITYADMIN.

Holds MANAGE GRANTS and inherits USERADMIN, so it grants and creates users. It creates no warehouses and sets no account parameters.

C ACCOUNTADMIN.

Would work and grants far more than asked, including account parameters and billing. Never the minimum.

D USERADMIN.

Creates users and roles, and lacks MANAGE GRANTS, so it cannot grant on objects it does not own.

ON THE EXAM

Module 8. Two capabilities required and two forbidden. Only SECURITYADMIN satisfies all four constraints.

> An account-level network policy permits only the corporate range. A security integration used by a partner tool has its own network policy permitting the partner's addresses. A user with no user-level policy connects through that integration from a partner address.

What happens?

A The connection is blocked. The account policy always takes precedence.

B The connection is blocked. Both policies must permit the address.

C The connection is allowed. The security integration policy overrides the account policy.

D The connection is allowed only if the address also appears in the account policy's blocked list.

◆ PAUSE AND DECIDE

What happens?

A Blocked. Account takes precedence.

Inverts the order. The account policy is the most general and therefore the weakest.

B Blocked. Both must permit.

There is no intersection rule. Precedence selects one policy, and the others are not consulted.

C Allowed. The integration policy overrides the account.

Documented precedence: user beats security integration beats account. With no user policy, the integration policy applies.

D Allowed if also in the blocked list.

Nonsensical. The blocked list denies; it never permits.

ON THE EXAM

Module 9. Precedence runs user, then security integration, then account. The most specific applicable policy wins outright.

> Compliance requires that every column holding a national insurance number is masked identically across 300 tables in four databases, including tables created next month. The columns are named inconsistently.

What is the appropriate design?

A Classification to identify the columns, a tag applied by tag map, and a masking policy attached to the tag.

B A masking policy attached to each identified column, with a monthly review.

C A row access policy on each table, filtering out unauthorised roles.

D Secure views over all 300 tables, exposing a masked version of the column.

◆ PAUSE AND DECIDE

What is the appropriate design?

A Classify, tag, attach the policy to the tag.

Classification finds columns by content rather than name, and tag-based masking protects new columns as the tag lands.

B Per-column policy plus monthly review.

Leaves a month-long gap for new tables, and 300 attachments to maintain.

C Row access policies.

Wrong policy type. Row access filters rows; this requirement is about a column value.

D 300 secure views.

300 objects to build and keep in step with schema changes, and analysts lose the base tables.

ON THE EXAM

Module 10. 'Named inconsistently' rules out anything matching on column name, which is why classification belongs in the answer.

> A Large warehouse runs a three-cluster multi-cluster configuration in Maximized mode for eight hours a day.

Roughly how many credits does it consume per day?

A 8 credits

B 64 credits

C 192 credits

D 24 credits

◆ PAUSE AND DECIDE

Roughly how many credits does it consume per day?

A 8 credits.

The hourly rate of a single Large cluster. Ignores both the cluster count and the hours.

B 64 credits.

8 times 8 hours. Correct for one cluster, and Maximized runs all three.

C 192 credits.

8 credits per hour, times 3 clusters, times 8 hours. Maximized runs every cluster the whole time.

D 24 credits.

8 times 3 clusters, for one hour. Ignores the eight-hour day.

ON THE EXAM

Modules 11 and 17. Size times clusters times time. Maximized mode is what makes all three run continuously.

> Files must be loaded from an S3 bucket. Several analysts hold the CREATE STAGE privilege. Security requires that no analyst can see the cloud credentials, and that access can be rotated in one place.

What should be configured?

A. A named internal stage, with files copied in from S3 beforehand.

B. An external stage with credentials supplied inline in CREATE STAGE.

C. A Snowflake SECRET object referenced by the stage.

D. A storage integration created by an administrator, referenced by the external stage.

◆ PAUSE AND DECIDE

What should be configured?

A Internal stage, copy the files in first.

Something must still read the bucket to copy them, needing the same credentials. It also duplicates storage.

B Inline credentials.

Exactly what the requirement forbids. Anyone reading the DDL or running SHOW STAGES could see them.

C A SECRET object.

Secrets serve API integrations and external access, notably Git. Cloud storage uses storage integrations.

D A storage integration.

Created once by an administrator, referenced by many stages, and credentials never appear in a stage definition.

ON THE EXAM

Modules 13 and 16. Four integration types, and picking the wrong one is the commonest error in this area.

> A bulk COPY of 3,000 files fails after twenty minutes with a numeric conversion error on one row. Nothing is loaded. The team wants the load to complete and to review rejected rows afterwards.

What should change?

A Set VALIDATION_MODE = RETURN_ALL_ERRORS on the nightly load.

B Set ON_ERROR = CONTINUE, then review COPY_HISTORY.

C Set FORCE = TRUE so the load ignores previously failed rows.

D Set PURGE = TRUE so failed files are removed and retried.

◆ PAUSE AND DECIDE

What should change?

A VALIDATION_MODE = RETURN_ALL_ERRORS.

Loads no data at all. The table would never be populated, which is worse than the current state.

B ON_ERROR = CONTINUE, then COPY_HISTORY.

Loads every row that parses, skips the rest, and the rejections are reviewable. Exactly the stated requirement.

C FORCE = TRUE.

Governs load metadata and reloading. It has nothing to do with error handling.

D PURGE = TRUE.

Removes files after a successful load. It does not change what happens on error.

ON THE EXAM

Module 14. The default for a bulk COPY is ABORT_STATEMENT, which is why one bad row destroyed 3,000 files of work.

> Files arrive in an S3 bucket at irregular intervals and must be loaded within a minute or two. The team wants no scheduled job and no application changes.

What should be configured?

A. A task on a one-minute schedule running COPY INTO.

B. Snowpipe Streaming, writing rows directly from the source system.

C. Snowpipe with auto-ingest, using a notification integration.

D. Snowpipe with the REST API, called by the application after each upload.

◆ PAUSE AND DECIDE

What should be configured?

A. A task every minute.
A scheduled job, which the scenario rules out, and it burns credits on every empty run.

B. Snowpipe Streaming.
Writes rows from an application. Files already exist here, and it would require application changes.

C. Auto-ingest with a notification integration.
Cloud event notifications trigger the load as files land. No schedule, no application change, about a minute.

D. Snowpipe REST API.
Meets the latency and requires the application to call it, which the scenario forbids.

ON THE EXAM

Module 15. Two constraints, no schedule and no application change, leave exactly one of Snowpipe's two trigger modes.

> A task graph has a root task on an hourly cron schedule and three child tasks chained with AFTER. All four were created successfully. Nothing has run for two days.

What is the most likely cause?

A Tasks are created suspended. The root task was never resumed.

B Child tasks require their own schedules, which were not set.

C The cron expression is missing a time zone, so it never matches.

D SUSPEND_TASK_AFTER_NUM_FAILURES suspended the graph after the first failure.

◆ PAUSE AND DECIDE

What is the most likely cause?

A Created suspended, never resumed.

Tasks are created in a suspended state and must be resumed. Nothing running at all is the signature.

B Child tasks need schedules.

The opposite is true. A child task cannot have a schedule; it uses AFTER.

C Missing time zone.

A cron expression without a time zone still runs, using the account default.

D Auto-suspended after failures.

Requires runs to have failed. Nothing has run at all, so there were no failures to count.

ON THE EXAM

Module 15. 'Created successfully' plus 'nothing has run' is the fingerprint. A failure would leave history; a suspended task leaves none.

> At 09:00 each day, sixty analysts run short dashboard queries against a Medium warehouse. Query Profile shows queries queuing for up to three minutes. No spilling occurs and pruning is good.

What should be done?

A Resize the warehouse to X-Large so each query finishes faster.

B Convert to a multi-cluster warehouse with the Standard scaling policy.

C Enable the Query Acceleration Service on the warehouse.

D Add a clustering key to the dashboard tables.

◆ PAUSE AND DECIDE

What should be done?

A. Resize to X-Large.

Scaling up helps one slow query. These queries are short; they are waiting for a slot, not running slowly.

B. Multi-cluster with Standard policy.

Queuing means concurrency. Standard starts a cluster immediately when a query queues, which suits a short morning peak.

C. Query acceleration.

Accelerates large scans with selective filters. Short dashboard queries are not its eligible shape.

D. A clustering key.

The scenario states pruning is already good. Clustering would change nothing.

ON THE EXAM

Module 17. Queuing means scale out, not up. Economy would be the wrong policy here, since short queries rarely justify a cluster.

> A query joining a 2 TB fact table to a small dimension table takes ninety minutes on a Medium warehouse. Query Profile shows 400 GB spilled to local storage and 90 GB to remote storage, with partitions scanned at 7% of total.

What is the primary problem?

A Inefficient pruning. 7% is too low.

B An exploding join. The dimension table has duplicate keys.

C Concurrency. The warehouse needs more clusters.

D Insufficient memory. Scale up, then optimise the query.

◆ PAUSE AND DECIDE

What is the primary problem?

A Pruning at 7%.

Inverts the ratio. 7% of partitions scanned is excellent pruning; low is good.

B An exploding join.

No row counts are given. The signature of an exploding join is rows out far exceeding rows in.

C Concurrency.

No queued time is mentioned, and one query cannot use two clusters.

D Insufficient memory. Scale up, then optimise.

400 GB local and 90 GB remote spill is the textbook memory signature, and remote spill is the clearest scale-up signal.

ON THE EXAM

Module 19. The pruning figure is the deliberate distractor. It looks like a problem until you notice low means good.

> A VARIANT column named payload contains an array at payload:items. Some rows have an empty items array. A query using LATERAL FLATTEN returns fewer rows than expected, and rows with empty arrays are missing entirely.

What should change?

A Add OUTER => TRUE to the FLATTEN call.

B Add RECURSIVE => TRUE to expand all sub-elements.

C Set MODE => 'ARRAY' so empty arrays are handled.

D Replace LATERAL with TABLE(FLATTEN(...)).

◆ PAUSE AND DECIDE

What should change?

A OUTER => TRUE.

OUTER defaults to FALSE, which omits rows that cannot be expanded. TRUE emits one row with NULLs instead.

B RECURSIVE => TRUE.

Expands sub-elements at deeper levels. It does not affect rows whose array is empty.

C MODE => 'ARRAY'.

MODE defaults to BOTH, which already covers arrays. It restricts rather than enables.

D TABLE(FLATTEN(...)).

For single-level arrays the two forms give the same result. It would change nothing.

ON THE EXAM

Module 21. OUTER is effectively an outer join. With the default, a row whose array is empty vanishes from the results.

> A team wants a function that accepts a product id and returns the full price history for that product, one row per price change, so it can be joined into reporting queries.

What should they build?

A. A scalar UDF returning an ARRAY, flattened by each caller.

B. A stored procedure returning tabular data, called before each query.

C. A UDTF, which returns a set of rows per input.

D. A vectorized Python UDF, which returns batches of rows.

◆ PAUSE AND DECIDE

What should they build?

A Scalar UDF returning an ARRAY.

Works, and pushes a FLATTEN into every calling query. It is a UDTF written the long way round.

B Tabular stored procedure.

Can be called in a FROM clause, and still needs a CALL and cannot be joined per row like a table function.

C A UDTF.

A table function returns a set of rows per input and joins naturally with LATERAL. Exactly the requirement.

D A vectorized Python UDF.

Vectorization processes input in batches for speed. It still returns one value per row.

ON THE EXAM

Module 22. 'One row per price change, joined into queries' is the definition of a table function.

> An engineer drops a permanent table at 14:00 on an Enterprise account where DATA_RETENTION_TIME_IN_DAYS is set to 5. At 16:00 the same day they need it back, with all data intact.

What should they do?

A Contact Snowflake Support to recover it from Fail-safe.

B Recreate the table and reload from the source system.

C Clone the table AT(OFFSET => -7200) to restore it alongside.

D Run UNDROP TABLE, which restores the most recent version.

◆ PAUSE AND DECIDE

What should they do?

A Fail-safe via Support.

Fail-safe applies only after Time Travel expires. Five days of retention covers this comfortably.

B Recreate and reload.

Unnecessary work when the table can be restored in one statement.

C CLONE AT(OFFSET).

You cannot clone a table that no longer exists. Cloning with Time Travel works on existing objects.

D UNDROP TABLE.

Restores a dropped table within the retention window, in one statement, with all data intact.

ON THE EXAM

Module 23. The table was dropped, not damaged. UNDROP is for dropped objects; Time Travel clauses are for existing ones.

> A provider wants to give a consumer access to a filtered subset of a table, plus a stream so the consumer can process changes incrementally, plus a scalar function that formats an identifier.

Which of these can be added to a share?

A All three, provided the provider owns them.

B The filtered subset as a secure view, and the function as a secure UDF. The stream cannot be shared.

C Only the table itself. Views and functions cannot be shared.

D The secure view and the stream. Functions cannot be shared.

◆ PAUSE AND DECIDE

Which of these can be added to a share?

A All three.

Streams cannot be shared at all, whoever owns them.

B Secure view and secure UDF; not the stream.

Secure views and secure UDFs are both shareable. Streams are on the not-shareable list, and consumers make their own.

C Only the table.

Too restrictive. Secure views, secure materialized views, secure UDFs, external and dynamic tables are all shareable.

D View and stream, not the function.

Inverts two of the three. Functions can be shared if secure; streams cannot be shared at all.

ON THE EXAM

Module 24. The documented workaround is to let the consumer create their own stream on the shared objects.

Answer key and domain map

Q	ANSWER	DOMAIN AND TOPIC
1-4	C, A, D, B	D1: metadata cache, editions, temporary tables, SQL API
5-6	A, D	D1: AI_FILTER, clustering for ranges
7-10	B, C, A, C	D2: SECURITYADMIN, policy precedence, tag-based masking, credit arithmetic
11-14	D, B, C, A	D3: storage integration, ON_ERROR, auto-ingest, tasks created suspended
15-18	B, D, A, C	D4: scale out, spilling, FLATTEN OUTER, UDTF
19-20	D, B	D5: UNDROP, what can be shared

ON THE EXAM

Five A, five B, five C, five D. If your wrong answers cluster in one row of this table, that is the domain to revisit.

What your score means

- **17 or more:** comfortable. Use practice exams 2 and 3 to confirm, and revisit module 26 the day before.
- **15 to 16:** on track. Read every explanation, including for the ones you got right.
- **12 to 14:** more revision needed. Find the domain your misses cluster in and rewatch it, not the whole course.
- **Below 12:** go back to the domain drills in modules 7, 12, 16, 22 and 25 before attempting exam 2.
- **A miss you cannot explain** matters more than the total. Read that explanation twice.

> ON THE EXAM
>
> Next: practice exam 2, twenty fresh questions at the same weighting. Leave a day between them if you can.

MODULE 29 · EXAM STRATEGY - ALL DOMAINS

Practice exam 2 – twenty questions

Same 6/4/4/4/2 weighting, twenty fresh questions, and a slightly harder set.

DOMAIN All five WEIGHT 100% MODULE 29 of 31

admin@sthithapragna.com

> An account consumes 300 warehouse credits on Monday and 40 cloud services credits. On Tuesday it consumes 20 warehouse credits and 35 cloud services credits.

On which day is cloud services billed, and why?

A Monday. 40 credits exceeds the flat monthly cloud services allowance.

B Tuesday. The allowance is 10% of that day's warehouse credits, so 2 credits are free and 33 are billed.

C Neither. Cloud services is never billed separately.

D Both. Cloud services is always billed in full.

◆ PAUSE AND DECIDE

On which day is cloud services billed, and why?

A Monday, a flat monthly allowance.

The allowance is daily and proportional, not flat or monthly. Monday's 10% of 300 is 30, and 40 exceeds it by 10.

B Tuesday. 10% of 20 is 2, so 33 are billed.

The allowance is 10% of that day's warehouse credits, calculated daily in UTC. A low-warehouse, high-metadata day is the expensive one.

C Never billed separately.

It is billed above the allowance, which is why the allowance exists.

D Always billed in full.

Ignores the allowance entirely. Most accounts never pay for cloud services.

ON THE EXAM

Module 1. Monday also exceeds its allowance by 10, so the honest answer is 'both, but Tuesday far more'. B states the mechanism correctly.

> A firm on Virtual Private Snowflake wants to publish a free public data product so any Snowflake customer can discover and consume it.

Can they, and why?

A Yes. VPS includes every Business Critical feature plus dedicated infrastructure.

B Yes, but only as a private listing to named accounts.

C No. Public listings require Enterprise Edition specifically.

D No. VPS accounts cannot use the Marketplace or publish public listings.

◆ PAUSE AND DECIDE

Can they, and why?

A VPS includes everything Business Critical has.

The common misconception. VPS is not a strict superset: it gains isolation and loses Marketplace access.

B Private listings only.

Close, and the question asks about a public product discoverable by any customer, which VPS cannot do.

C Public listings need Enterprise.

Sharing and listings are Standard Edition features. Edition is not the barrier for anyone else.

D VPS cannot use the Marketplace or public listings.

Documented. Isolation is the point of VPS, and public distribution is what it gives up.

ON THE EXAM

Module 2. The edition ladder is cumulative everywhere except here. VPS trades reach for isolation.

> SYSTEM$CLUSTERING_INFORMATION on a large table returns an average clustering depth of 1.0 for the defined clustering key.

What does that indicate?

A The table is optimally clustered on that key. Depth 1 means partitions do not overlap.

B Clustering has failed. Depth should be as high as possible.

C The table is empty. Only empty tables return 1.0.

D The clustering key uses too many columns and should be reduced.

◆ PAUSE AND DECIDE

What does that indicate?

A Optimally clustered. Partitions do not overlap.

Depth starts at 1 and smaller is better. A depth of 1 means no overlapping micro-partitions for that key.

B Depth should be high.

Inverted. A large depth means many overlapping partitions, so a query must scan more of them.

C The table is empty.

An empty table returns 0, not 1. The scenario also says the table is large.

D Too many columns in the key.

Depth measures overlap, not key width. Three or four columns is the guidance, and depth does not report it.

THE SIGNAL

Module 3. Depth is 0 for an empty table, 1 at best, and larger is worse. Two of the distractors invert that scale.

> A dynamic table has been running for six months. A team needs to add a column to its defining SELECT and runs ALTER DYNAMIC TABLE to change the query. The statement fails.

Why, and what is required?

A. The target lag must be set to DOWNSTREAM before the query can be altered.

B. Dynamic tables must be suspended before any ALTER statement.

C. A dynamic table's query is immutable. It must be dropped and recreated.

D. Only the owner may alter it, and the team is using a different role.

◆ PAUSE AND DECIDE

Why, and what is required?

A Set TARGET_LAG to DOWNSTREAM first.

DOWNSTREAM ties refresh to consumers. It has nothing to do with changing the query.

B Suspend before altering.

Invented. Suspension does not unlock the query definition.

C The query is immutable. Drop and recreate.

Documented. Other properties such as target lag can be altered; the defining query cannot.

D Only the owner may alter it.

Ownership governs who may alter, not whether the query is alterable at all.

THE SIGNAL

Module 4. Target lag and warehouse can be altered. The query cannot. That asymmetry is what the question tests.

> A team wants Snowflake to execute SQL files held in a private GitHub repository. They have created a secret holding a personal access token.

What else is required, in order?

A A storage integration, then an external stage pointing at the repository.

B An API integration referencing the secret, then a GIT REPOSITORY object.

C A notification integration, then a task that polls the repository.

D Nothing further. A secret alone allows EXECUTE IMMEDIATE FROM a repository URL.

◆ PAUSE AND DECIDE

What else is required, in order?

A Storage integration and external stage.

Storage integrations address cloud object storage. A Git repository is reached over HTTPS.

B API integration, then GIT REPOSITORY.

The documented chain: secret, then an API integration with API_PROVIDER = git_https_api, then the repository object.

C Notification integration and a polling task.

Notification integrations carry cloud messaging events. Nothing here involves messaging.

D The secret is enough.

A secret holds credentials. Something must still define the connection to the endpoint.

THE SIGNAL

Modules 5 and 16. A Git repository behaves as a special kind of stage once created. Getting there takes three objects.

> Business users ask questions of a sales database in plain English using Cortex Analyst. The generated SQL runs, but answers are often subtly wrong: 'active customer' is interpreted inconsistently and revenue excludes refunds.

What is the most likely cause?

A Cortex Analyst is trained on general data and cannot learn business definitions.

B Cortex Search should be used instead, since it understands context better.

C The analysts lack privileges on some tables, so partial data is returned.

D The semantic model is too thin. It must describe business concepts and metric handling, not just the schema.

◆ PAUSE AND DECIDE

What is the most likely cause?

A It cannot learn business definitions.

That is precisely what the semantic model supplies. The capability exists and was underused.

B Use Cortex Search.

Search is unstructured retrieval. Pointing a structured question at it gives an answer that is valid and useless.

C Missing privileges.

Would produce access errors or missing rows, not a wrong interpretation of 'active customer'.

D **The semantic model is too thin.**

A schema cannot express what the business means by active customer, or how refunds affect revenue. That is the semantic model's job.

THE SIGNAL

Module 6. The risk with Analyst is not security, it is quality. SQL that runs cleanly and answers the wrong question.

A future grant of SELECT on future tables exists at the database level for role REPORTING. A different future grant of SELECT on future tables is then created at the schema level for role AUDIT, in one schema of that database. A new table is created in that schema.

Who can select from it?

A Both roles. Future grants at different levels are additive.

B Neither. Conflicting future grants cancel each other.

C AUDIT only. Schema-level future grants take precedence and the database-level grant is ignored.

D REPORTING only. Database-level grants always outrank schema-level ones.

◆ PAUSE AND DECIDE

Who can select from it?

A Both. Additive.

The intuitive answer and not how it works. Precedence selects one level; the other is not applied.

B Neither. They cancel.

Invented. Precedence resolves the conflict rather than voiding both.

C AUDIT only. Schema level wins.

Documented: schema-level future grants take precedence, and the database-level grants are then ignored.

D REPORTING only. Database outranks schema.

Inverted. The more specific level wins, as with network policies.

THE SIGNAL

Module 8. The same 'most specific wins' principle as network policy precedence, applied to a different object.

> A bank on Business Critical has Tri-Secret Secure enabled. An auditor asks what happens to query results if the customer-managed key is revoked in the cloud provider's KMS.

What is the correct answer?

A Snowflake can no longer decrypt the data, and it becomes unreadable including to Snowflake.

B Snowflake falls back to its own key, so queries continue uninterrupted.

C Existing data stays readable; only newly written data is affected.

D Snowflake Support can restore access from Fail-safe within 7 days.

◆ PAUSE AND DECIDE

What is the correct answer?

A Unreadable, including to Snowflake.

The composite master key needs both halves. Revoke yours and the composite cannot be formed. That is the feature, and the risk.

B Falls back to Snowflake's key.

Would defeat the entire purpose. If Snowflake could proceed alone, the customer key would guarantee nothing.

C Only new data affected.

The composite key wraps table master keys for existing data too. Everything becomes undecryptable.

D Fail-safe restores access.

Fail-safe recovers dropped data. It cannot decrypt data whose key half has been revoked.

THE SIGNAL

Module 9. This is the intended behaviour, not a failure mode. It is why the documentation warns so heavily about safeguarding the key.

> Classification has tagged a postcode column. The privacy category assigned is QUASI_IDENTIFIER rather than IDENTIFIER.

What does that distinction mean?

A The column was only partially scanned, so the classification is provisional.

B Quasi-identifiers are not considered sensitive and need no masking.

C The semantic category could not be determined, so a fallback was used.

D A postcode does not identify a person alone, but may do so combined with other attributes.

◆ PAUSE AND DECIDE

What does that distinction mean?

A Provisional because partially scanned.

Classification does sample up to 10,000 rows, and that affects whether a column is found, not which category it gets.

B Not sensitive, no masking needed.

Quasi-identifiers are a re-identification risk in combination. That is the whole reason the category exists.

C A fallback for unknown semantic category.

The two categories are independent. Privacy category is not a fallback for semantic category.

D Not identifying alone, identifying in combination.

The documented meaning. Postcode plus date of birth plus sex is the classic re-identification example.

THE SIGNAL

Module 10. Every classified column gets both a semantic category, what it is, and a privacy category, how dangerous it is.

> A FinOps team builds a dashboard on SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY and wants it to show credit consumption for the current hour so they can react to spikes.

What is wrong with the design?

A WAREHOUSE_METERING_HISTORY does not include credits, only query counts.

B ACCOUNT_USAGE latency of 45 minutes to 3 hours makes current-hour reporting impossible.

C The view requires ORGADMIN, which FinOps will not be granted.

D Nothing is wrong. ACCOUNT_USAGE is the authoritative real-time source.

◆ PAUSE AND DECIDE

What is wrong with the design?

A It has no credits, only query counts.

It reports credits consumed per warehouse per hour. That is its purpose.

B Latency makes current-hour reporting impossible.

ACCOUNT_USAGE views lag 45 minutes to 3 hours. The current hour is never fully present.

C Requires ORGADMIN.

ACCOUNT_USAGE is ACCOUNTADMIN by default and can be granted onward. ORGADMIN is for organization-level tasks.

D It is the real-time source.

It is authoritative and not real-time. Those are different properties, and this pair exists because you cannot have both.

THE SIGNAL

Module 11. Same pair as exam 1, asked from the cost side. Authoritative and current are different properties.

> A nightly COPY loads a single 90 GB compressed file from a stage on a 3X-Large warehouse. It takes six hours. Query Profile shows no spilling and the warehouse largely idle.

What should change?

A Split the file into roughly 100 to 250 MB compressed files so loads can run in parallel.

B Resize to 4X-Large so more threads are available.

C Enable the Query Acceleration Service on the loading warehouse.

D Switch the file format from CSV to Parquet, which loads in parallel automatically.

◆ PAUSE AND DECIDE

What should change?

A Split into 100 to 250 MB files.

Parallel load operations cannot exceed the number of files. One file means one thread, whatever the warehouse size.

B Resize to 4X-Large.

More threads with nothing to do. The warehouse is already largely idle at 3X-Large.

C Query acceleration.

Accelerates scan and filter work on eligible queries. It does not parallelise a single-file load.

D Switch to Parquet.

Format does not create parallelism. One Parquet file is still one file.

THE SIGNAL

Module 13. 'Warehouse largely idle' is the tell. Idle threads mean there is no work to distribute, not that the warehouse is small.

> A partner adds a new column to the middle of their nightly CSV export without notice. The COPY continues to succeed for three weeks before anyone notices that values are landing in the wrong columns.

What prevents this?

A ON_ERROR = ABORT_STATEMENT, so the load fails loudly.

B VALIDATION_MODE on every load, to check files before loading.

C MATCH_BY_COLUMN_NAME, which maps staged columns to target columns by name.

D ENABLE_SCHEMA_EVOLUTION, which adds the new column to the target table.

◆ PAUSE AND DECIDE

What prevents this?

A ABORT_STATEMENT.

Aborts on error, and no error occurred. The shifted values were type-compatible, which is why it succeeded.

B VALIDATION_MODE every load.

Loads no data at all, so the table would never be populated. It also would not flag type-compatible shifts.

C MATCH_BY_COLUMN_NAME.

Maps by name rather than position, so an inserted column cannot shift everything after it.

D ENABLE_SCHEMA_EVOLUTION.

Genuinely useful alongside, and on its own it does not fix positional mapping of the existing columns.

THE SIGNAL

Module 14. This is the failure that raises no error at all, which is why it ran for three weeks.

> A team loads files with both a bulk COPY into TABLE_A and a Snowpipe into TABLE_B, from the same stage. They ask how long Snowflake will remember which files have been loaded, in each case.

What are the retention periods?

A 64 days for both, since load metadata is a table property.

B 64 days for the COPY on the table, and 14 days for the pipe on the PIPE object.

C 14 days for both, since Snowpipe governs all load history.

D 7 days for both, matching the ALTER PIPE REFRESH window.

◆ PAUSE AND DECIDE

What are the retention periods?

A 64 days for both.

Snowpipe does not use the table's load metadata. Its history lives on the pipe.

B 64 on the table, 14 on the pipe.

Correct on both counts, including where each lives. That location is why recreating a pipe wipes its history.

C 14 for both.

Bulk COPY load metadata is 64 days on the target table.

D 7 days for both.

7 days is the ALTER PIPE REFRESH window, a third number in this area and unrelated to load metadata.

THE SIGNAL

Modules 14 and 15. Three numbers live here: 64 days on the table, 14 on the pipe, 7 for REFRESH. Keep them apart.

> A stream sits on a table with DATA_RETENTION_TIME_IN_DAYS set to 1. The consuming task has been suspended for nine days. An engineer wants to know whether the change records are still available.

What is the position?

A Gone. Retention is 1 day, so the stream went stale on day two.

B Gone, but recoverable from Fail-safe via Snowflake Support.

C Available. Retention was extended automatically to 90 days because the account is on Enterprise.

D Probably available. Retention is extended for an unconsumed stream up to MAX_DATA_EXTENSION_TIME_IN_DAYS, 14 by default. Check STALE_AFTER.

◆ PAUSE AND DECIDE

What is the position?

A Gone on day two.

Reasons from the retention setting alone, which is exactly what the extension exists to prevent.

B Recoverable from Fail-safe.

Fail-safe recovers dropped permanent objects. It has no relationship to stream offsets.

C Extended to 90 days on Enterprise.

The extension is bounded by MAX_DATA_EXTENSION_TIME_IN_DAYS, default 14, regardless of edition. 90 days is Time Travel's maximum.

D Probably available. Check STALE_AFTER.

Retention under 14 days is temporarily extended to the stream's offset. Nine days is inside the default bound.

THE SIGNAL

Module 15. Staleness is data loss, so the extension matters. Check STALE_AFTER rather than inferring from the retention setting.

> A monthly board query has returned instantly at no compute cost every month for a year, because the team runs it twice each month-end. This month the finance lead runs it alone, six weeks after the previous run.

What happens, and why?

A It returns instantly. The result cache holds results for a year.

B It fails, because the cached result was purged and the query must be recreated.

C It runs and consumes credits. 24 hours from last use, capped at 31 days from first execution.

D It returns instantly, because monthly queries are exempt from cache expiry.

◆ PAUSE AND DECIDE

What happens, and why?

A Cached for a year.

A year is ACCOUNT_USAGE retention, not the result cache. Two different numbers from two different features.

B It fails.

An expired result never causes a failure. The query simply runs and regenerates it.

C Runs and bills. 24 hours from last use, 31 days maximum.

Six weeks exceeds both limits. The twice-a-month habit was keeping the 24-hour window alive.

D Monthly queries are exempt.

Invented. There is no exemption based on frequency.

THE SIGNAL

Module 18. The scenario explains the previous year's behaviour: two runs within 24 hours meant the second was free.

> Search optimization is enabled on a large events table with ALTER TABLE events ADD SEARCH OPTIMIZATION. Point lookups on the id column improve. Queries filtering payload:device_id, a path inside a VARIANT column, do not.

Why?

A Table-level enablement does not cover semi-structured columns. VARIANT needs an explicit ON EQUALITY clause.

B Search optimization does not support VARIANT columns. Flatten the payload first.

C The search access path is still building for VARIANT columns, which are processed last.

D VARIANT paths must be cast before search optimization applies.

◆ PAUSE AND DECIDE

Why?

A Table-level skips semi-structured. Use ON EQUALITY.

Documented explicitly. VARIANT, OBJECT and ARRAY are not optimised unless named in an ON clause.

B VARIANT is unsupported.

Well supported: equality, IN, ARRAY_CONTAINS, ARRAYS_OVERLAP, SEARCH, substring and NULL checks. It just needs naming.

C Still building, VARIANT last.

Building does take time, and no documented ordering leaves VARIANT until the end.

D Cast the path first.

Matching a VARIANT element against a constant works without an explicit cast.

THE SIGNAL

Module 20. The other columns improving is the clue. The feature works; it was never switched on for the column that matters.

> A report has slowed from minutes to hours. Query Profile shows a Join node at 88% of execution time, with 1.8 million rows entering one side, 3.2 million the other, and 9.1 billion rows leaving. Remote spilling is heavy.

What should be done first?

A Scale up. The remote spilling is the primary problem.

B Add a clustering key to both source tables to reduce the scan.

C Enable the Query Acceleration Service to offload the join.

D Fix the join condition. The row multiplication indicates a missing or too-loose predicate.

◆ PAUSE AND DECIDE

What should be done first?

A Scale up for the spilling.

The spilling is a consequence. 9.1 billion rows exhausts any warehouse, and a bigger one produces them faster.

B Cluster both source tables.

Addresses scanning. The inputs are already small; the problem is what the join does with them.

C Query acceleration.

Accelerates scan and filter work, not joins, and it cannot help a query producing rows that should not exist.

D Fix the join condition.

Rows out exceed rows in roughly two-thousandfold. That is the documented exploding-join signature.

THE SIGNAL

Module 19. The spilling is real and downstream. Fixing the cause removes it; scaling up merely pays more for the same mistake.

A stored procedure must set a session parameter for the caller and then read from a table the caller can access. It is created with EXECUTE AS OWNER and fails when it attempts to set the parameter.

Why?

A Session parameters can only be set by ACCOUNTADMIN, whatever the rights model.

B Owner's rights procedures are not permitted to change session state. Use caller's rights.

C The procedure must be created with a RETURNS clause before it can set parameters.

D Owner's rights is not the default, so the procedure was misconfigured.

◆ PAUSE AND DECIDE

Why?

A Only ACCOUNTADMIN can set them.

Session parameters are routinely set by ordinary roles. The rights model is the constraint here.

B Owner's rights cannot change session state. Use caller's rights.

Documented. Caller's rights procedures can view, set and unset session variables, and those changes persist after the CALL.

C A RETURNS clause is needed first.

Every CREATE PROCEDURE needs a RETURNS clause anyway, so the procedure would not have been created without one.

D Owner's rights is not the default.

It is the default. The scenario also states it was chosen explicitly.

THE SIGNAL

Module 22. Owner's rights buys delegated privilege and gives up session control. Caller's rights is the reverse trade.

> A production database containing tasks and clustered tables is cloned to create a staging environment. A week later the staging team reports that nothing has refreshed and clustering has not been maintained.

What is the explanation?

A Cloned tasks are created suspended, and Automatic Clustering starts suspended on the clone.

B Clones are read-only until ownership is transferred from the cloning role.

C Clones share micro-partitions with the source, so they cannot be reclustered independently.

D The clone inherited no grants, so the tasks lack privileges to run.

◆ PAUSE AND DECIDE

What is the explanation?

A Tasks and clustering both start suspended.

Documented for both. The clustering key is replicated; the maintenance is not resumed.

B Read-only until ownership transfers.

Invented. A clone is fully independent and writable from creation.

C Cannot be reclustered independently.

Sharing partitions is how cloning works, and the clone diverges freely as either object changes.

D No grants, so tasks cannot run.

Cloning a database does propagate child object grants. And a suspended task would not run even with every privilege.

THE SIGNAL

Module 23. Two separate defaults, one symptom each, and both are on the list of things a clone does not inherit.

> A provider shares data with a partner through a reader account. The partner runs increasingly heavy queries. The provider wants to limit the spend without removing access.

What should they do?

A. Ask Snowflake to bill the partner directly for reader account compute.

B. Convert the reader account to a full consumer account, which shifts the cost.

C. Apply a resource monitor to the reader account's warehouse, with a quota and a suspend trigger.

D. Nothing. Reader account compute is uncapped by design.

◆ PAUSE AND DECIDE

What should they do?

A Have Snowflake bill the partner.

The provider creates, owns and pays for a reader account. There is no billing relationship with the partner.

B Convert to a full consumer account.

Not a conversion. The partner would need their own Snowflake account, which is why a reader account was chosen.

C A resource monitor on the reader account.

The provider owns the account, so they can cap its warehouse with a quota and suspend trigger. Access remains.

D Uncapped by design.

The provider owns the account and has the usual controls, including resource monitors.

THE SIGNAL

Modules 11 and 24. The provider pays, and because they own the account they also hold every cost control it has.

Answer key and domain map

Q	ANSWER	DOMAIN AND TOPIC
1-3	B, D, A	D1: cloud services allowance, VPS and Marketplace, clustering depth
4-6	C, B, D	D1: dynamic table immutability, Git chain, semantic model
7-10	C, A, D, B	D2: future grant precedence, Tri-Secret Secure, privacy category, ACCOUNT_USAGE latency
11-14	A, C, B, D	D3: file count and parallelism, MATCH_BY_COLUMN_NAME, 64 vs 14 days, stream extension
15-18	C, A, D, B	D4: 31-day cache ceiling, SOS on VARIANT, exploding join, owner's rights
19-20	A, C	D5: what a clone does not inherit, reader account cost control

ON THE EXAM

Five of each letter again, and no two adjacent questions share an answer. Compare your domain pattern with exam 1.

Comparing the two exams

- **A lower score here than on exam 1 is expected.** This set leans on details rather than headline facts.
- **Compare the domain pattern, not the totals.** A domain that let you down twice is the one to revisit.
- **Questions 1, 7, 13 and 15 all turn on a number.** If those caught you, the numbers are worth a dedicated pass.
- **Questions 11, 12, 17 and 19 are 'nothing is broken' or defaults.** That shape recurs more than any other.

> ON THE EXAM
>
> Next: practice exam 3, the hardest of the three, then module 00 to open the course.

MODULE 30 · EXAM STRATEGY - ALL DOMAINS

Practice exam 3 – twenty questions

The hardest of the three. Twenty more fresh topics, same 6/4/4/4/2 weighting.

DOMAIN All five WEIGHT 100% MODULE 30 of 31

admin@sthithapragna.com

> A warehouse with AUTO_SUSPEND set to 30 seconds serves a job that runs one three-second query every two minutes, for eight hours a day.

What is the most likely effect on cost?

A Minimal. Per-second billing means roughly 24 minutes of compute is billed per day.

B Minimal, because queries under 60 seconds are not billed at all.

C High, because a 30-second auto-suspend keeps the warehouse running continuously.

D High. Each resume bills a 60-second minimum, so about 240 minutes are billed for 12 minutes of work.

◆ PAUSE AND DECIDE

What is the most likely effect on cost?

A Minimal. 24 minutes billed.

Applies per-second billing and ignores the minimum. It would be right if the warehouse never suspended.

B Queries under 60 seconds are free.

Inverts the rule. The 60-second minimum is a floor on billing, not an exemption from it.

C It runs continuously.

A 30-second auto-suspend on a job running every two minutes means it suspends between every run.

D 240 minutes billed for 12 minutes of work.

240 resumes a day, each billing a 60-second minimum. Lengthening auto-suspend is cheaper, not more expensive.

ON THE EXAM

Module 1. The counter-intuitive fix: a longer auto-suspend costs less here, and also keeps the warehouse cache warm.

> A healthcare analytics firm will store patient records subject to HIPAA. They also need customer-managed encryption keys and private connectivity from their VPC.

What is the minimum edition?

A Enterprise, since it includes all governance features.

B Business Critical, which adds PHI support, Tri-Secret Secure and PrivateLink.

C Standard, with masking policies applied to the sensitive columns.

D Virtual Private Snowflake, which is the only edition supporting HIPAA.

◆ PAUSE AND DECIDE

What is the minimum edition?

A Enterprise.

Has masking and classification, and none of the three requirements here. All are Business Critical features.

B Business Critical.

PHI and HIPAA support, Tri-Secret Secure for customer-managed keys, and PrivateLink all arrive together at this tier.

C Standard with masking.

Masking policies are Enterprise, not Standard. And masking does not satisfy a compliance or connectivity requirement.

D VPS only.

Business Critical supports HIPAA. VPS adds dedicated hardware and loses Marketplace access.

ON THE EXAM

Module 2. Three requirements, all landing on the same tier. PHI, customer-managed keys and PrivateLink are the Business Critical trio.

> A 30 TB table has a wide unused text column. A team drops the column expecting storage costs to fall. Three weeks later, with almost no DML on the table, storage is unchanged.

Why?

A Time Travel retains the pre-drop version, and storage will fall when retention expires.

B The column was moved to Fail-safe and will be released after seven days.

C Micro-partitions are immutable. DROP COLUMN is a metadata change, and the data stays until partitions are rewritten.

D Storage is billed on uncompressed logical size, which a dropped column does not change.

◆ PAUSE AND DECIDE

Why?

A Time Travel holds the old version.

Attractive, and Time Travel does cost storage. But even after it expired, the dropped column's data would remain in those partitions.

B It is in Fail-safe.

Fail-safe covers dropped tables and historical versions, not a column inside a live table. Seven days have long passed.

C Immutability. DROP COLUMN is metadata only.

Documented behaviour. Snowflake does not rewrite every partition to remove a column, so the data remains until they are rewritten anyway.

D Billed on logical size.

Storage is billed on compressed size. Logical size is not the billing basis.

ON THE EXAM

Module 3. Immutability cuts both ways: it is why Time Travel works, and why dropping a column reclaims nothing.

> A team needs Snowflake to both read and write Apache Iceberg tables, with full platform support including Time Travel and cloning.

Which configuration is required?

A Iceberg tables with Snowflake as the catalog.

B Iceberg tables using an external catalog such as AWS Glue.

C External tables over the Iceberg data files.

D A catalog integration alone, which enables read-write on any Iceberg table.

◆ PAUSE AND DECIDE

Which configuration is required?

A Snowflake as the catalog.

Snowflake-managed Iceberg tables are read-write with full platform support, including Time Travel and cloning.

B An external catalog.

Externally managed Iceberg tables are read-only from Snowflake. Another engine owns the writes.

C External tables.

Read-only, and a different feature. External tables read files; they are not Iceberg tables.

D A catalog integration alone.

A catalog integration connects to an external catalog, which is the read-only flavour. It does not confer write access.

ON THE EXAM

Module 4. The two Iceberg flavours differ on one thing: who owns the writes. Snowflake as catalog is the read-write one.

> A new analyst cannot connect from a client on the corporate network; the connection times out. Colleagues on the same network connect normally, and the analyst can sign in to Snowsight in a browser.

What is the most appropriate first step?

A Add the analyst's IP to a network policy allowed list.

B Switch the analyst to the SQL API, which avoids proxy problems.

C Reinstall the driver, since automatic upgrades were disabled from SnowSQL 1.3.0.

D Run SnowCD, the connectivity diagnostic tool, to identify which host or port is unreachable.

◆ PAUSE AND DECIDE

What is the most appropriate first step?

A Add the IP to a network policy.

A network policy blocks with an explicit denial, not a timeout. Colleagues on the same network connect fine.

B Switch to the SQL API.

Works around the symptom without diagnosing it, and changes the analyst's tooling permanently.

C Reinstall the driver.

True that upgrades were disabled from 1.3.0, and nothing suggests a version problem. A true statement that answers nothing.

D Run SnowCD.

A diagnostic rather than a way to connect. It reports which hosts and ports are unreachable, which is exactly what is unknown.

ON THE EXAM

Module 5. SnowCD is the odd row in the connectivity table: not a way to connect, but a way to find out why you cannot.

> A security review finds that every user in a new account can call Cortex AI functions. Nobody granted anything, and the security team assumed Cortex was disabled until enabled.

What is the explanation and the remedy?

A Cortex inherits ACCOUNTADMIN by default. Restrict ACCOUNTADMIN membership.

B Someone granted CORTEX_USER to each user directly. Revoke it from the users.

C CORTEX_USER and USE AI FUNCTIONS are granted to PUBLIC by default. Revoke from PUBLIC, then grant to specific roles.

D Cortex is enabled per warehouse. Disable it on all but one warehouse.

◆ PAUSE AND DECIDE

What is the explanation and the remedy?

A It inherits ACCOUNTADMIN.

Invented. Access comes from a database role plus an account privilege, both held via PUBLIC.

B Granted to each user directly.

Database roles cannot be granted to users at all, so this could never have happened.

C Both granted to PUBLIC by default.

PUBLIC is granted automatically to every user and role, so Cortex has been available since the account was created.

D Enabled per warehouse.

Warehouses supply compute. They are not an access control boundary for any feature.

ON THE EXAM

Module 6. 'Nobody granted anything' is the phrase. Nothing was enabled, because it was on from account creation.

> An engineer creates a database and several tables while their session role is ACCOUNTADMIN. A custom role hierarchy exists with its top role granted to SYSADMIN. The data team, using roles beneath SYSADMIN, cannot see the new objects.

Why, and what is the fix?

A ACCOUNTADMIN objects are hidden by design. Drop and recreate them under SYSADMIN.

B Privileges are inherited upward, not downward. Transfer ownership with GRANT OWNERSHIP to a role in the hierarchy.

C The team needs SELECT granted on each table. Ownership does not affect visibility.

D Their sessions must be restarted, since role changes take effect only on reconnect.

◆ PAUSE AND DECIDE

Why, and what is the fix?

A Hidden by design. Drop and recreate.

Not hidden by design, and dropping a populated database is drastic when ownership can be transferred.

B Inheritance is upward. Transfer ownership.

ACCOUNTADMIN is not a superuser. Granting role X to role Y gives Y the privileges, so nothing flows downward.

C Grant SELECT on each table.

Restores reading and leaves ACCOUNTADMIN owning production objects, so the problem recurs with every new table.

D Restart sessions.

Changes nothing. No privilege exists to refresh.

ON THE EXAM

Module 8. Objects are owned by the role active at creation. This is why 'do not make ACCOUNTADMIN a default role' is documented advice.

> An unattended orchestration tool authenticates with a username and password. After a behaviour change bundle, its sign-ins fail while interactive Snowsight sessions are unaffected.

What is the correct fix?

A Create the identity as TYPE = SERVICE and authenticate with key-pair, OAuth, a PAT or workload identity federation.

B Store an MFA push approval in the tool's configuration so it can complete the second factor.

C Grant the tool's user ACCOUNTADMIN, which is exempt from authentication policies.

D Move the tool to SSO, so federated authentication bypasses the requirement.

◆ PAUSE AND DECIDE

What is the correct fix?

A TYPE = SERVICE with key-pair, OAuth, PAT or WIF.

Service users are exempt from MFA enforcement and cannot use passwords. The documented answer for programmatic access.

B Store the MFA approval.

Not a documented pattern, and storing a second factor beside the first defeats the point of having two.

C Grant ACCOUNTADMIN.

Roles govern what you may do after authenticating. They exempt nobody from authenticating.

D Move to SSO.

SSO users are genuinely exempt from the Snowsight enforcement, and SSO is an interactive browser flow an unattended job cannot complete.

ON THE EXAM

Module 9. 'Interactive sessions unaffected' is the clue: MFA enforcement targets password sign-ins by human users.

> A masking policy hashes an email column for the analyst role. An analyst reports that a join between two tables on that email column now returns almost no rows, and a WHERE clause filtering to a known address returns nothing.

What is happening?

A. A row access policy must have been applied at the same time.

B. The masking policy is misconfigured and is filtering rows as well as values.

C. Masking is applied wherever the column is referenced, including JOIN and WHERE predicates.

D. Masked columns cannot be used in joins, so the query should have raised an error.

◆ PAUSE AND DECIDE

What is happening?

A. A row access policy too.
Would explain missing rows, and would not explain a WHERE clause failing on a value known to exist.

B. Misconfigured, filtering rows.
Masking policies act on column values only. Removing rows is what a row access policy does.

C. Masking applies in JOIN and WHERE too.
Documented and deliberate. The join compares hashes to plain text, and the WHERE compares a hash to a literal.

D. Masked columns cannot be joined.
They can be. The join runs and matches nothing, which is why there is no error to help diagnose it.

ON THE EXAM

Module 10. Deliberate design: if masking applied only to output, values could be recovered by filtering for them one at a time.

> A resource monitor's suspend trigger fires at 11:00 on the fifth of a monthly interval. An engineer runs ALTER WAREHOUSE ... RESUME, which appears to succeed, and the warehouse suspends again seconds later.

What will restore service?

A Nothing until the first of next month, when the interval resets.

B Changing the trigger to SUSPEND_IMMEDIATE, which releases the current suspension.

C Resizing the warehouse, which the monitor treats as a new object.

D Increasing the credit quota, raising the suspend threshold, unassigning the warehouse, or dropping the monitor.

◆ PAUSE AND DECIDE

What will restore service?

A Nothing until next month.

The next interval is one of five remedies, not the only one. Twenty-five days of downtime is not the answer.

B Switch to SUSPEND_IMMEDIATE.

Changes how future suspensions behave, and makes them harsher. It does not release the current one.

C Resize the warehouse.

Invented. Resizing does not create a new object, and the monitor assignment persists.

D Raise quota or threshold, unassign, or drop the monitor.

Four of the five documented ways to release a suspension. The fifth is waiting for the next interval.

ON THE EXAM

Module 11. RESUME appearing to succeed then failing again is the fingerprint: the trigger is still active.

> A nightly file set must be loaded into three different tables, with columns cast and reordered for one of them. Four engineers run the pipeline on rotation. An engineer proposes staging the files on the first table's table stage.

What is wrong?

A Table stages serve one table, cannot be granted to other users, and do not support transformation during load.

B Table stages accept only one file per load, so the set must be combined first.

C Nothing, provided each engineer is granted USAGE on the table stage.

D Table stages cannot hold CSV. Only named stages support structured formats.

◆ PAUSE AND DECIDE

What is wrong?

A Three restrictions, all of which apply here.

Multiple tables, multiple users, and transformation during load. Every one is something a table stage cannot do.

B One file per load.

Invented. Any stage type holds many files, and loading many at once is what gives you parallelism.

C Grant USAGE to each engineer.

A table stage is not a database object and has no grantable privileges. There is nothing to grant on.

D Table stages cannot hold CSV.

All stage types hold any supported format. Format is a property of the file and the COPY, not the stage.

ON THE EXAM

Module 13. Every table stage restriction follows from one fact: it is not an object, so there is nothing to grant and nothing to alter.

> A partner requires a single CSV file of about 800 MB with a header row, delivered weekly to an external stage. An engineer runs COPY INTO and finds dozens of small headerless files.

Which options are needed?

A. OVERWRITE = TRUE, which replaces the many files with one combined file.

B. SINGLE = TRUE, HEADER = TRUE, and MAX_FILE_SIZE raised above its 16 MB default.

C. PARTITION BY a constant expression, forcing all rows into one partition.

D. Nothing. Unload to an internal stage instead, which always produces one file with headers.

◆ PAUSE AND DECIDE

Which options are needed?

A OVERWRITE = TRUE.

Controls whether existing files at the target are replaced. It does not combine output into one file.

B SINGLE, HEADER, and a raised MAX_FILE_SIZE.

Three defaults, three symptoms. SINGLE is FALSE, HEADER is FALSE, and 800 MB exceeds the 16 MB default.

C PARTITION BY a constant.

Partitioning splits output into more files, not fewer, and cannot omit the partition column from them.

D Unload to an internal stage.

Stage type does not change file count or headers. The same defaults apply wherever you unload to.

ON THE EXAM

Module 14. Every symptom traces to a default nobody chose, and the 16 MB maximum is the one people never expect.

> A Kafka connector is configured with snowflake.ingestion.method set to SNOWPIPE and snowflake.enable.schematization set to TRUE. Messages arrive correctly, but every target table has only RECORD_CONTENT and RECORD_METADATA columns.

Why?

A ENABLE_SCHEMA_EVOLUTION defaults to TRUE and must be disabled first.

B JSON cannot be schematized. Only Avro carries enough type information.

C The connector needs a storage integration before it can create columns.

D Schema detection and evolution works only with SNOWPIPE_STREAMING, not with file-based Snowpipe.

◆ PAUSE AND DECIDE

Why?

A Disable ENABLE_SCHEMA_EVOLUTION.

Backwards, and it defaults to FALSE. It must be set to TRUE, by a role with OWNERSHIP on the table.

B Only Avro can be schematized.

JSON schematizes perfectly well. The ingestion method is the constraint, not the message format.

C A storage integration is needed.

Storage integrations serve external stages. Nothing here reads cloud object storage.

D It works only with SNOWPIPE_STREAMING.

Documented explicitly. The file-based Snowpipe method does not support schema detection and evolution at all.

ON THE EXAM

Module 16. The files-versus-rows split again. Only the row-based path can shape a table to the data.

> A task is scheduled every five minutes with OVERLAP_POLICY left at its default. One run takes eleven minutes. A team asks what happened to the runs scheduled during it.

What happened?

A. They queued and ran back to back once the long run finished.

B. They ran concurrently, since tasks default to allowing overlap.

C. Those scheduled times were skipped, because the default policy is NO_OVERLAP.

D. The task was auto-suspended after the second missed run.

◆ PAUSE AND DECIDE

What happened?

A They queued and ran back to back.

Under NO_OVERLAP the scheduled time is skipped, not deferred. Nothing accumulates in a queue.

B They ran concurrently.

Inverts the default. Overlap must be permitted explicitly with ALLOW_CHILD_OVERLAP or ALLOW_ALL_OVERLAP.

C Skipped, because NO_OVERLAP is the default.

If a task is still running when the next scheduled time arrives, that scheduled time is skipped.

D Auto-suspended after missed runs.

SUSPEND_TASK_AFTER_NUM_FAILURES counts failures. A skipped run is not a failure.

ON THE EXAM

Module 15. Skipped, not deferred. OVERLAP_POLICY replaces the deprecated ALLOW_OVERLAPPING_EXECUTION, so recognise both names.

> An analytics team reports that the first two or three queries after any idle period take four times longer than usual, then everything is fast. The queries vary, so the result cache does not apply. AUTO_SUSPEND is 60 seconds.

What is happening?

A Suspending drops the warehouse local disk cache. A longer auto-suspend preserves it, at the cost of idle credits.

B Auto-resume takes several minutes, so the first queries wait for the warehouse.

C The metadata cache is rebuilt on resume, which the first queries wait for.

D The 60-second billing minimum throttles the first queries after a resume.

◆ PAUSE AND DECIDE

What is happening?

A The local disk cache is dropped on suspend.

Documented. The next queries re-read from storage until the cache warms again, which is the other half of the auto-suspend trade.

B Auto-resume takes minutes.

Resume is typically a second or two, and it would affect only the first query rather than the first two or three.

C The metadata cache is rebuilt.

The metadata cache lives in cloud services and survives suspension entirely. Only the warehouse cache is lost.

D The 60-second minimum throttles queries.

A billing rule, not a performance one. It affects what you pay, never how fast a query runs.

ON THE EXAM

Module 18. C is the best distractor: a real cache, placed in the wrong layer. Two of the three survive suspension.

> A query combines twelve monthly partitions of a table using UNION between each pair. Rows cannot appear in two months. Query Profile shows UnionAll operators with an Aggregate above them taking 47 percent of execution time. The SQL contains no GROUP BY.

What is that Aggregate doing?

A Computing statistics for the optimiser, which cannot be avoided.

B Duplicate elimination, because UNION deduplicates. Use UNION ALL.

C Populating the result cache, which can be disabled.

D Partition pruning across the twelve scans, which a clustering key would remove.

◆ PAUSE AND DECIDE

What is that Aggregate doing?

A Optimiser statistics.

Statistics come from micro-partition metadata, not from a runtime operator consuming 47% of a query.

B Duplicate elimination. Use UNION ALL.

The documented signature: a UnionAll with an unexpected Aggregate above it. Three characters remove 47% of the work.

C Populating the result cache.

Not an operator in the tree, and disabling it would make repeat runs slower rather than faster.

D Partition pruning.

Pruning happens at the TableScan using metadata, before any data is read. It never appears as an Aggregate.

ON THE EXAM

Module 19. 'No GROUP BY' is the whole clue. An Aggregate you did not ask for is doing work you did not know you requested.

> A team enables the Query Acceleration Service on a warehouse. They then query ACCOUNT_USAGE.QUERY_ACCELERATION_ELIGIBLE expecting to see their accelerated queries, and the view returns nothing.

What does that mean?

A QAS failed to enable. Check ENABLE_QUERY_ACCELERATION on the warehouse.

B The account is not on Enterprise Edition, so the view is empty.

C The view lists only eligible queries not yet accelerated. Once QAS works, they disappear from it.

D The view populates only after the maximum scale factor is set above zero.

◆ PAUSE AND DECIDE

What does that mean?

A QAS failed to enable.

Worth checking, and an empty view is the expected result of QAS working rather than failing.

B Not on Enterprise.

QAS could not have been enabled at all without Enterprise. The scenario says it was.

C It lists only queries not yet accelerated.

Documented. An empty result looks like a problem and is usually success. Latency is also up to 180 minutes.

D Needs a scale factor above zero.

The scale factor caps how much compute may be leased. It does not gate the view.

ON THE EXAM

Module 20. An empty result that means success is unusual enough to be worth remembering. Latency here is up to 180 minutes.

> A pipeline loads device telemetry as JSON into a VARIANT column. Most documents load. A few devices producing very long event histories in a single message fail. The JSON validates correctly and the file format is confirmed.

What is the most likely cause?

A Deeply nested JSON requires ENABLE_SCHEMA_EVOLUTION on the target table.

B PARSE_JSON cannot handle arrays with more than 1,000 elements.

C MATCH_BY_COLUMN_NAME is required, since VARIANT columns match by name.

D The documents exceed the 16 MB VARIANT limit. Split each message into several rows.

◆ PAUSE AND DECIDE

What is the most likely cause?

A Nesting needs schema evolution.

Schema evolution adds columns as sources change. It says nothing about the size or depth of one document.

B A 1,000-element array cap.

Invented. There is no element-count limit, only the overall size limit.

C MATCH_BY_COLUMN_NAME.

Maps staged columns to target columns by name. It does not affect whether a document fits in a VARIANT.

D They exceed the 16 MB VARIANT limit.

VARIANT is capped at 16 MB per row, compressed. Long event histories in one message is how that limit is reached.

ON THE EXAM

Module 21. 'Most load, a few fail, and those few are the largest' is the shape of a size limit rather than a format problem.

> Two retailers want to know how many customers they have in common and how those customers behaved, without either seeing the other's customer records or identifiers.

What fits?

A Each shares a secure view over hashed email addresses with the other.

B A data clean room, where linked data is reachable only through approved templates.

C A reader account for each party, isolating the data from both organisations.

D A listing on the Marketplace, restricted to the two accounts.

◆ PAUSE AND DECIDE

What fits?

A Secure views over hashed emails.

A share grants read access to every row. Hashes can be matched against a known list, so identifiers stay exposed.

B A data clean room.

Data linked in cannot be queried or viewed directly by either party. Join and column policies constrain what may be matched and returned.

C Reader accounts.

For consumers with no Snowflake account. Both parties are customers, and it isolates nothing.

D A private listing.

Still a share, so it grants read access to rows. Restricting who receives it does not stop them reading it.

ON THE EXAM

Module 24. 'Neither may see the other's identifiers' rules out every sharing option, because a share grants read access by definition.

> A company on Standard Edition has replicated a production database to another region for eight months and queries it for reporting. During a regional outage they attempt to promote the secondary so writes can continue. The statement fails.

Why?

A Failover and failback require Business Critical. Database replication alone is Standard and gives a read-only secondary.

B The secondary must be refreshed immediately before promotion, and the primary is unreachable.

C Promotion requires ORGADMIN, which the team was not using.

D Secondary databases can never be promoted. A new primary must be built from a clone.

◆ PAUSE AND DECIDE

Why?

A Failover needs Business Critical.

The documented boundary. Standard replicates a database and gives a read-only secondary; promoting it is a Business Critical capability.

B A refresh is required first.

Good practice before a planned failover, and not what blocks this. An unplanned failover accepts data loss to the last refresh.

C Promotion requires ORGADMIN.

Account administrators manage failover groups. ORGADMIN handles organization-level tasks such as creating accounts.

D Secondaries can never be promoted.

Promotion is exactly what failover is. It simply requires the right edition.

ON THE EXAM

Modules 2 and 25. Eight months of successful replication proves nothing about failover. They sit on opposite sides of the line.

Answer key and domain map

Q	ANSWER	DOMAIN AND TOPIC
1-3	D, B, C	D1: 60-second minimum, Business Critical trio, immutability and DROP COLUMN
4-6	A, D, C	D1: Iceberg flavours, SnowCD, Cortex granted to PUBLIC
7-10	B, A, C, D	D2: ownership and upward inheritance, TYPE = SERVICE, masking in predicates, monitor recovery
11-14	A, B, D, C	D3: table stage restrictions, unload defaults, Kafka schematization, OVERLAP_POLICY
15-18	A, B, C, D	D4: warehouse cache on suspend, UNION without ALL, QAS eligible view, VARIANT 16 MB
19-20	B, A	D5: clean room, failover edition boundary

ON THE EXAM

Five of each letter, no two adjacent questions sharing an answer. Sixty distinct topics across the three exams.

After three exams

- **Sixty distinct topics tested** across the three papers, with no repeats. Your pattern across all three is the real signal.
- **A domain missed in two or three exams** needs the module rewatched. Missed in one is noise.
- **This set was the hardest deliberately.** A lower score here than on exam 1 is expected, not a warning.
- **The day before:** rewatch module 26, the confusion pairs, and skim module 27's six shapes. Nothing new.
- **Confirm the exam format** on Snowflake's own page before booking. Question count, time and fee do change.

> ON THE EXAM
>
> Next: module 00, the introduction. Built last so its module list reads the real runtimes from every manifest.